Secure Asset Guard

Master's in Industrial Electronics and Computers Engineering

Professor:
Adriano Tavares

João Mendes - PG60206

Tiago Oliveira - PG60217

List of contents

1. Problem Statement

In an increasingly volatile world marked by security threats and environmental challenges, institutions and businesses that handle critical assets, such as historical documents, pharmaceuticals, artworks, or scientific equipment, face significant risks from unauthorized access, theft, and degradation due to uncontrolled conditions. Global burglary rates remain a pressing concern, and hundreds of thousands of incidents still occur annually, often targeting valuable storage areas. These threats are exacerbated by inadequate environmental monitoring in traditional storage solutions, leading to irreversible damage from factors such as humidity, temperature fluctuations, or poor air quality, which can result in substantial financial losses. Furthermore, manual inventory management and access logging contribute to inefficiencies and errors, with small enterprises often lacking integrated systems

The SecureAssetGuard project addresses these interconnected issues by developing an embedded system that integrates multi-sensor perimeter intrusion detection with visual and audible responses, a multi-layered security system, and an intelligent vault featuring environmental monitoring with automated response mechanisms, as well as integrated inventory tracking. This unified platform offers strong protection against breaches, maintains ideal storage conditions to prevent asset degradation, and provides real-time insights through a user-friendly interface, delivering a cost-effective and scalable solution for critical asset management in environments like archives, laboratories, and small data centers.

2. Requirements and Constraints

In all projects, a list of requirements and constraints is mandatory; these can be divided into two major groups: Functional and Non-Functional. On the other hand, constraints can be divided into Technical and Non-Technical.

2.1 Requirements

2.1.1 Functional

- Control access to the interior of the room and the vault
- Audible and visual response to intruders or unauthorized access attempts
- Manage vault inventory and monitor its conditions (e.g., temperature and humidity)
- User interface with logs and real-time data of access to the room, vault, inventory, and its conditions

2.1.2 Non-Functional

• Provide a friendly user interface
• Secure

2.2 Constraints

2.2.1 Technical

• Use the Raspberry Pi Platform
• Use an embedded Linux image generated using Buildroot
• Software must be written in C++

2.2.2 Non-Technical

• Small Team (Only 2 people)
• Project Deadline
• Limited Budget

3. System Overview

With the previously defined requirements and constraints in mind, a high-level overview of the integrated system was created for the Secure Asset Guard. The core of the system is the **Control unit**, which contains the Raspberry Pi, which coordinates the interaction between the various subsystems:

- **Movement detection:** Detect unauthorized presence around the protected area.
- **Access control system:** Manages authentication for room and vault entry.
- **Inventory management module:** Tracks valuable assets stored inside the vault and monitors their conditions.
- **Environmental monitoring:** Measures temperature, humidity, and other parameters within the vault to prevent asset degradation.
- **Historical database:** Records all access attempts, alerts, and environmental data for auditing.
- **User interface:** Provides a user-friendly interface giving real-time information and alerts.

This modular design ensures efficient protection, management, and monitoring of critical assets with clear separation of concerns.

The following figure offers a high-level illustration of the system, showing how the sensors, security system, database, and application work together to ensure effective protection, monitoring, and management of valuable assets.

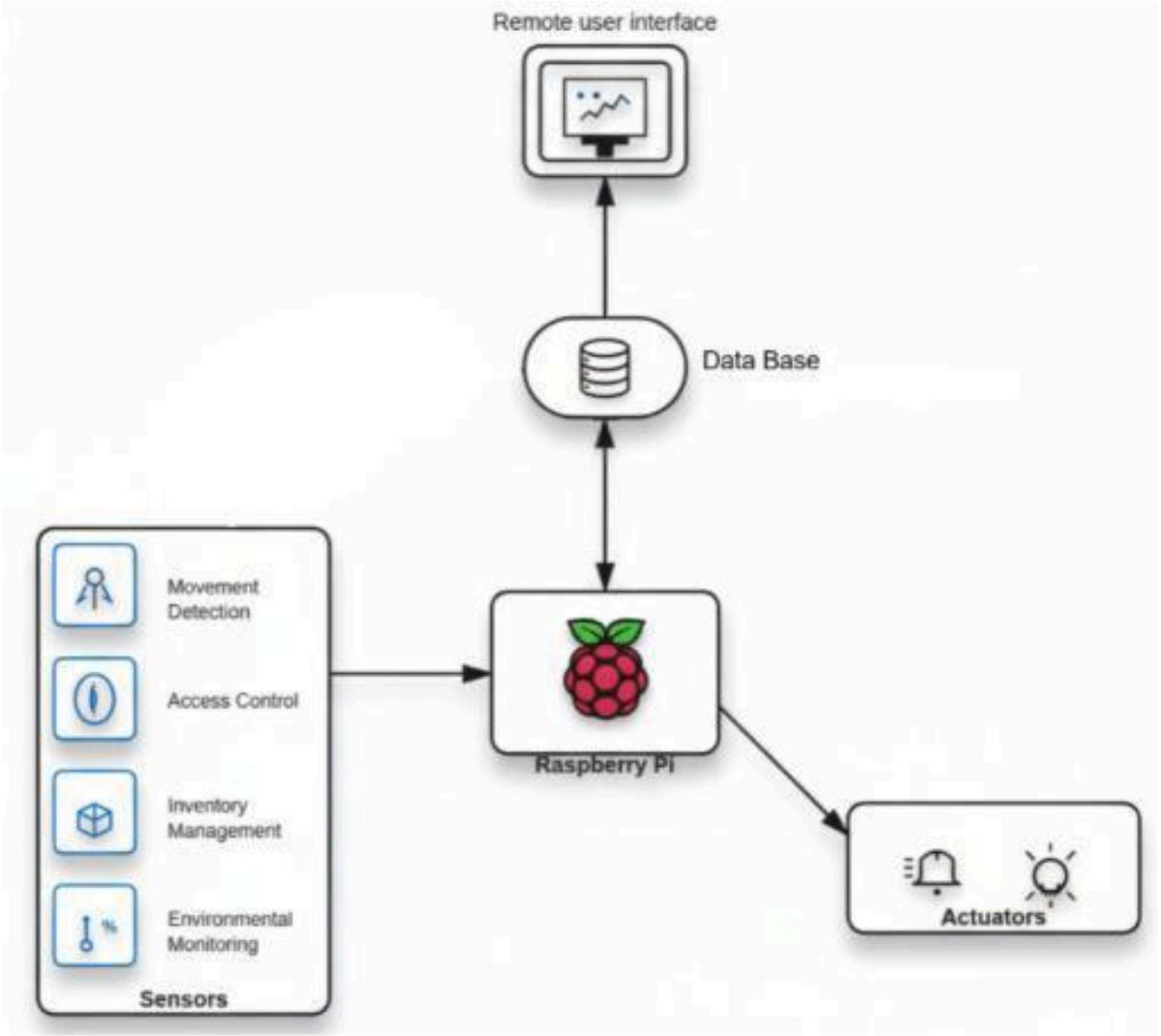


Figure 1. System Overview

4. Gantt Diagram

Task Name	Start	End	Duration
Project Plan	2025-09-19	2025-09-26	6 days
Problem statement	2025-09-19	2025-09-25	5 days
Requirements and Constraints	2025-09-22	2025-09-26	5 days
Analysis Phase	2025-09-29	2025-10-06	6 days
Market Study	2025-09-29	2025-10-01	3 days
System Architecture	2025-10-02	2025-10-06	3 days
Design Phase	2025-10-07	2025-10-20	10 days
Hardware component specification	2025-10-07	2025-10-13	5 days
Hardware interface specification	2025-10-07	2025-10-13	5 days
Software component specification	2025-10-13	2025-10-17	5 days
Software interface specification	2025-10-13	2025-10-17	5 days
Start-up/Shutdown process specification	2025-10-17	2025-10-20	2 days
Errors handling specification	2025-10-17	2025-10-20	2 days
Implementation	2025-10-21	2026-01-05	55 days
Design verification	2025-10-21	2025-10-24	4 days
Hardware implementation	2025-10-24	2025-12-24	44 days
Software implementation	2025-10-24	2025-12-24	44 days
Verification	2025-12-26	2026-01-05	7 days

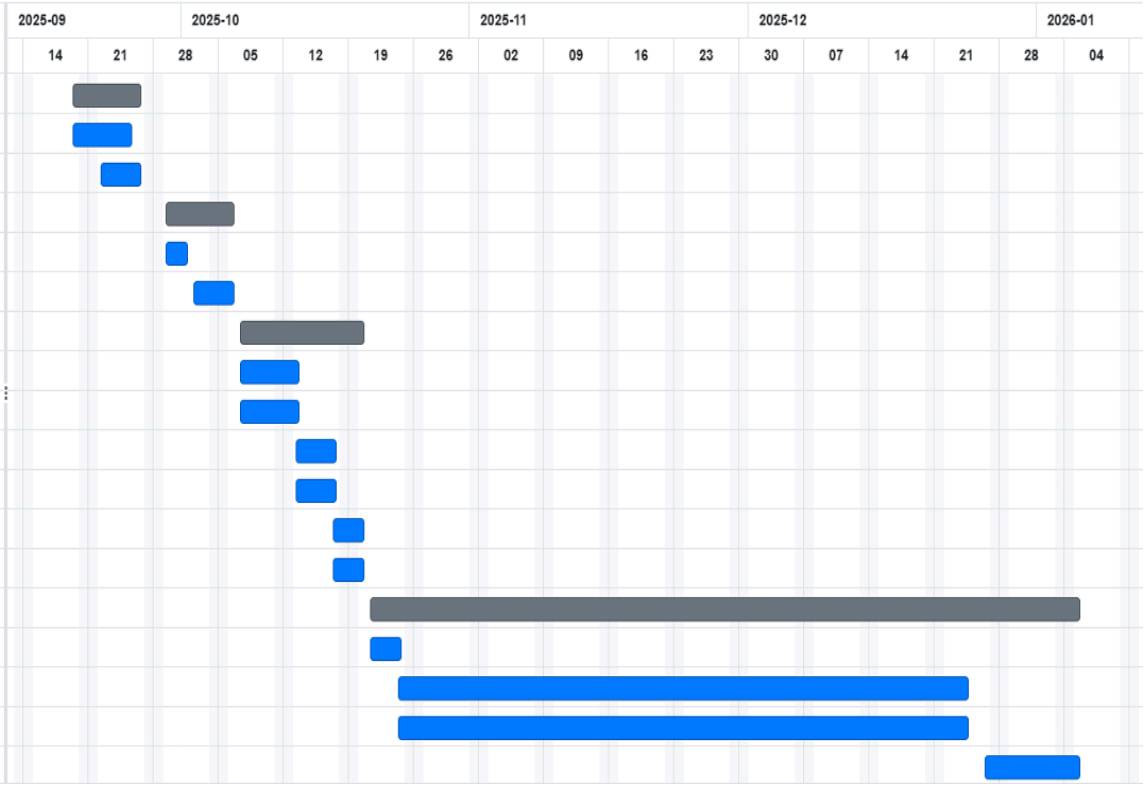


Figure 2. Gantt Diagram

5. Analysis

5.1. Market Study

The global markets for physical security, integrated access control, and environmental monitoring are growing rapidly, driven by increased demand for asset protection and precise control of sensitive environments. In 2024, the physical security market was valued at about USD 131.6 billion and is expected to reach USD 220.4 billion by 2033, with a compound annual growth rate (CAGR) of 5.6%. The integrated access control segment is growing even faster, predicted to surpass USD 106 billion by 2032 with a CAGR of 10.3%. Meanwhile, environmental monitoring systems—essential for maintaining vault conditions—are forecasted to grow to USD 40.6 billion by 2033 at a 6% annual rate. This strong growth highlights significant opportunities for multifunctional solutions that combine access control, environmental monitoring, and digital asset management, especially for small and medium-sized enterprises (SMEs).

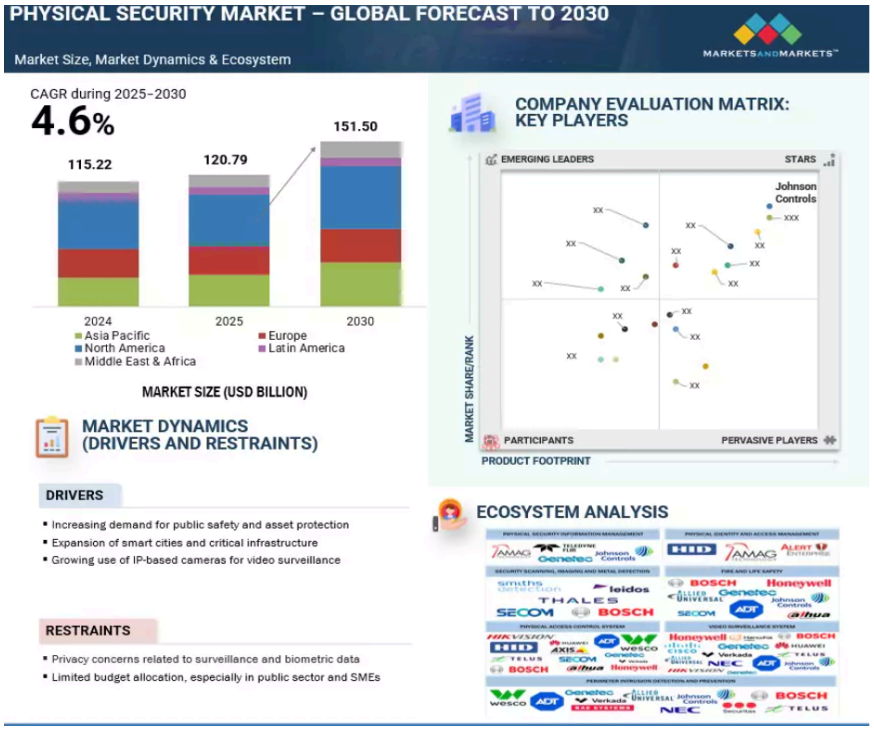


Figure 3. Physical Security Market Overview

5.1.1. Similar products

Several commercially available products address individual aspects of security and monitoring, such as environmental sensing or access control. However, they lack the full integration offered by Secure Asset Guard. Key examples include:

- **SECURAM Safe Monitor** - An intelligent device that monitors temperature, humidity, vibration, and door status inside safes, providing real-time alerts via mobile applications. Price range: USD 160-300.

Figure 4. SECURAM Safe Monitor.

- **Godrej NX PRO Luxe Digital & Biometric Safe (48 Liters) -** A biometric and digital lock safe tailored for homes and small businesses, featuring robust physical security with fingerprint authentication. Approximate price: USD 1,200.

Figure 5. Godrej NX PRO Luxe Digital & Biometric Safe (48 Liters).

- **SecuraCrib** - An automated secure storage system that transforms existing spaces into controlled inventory storerooms. It includes motion-sensing cameras (4–32), access control via keypad, badge/FOB reader, or facial recognition, and 24/7 transaction monitoring with real-time reporting. SecuraCrib supports unlimited SKU capacity without size or shape limitations, industrial-grade equipment with a three-year warranty, and customizable add-ons like RFID integration. Price available upon request.

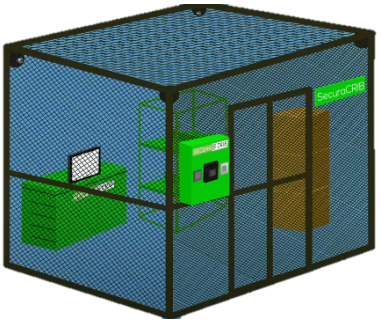


Figure 6. SecuraCrib.

These products incorporate discrete functions, such as environmental monitoring, biometric access, and IoT-based sensing, but fall short of providing a fully unified solution.

5.1.2. Target Market

Secure Asset Guard is primarily designed for SMEs and institutions that require multifaceted protection for physical assets and controlled environments, including:

- Financial institutions with vault security needs.
- Healthcare facilities maintaining environmental controls for pharmaceuticals and samples.
- Data centers and offices demanding advanced access control and environment monitoring.
- Museums and cultural institutions preserving sensitive items.

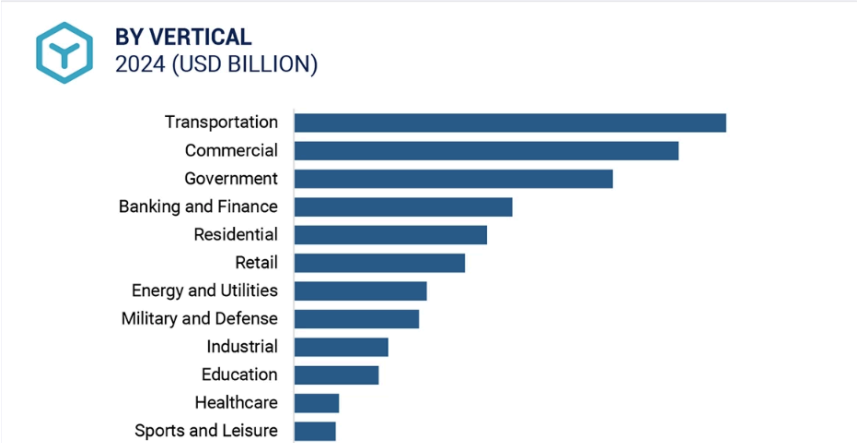


Figure 7. Revenue by Industry Segment.

This market segment bridges the gap between basic consumer systems and overly complex enterprise solutions, prioritizing effective yet accessible security.

5.1.3. Why choose our product?

Secure Asset Guard stands out by delivering a comprehensive, integrated solution that addresses the limitations of existing products:

- Multi-level secure access control with robust authentication mechanisms.
- Automated environmental monitoring for conditions inside safes or secured rooms.
- Digital asset management and real-time event tracking through a unified remote application.
- A cost-effective option tailored for SMEs, balancing security, efficiency, and simplicity without the overhead of enterprise systems.

This holistic approach enhances security, streamlines operations, and provides greater peace of mind for users managing high-value assets and sensitive environments

5.2. System architecture

System architecture defines the core structure and operational flow of the system, outlining its key components and how they work together to achieve the system's goals with optimal performance. This structure is divided into two main areas: Hardware and Software, both essential to the system's overall function.

5.2.1. Hardware architecture

The hardware architecture forms the physical foundation of the Secure Asset Guard system, defining the arrangement and interconnection of its core components to ensure robust monitoring, access control, and environmental management of secure rooms and vaults. This architecture is centered around a central control unit, with supporting sensors and actuators distributed to meet the system's security and operational goals. The primary components are organized as follows:

- **Grid Power**: To power the Control Box, power is received from the grid and transformed to appropriate voltage levels (e.g., 5V for Raspberry Pi, 3.3V for sensors) to supply all components efficiently.
- **Raspberry Pi**: The Raspberry Pi serves as the central control unit, processing sensor data, managing actuator operations, and coordinating security events such as access control and alerts.
- **Buzzer and LED**: The buzzer and LED are used to alert users about intruders, errors, or critical conditions, activated via GPIO pins on the Raspberry Pi.
- **RFID Readers**: The RFID Reader supports dual purposes—access control for room entry and inventory tracking within the vault.
- **Fingerprint Sensor**: The fingerprint sensor provides an additional layer of access control for vault entry, interfacing with the Pi.
- **Servo Motor**: A servo motor controls the opening and closing of the secure room door and vault, driven by PWM signals from the Raspberry Pi .
- **Temperature and Humidity Sensor**: This sensor monitors vault conditions to ensure optimal preservation.
- **Reed Switch Sensor**: The reed switch detects the open/closed status of the secure room door and vault.
- **PIR Sensor**: The PIR sensor detects movement inside the secure room, triggering alerts when unauthorized activity is sensed.

- **Fan**: The fan actuates to regulate temperature inside the vault, activating when the temperature exceeds a predefined threshold (e.g., 30°C).

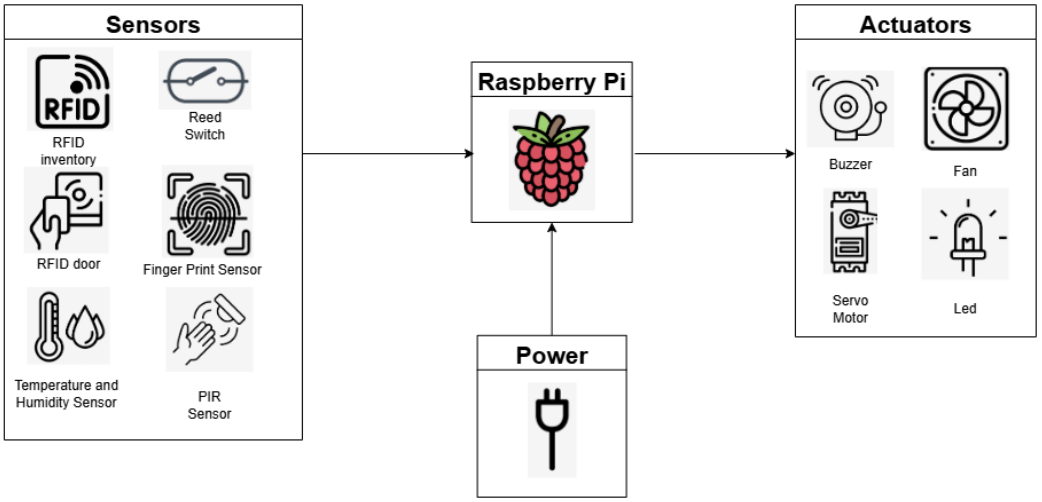


Figure 8. Hardware architecture.

5.2.2. Software architecture

The software architecture serves as a high-level depiction of how the structural organization of the software system is arranged, showing how its individual components or modules communicate with one another.

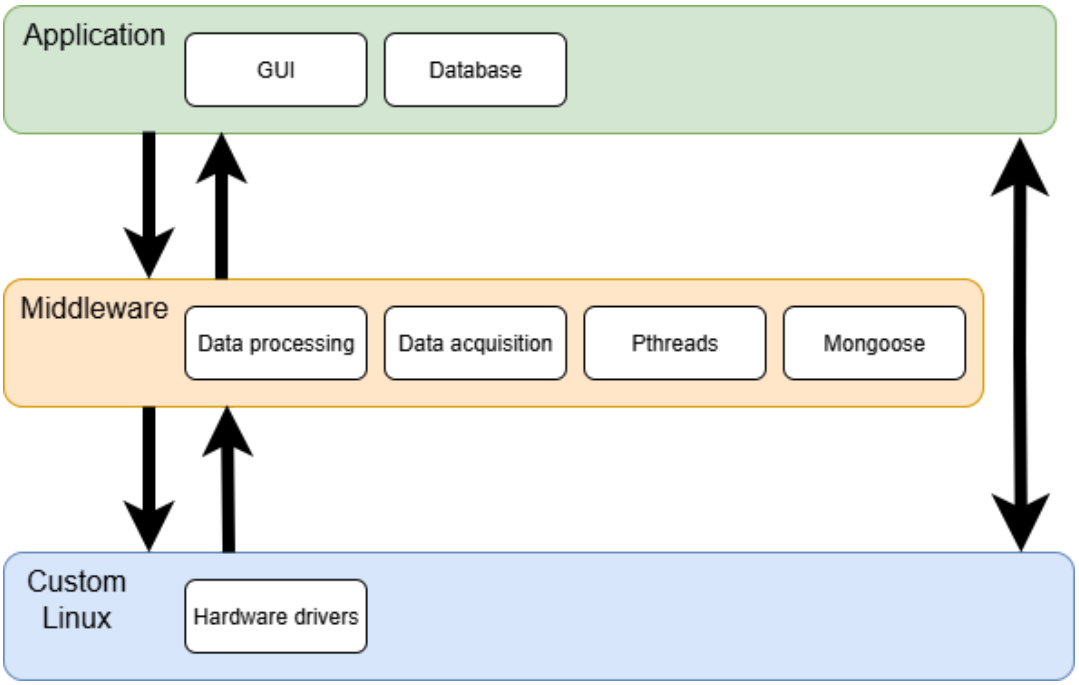


Figure 9. System software architecture.

In our system, this architecture is divided into three different layers:

- **Application layer:**
  - GUI: Provides the interface between the user and the lower layers of the system.
  - Database: Stores important data that will be displayed at the user interface.
- **Middleware Laye**r:
  - Data processing: Pre-processing is needed for the data before going into the application layer.
  - Data acquisition: Raw data is collected directly from sensors or hardware devices.
  - Pthreads: Enables multitasking by allowing multiple operations to run concurrently and efficiently within the system.
  - Mongoose: Acts as a bridge between the web interface and the database, managing all HTTP communication.
- **Custom Linux OS Layer**:
  - Hardware Drivers: Enable upper layers to utilize system hardware as sensors and actuators; device drivers are required.

5.3. Technology Stack

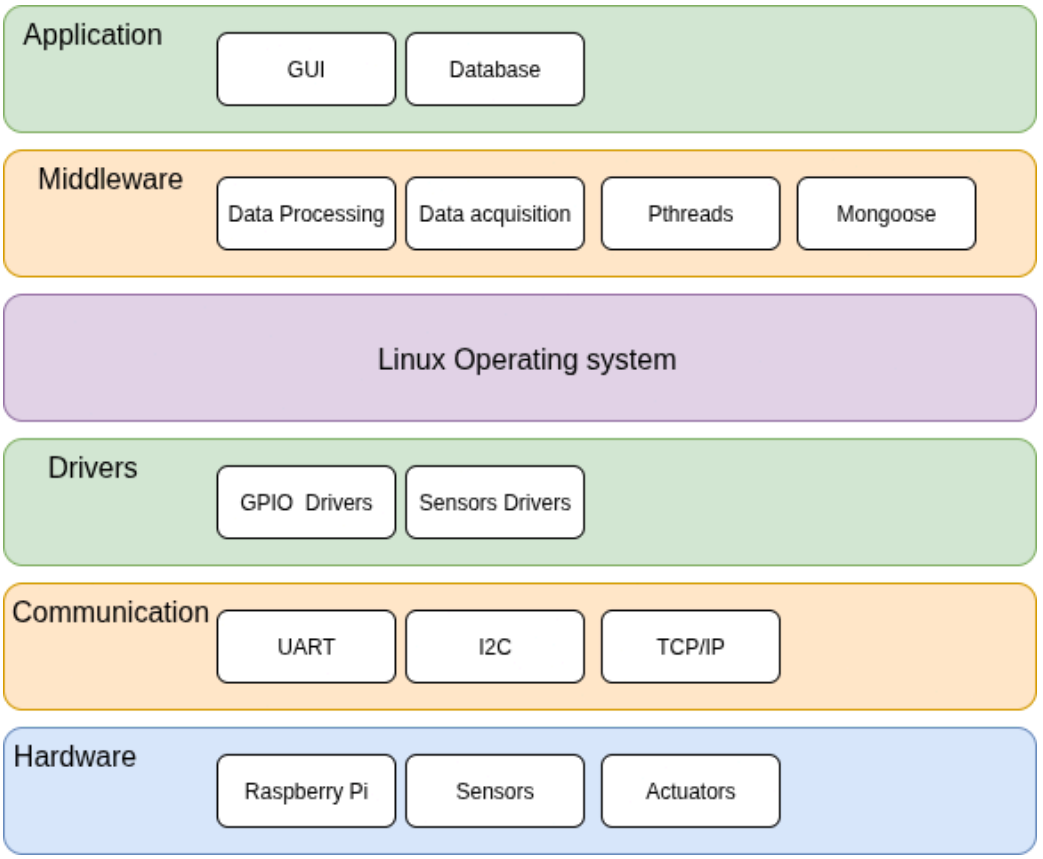


Figure 10. Technology stack.

Above, in Figure 10, the technology stack of the Secure Asset Guard can be found.

At the bottom layer, we have the hardware composed by the Raspberry Pi, sensors, and actuators. Above this, the communication protocols including UART, I2C, and TCP/IP facilitate data exchange between hardware components. Following that, the drivers layer provides interfaces such as GPIO drivers and sensors drivers to interact with the underlying hardware. On top of this, the Linux operating system runs a custom embedded image generated using Buildroot, ensuring efficient resource management and system stability.

At the middleware level, we can find data processing and data acquisition modules for handling sensor inputs and environmental monitoring, alongside Pthreads for multithreading support and Mongoose for embedded web server functionality, all integrated with custom components written in C++. Finally, at the application level, the GUI provides a user-friendly interface for real-time monitoring and control, while the database manages logs, inventory tracking, and historical data for access attempts, alerts, and vault conditions.

5.4. System Analysis

Once the system is clearly defined, it's crucial to examine in greater detail how its various components interact with one another and how the system as a whole engages with its environment.

5.4.1. System Events

A critical aspect of defining a system's behavior is identifying the events it will encounter. To accomplish this, a list of events was established.

Table 1. System Events

Event	System Response	Source/Trigger	Type
Power On	Initialize System	User	Async
Read Sensors	Periodic acquisition of sensor data (temperature, humidity, and RFID)	System (Timer)	Sync
Fingerprint scan to access the vault	Biometric authentication	User	Async
Movement detection Inside the room	Intruder verification	User	Async
Intrusion Warning	Activates buzzer and visual alert on movement	PIR sensor/System	Async
Unauthorized Access Alert	Alert when invalid access is detected	System	Async
Temperature Threshold Exceeded	Activate the fan	System/ temperature sensor	Async
Servo Control Command	Lock/unlock door mechanism	System	Sync
Door Opened	Log access and start timer	Reed Switch	Async
Door Closed	Lock the door and update the state	Reed Switch	Async
Safe Door Closed	Initiate RFID inventory scan	Reed Switch	Async
Save Data	Store the sensor values in the database	System	Sync
Update Application	Change the content presented on the App	System	Async
Modify Users or Inventory Details	User changes access or inventory info via app	User	Async

5.5. Use case diagrams

To better visualize how different actors interact with the system, a use cases diagram can be of great help. In this section, a use cases diagram for the Secure Asset Guard System will be depicted.

5.5.1. Main System

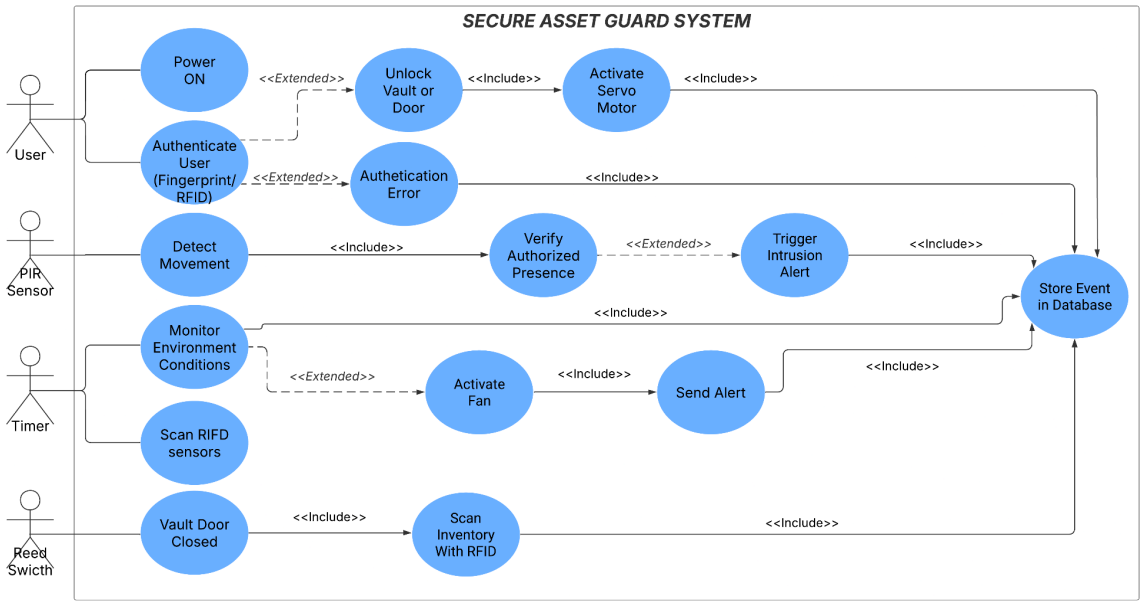


Figure 11. Use case diagram of the system.

In Figure 11, it is possible to find the use cases of the Secure Asset Guard System. For this system, the main actors are the User, the PIR Sensor, the Timer, and the Reed Switch. The User will be able to power on the system or authenticate via fingerprint or RFID, which may lead to handling authentication errors if verification fails. Upon successful authentication, the user can unlock the vault or door, including activation of the servo motor to facilitate access. Another actor is the PIR Sensor, which detects movement in the monitored area, extending to verify authorized presence and potentially triggering an intruder alert if unauthorized activity is identified, after which events are stored in the database. The system also continuously monitors environmental conditions, which can extend to activating the fan or sending alerts as needed. The Timer triggers periodic scanning of RFID sensors to maintain security and inventory checks. Finally, the Reed Switch detects when the vault door is closed, including scanning inventory with RFID to ensure asset integrity. When faults occur, such as unauthorized access or environmental issues, the system attempts to recover by sending alerts or suspending operations until resolved.

5.5.2. Interface

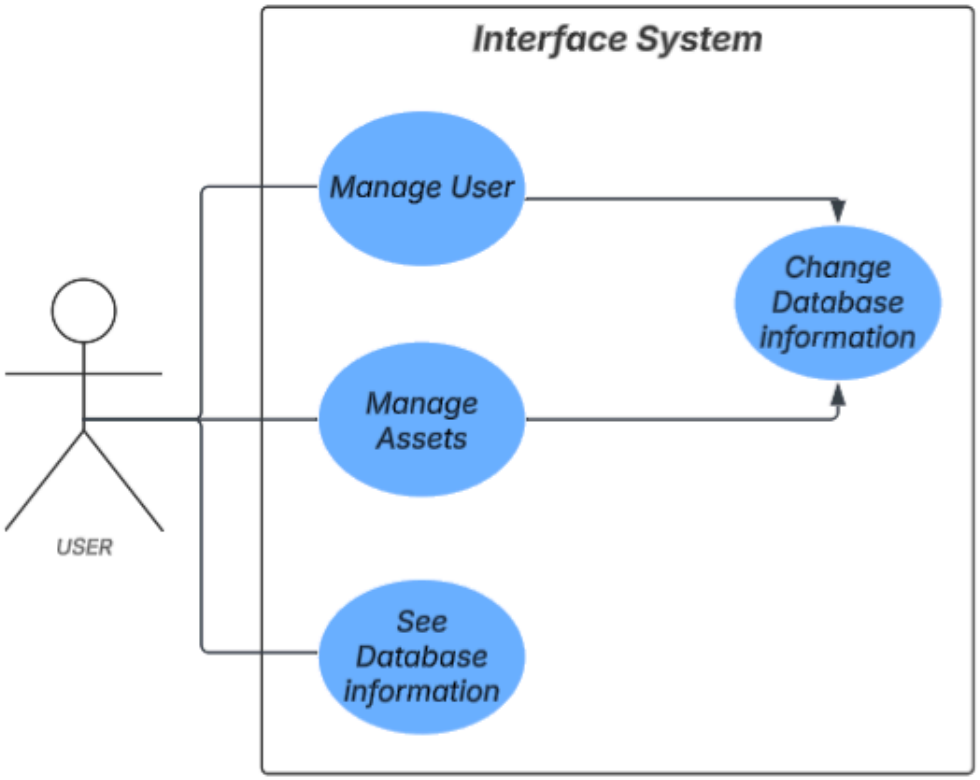


Figure 12. Use Case Diagram for the interface.

In Figure 12, it is possible to find the use cases of the Interface System. For the Interface System, the main actor is the User. The User will be able to manage users, which extends to changing database information when necessary. Besides this, the User can manage assets and see database information to monitor or review stored data.

5.6. Entity Relationship Diagram Overview

The entity relationship diagram provides a visual representation of the main entities in the database and illustrates how they are structured and connected to organize the stored data. The database is divided into six distinct blocks, all of which are related. The main system has a one-to-many relationship with each of the other blocks.

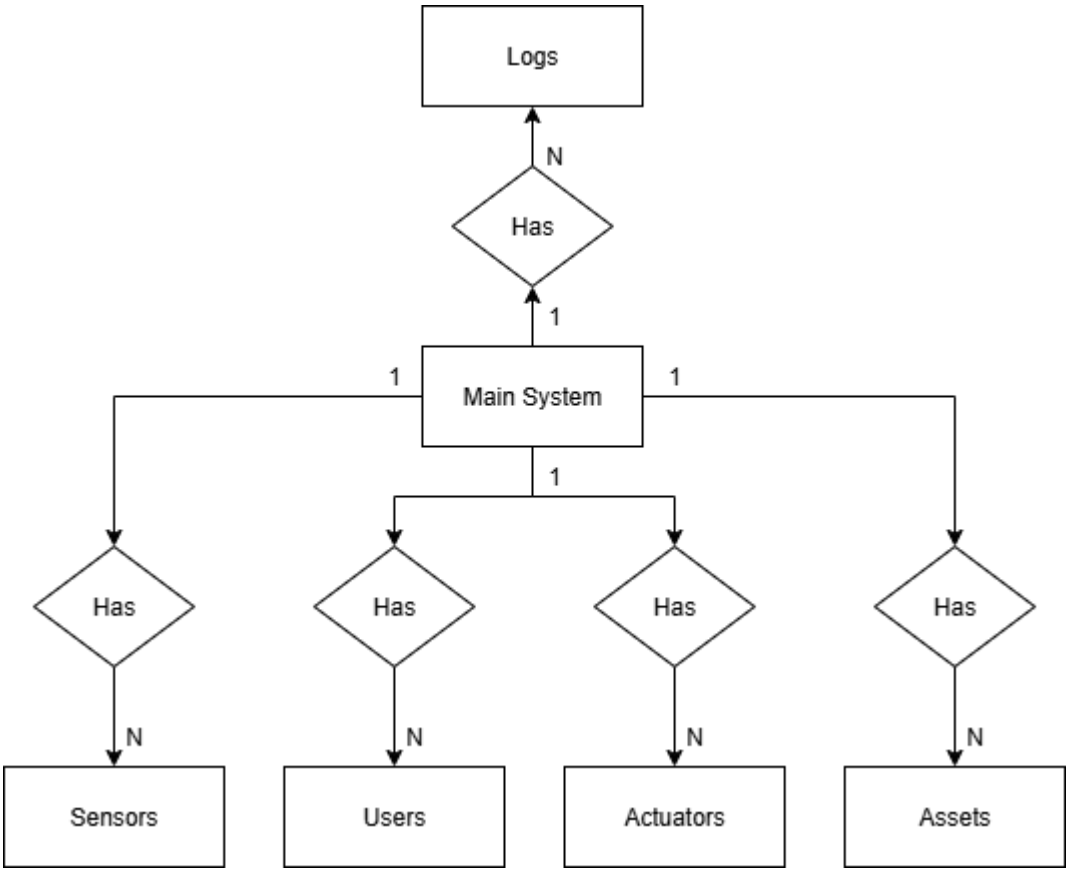


Figure 13. Entity relationship diagram.

5.7. State chart

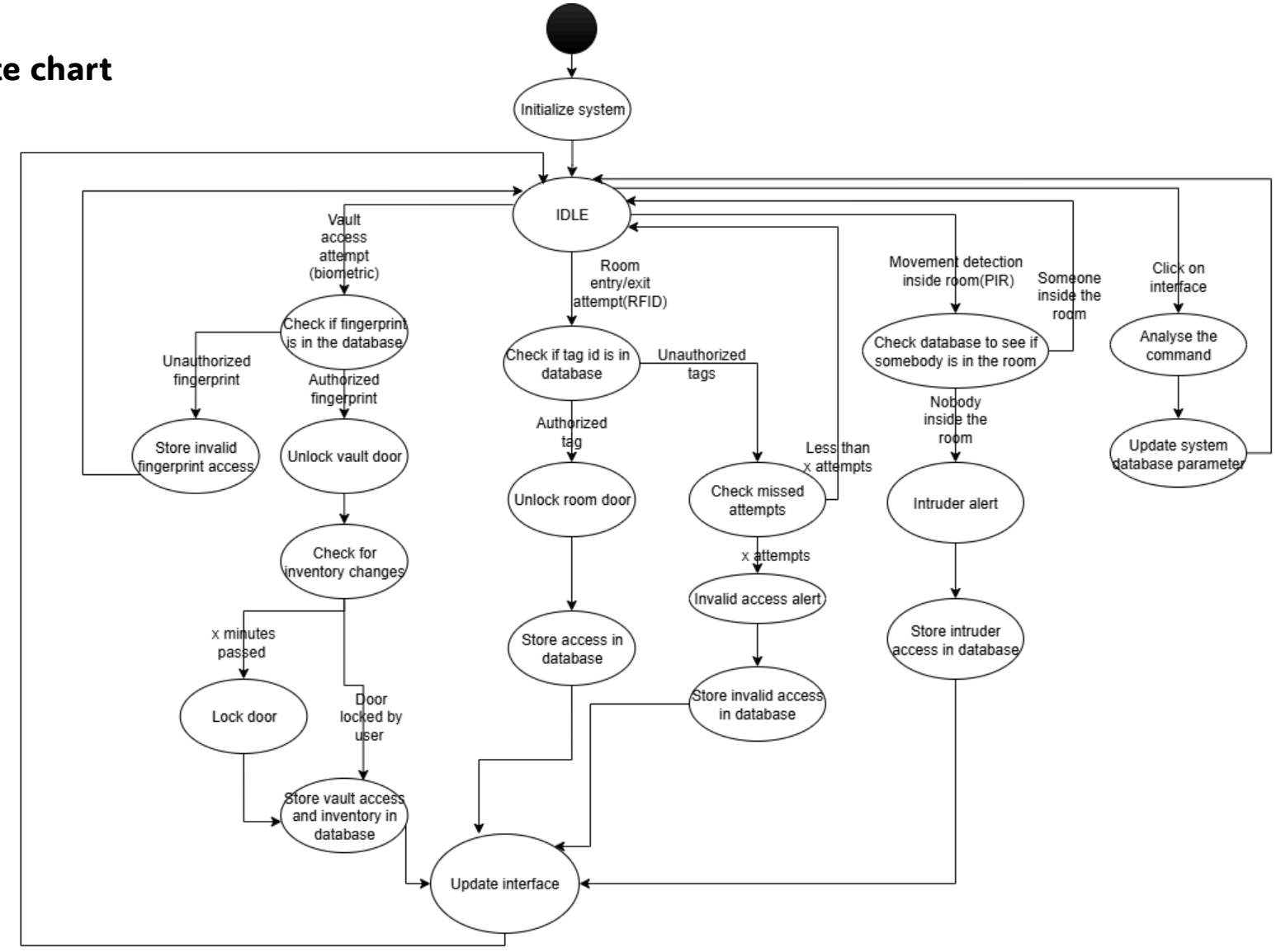


Figure 14. System state chart

A state machine diagram visually models the system's behavior and state transitions in response to different events and inputs. In our system, after power up we have to initialize it to make sure everything works properly. After the system enters the state IDLE, there are five different paths:

- If there is someone trying to enter or exit the room, the RFID tag is checked and searched for in the database to decide whether the room remains locked or is unlocked. If we try to unlock it with an unauthorized tag for more than a certain number of attempts, there is an invalid access alert. Both valid and invalid accesses are stored in the database and can be checked in the user interface.
- If there is movement detection inside the room it is verified the presence of somebody in the room is verified using the access data stored in the database. If there is no access and there is movement, there is an intruder alert that will be stored in the database and can be checked in the user interface.
- If the signal to measure metrics is triggered (periodic trigger), the system acquires and analyzes the data from the temperature and humidity sensors. If the temperature reaches the threshold value, the fan will be activated or deactivated. Both temperature and humidity are stored in the database and can be checked in the user interface.
- If there is someone trying to open the vault, the fingerprint will be looked for in the database to decide whether the vault is opened or not. Both invalid and valid accesses are stored in the database and can be checked in the user interface. After a valid access, the system checks for inventory changes over a set period, then locks the door and stores the vault access details along with inventory updates in the database.
- If there is a click on the screen, it will be analyzed, and the user can change parameters such as the IDs who have access to the room, the fingerprints that have access to the vault, and change inventory information. The user can also check all of the available data.

5.8. Sequence Diagram

The sequence diagram provides a way to visualize how different events occur in time and how they influence the response of the system.

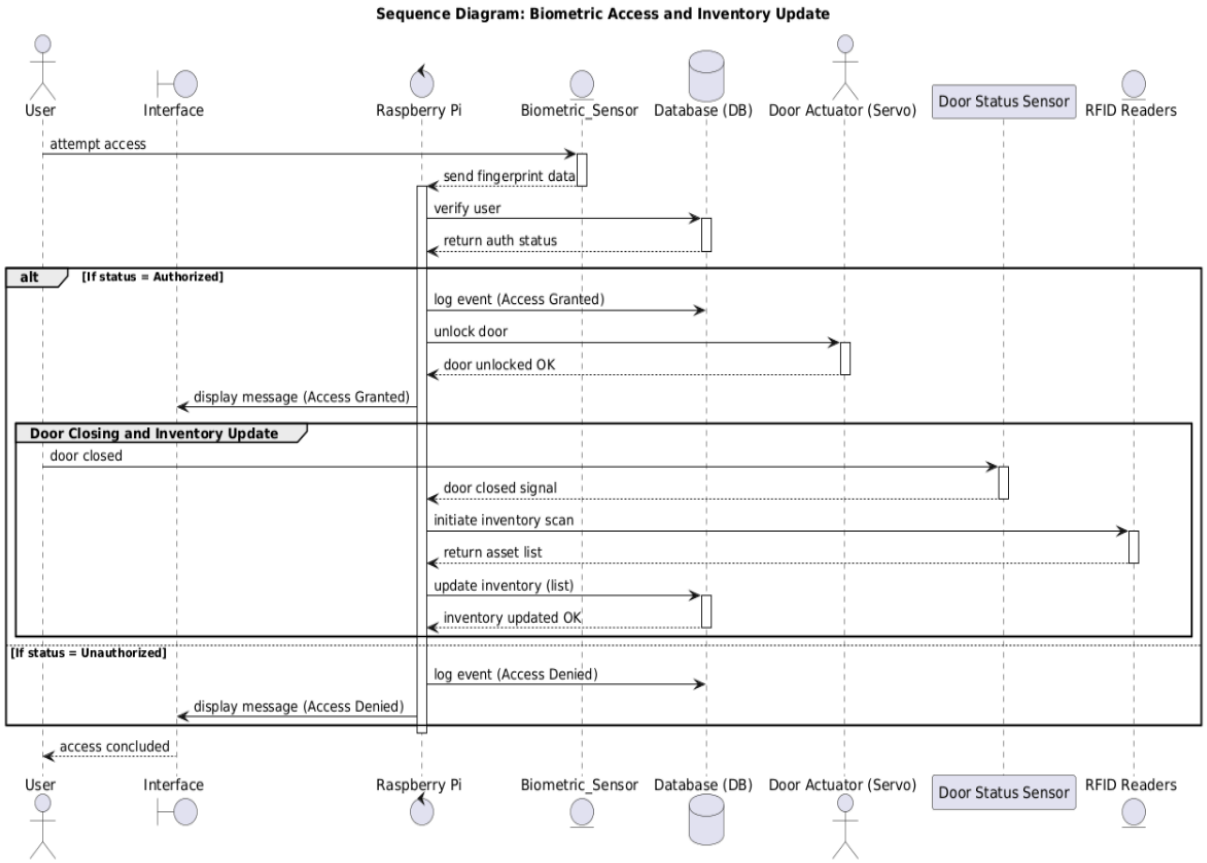


Figure 15. Sequence diagram of biometric access and inventory.

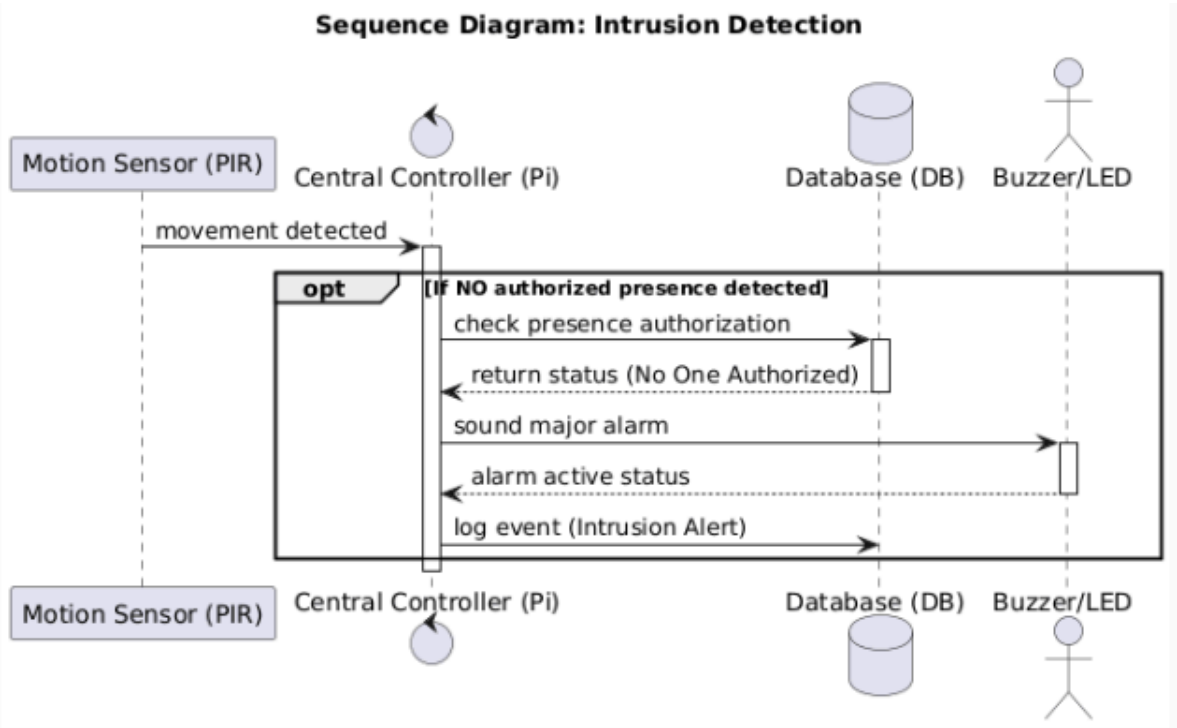


Figure 16. Sequence diagram of intrusion detection.

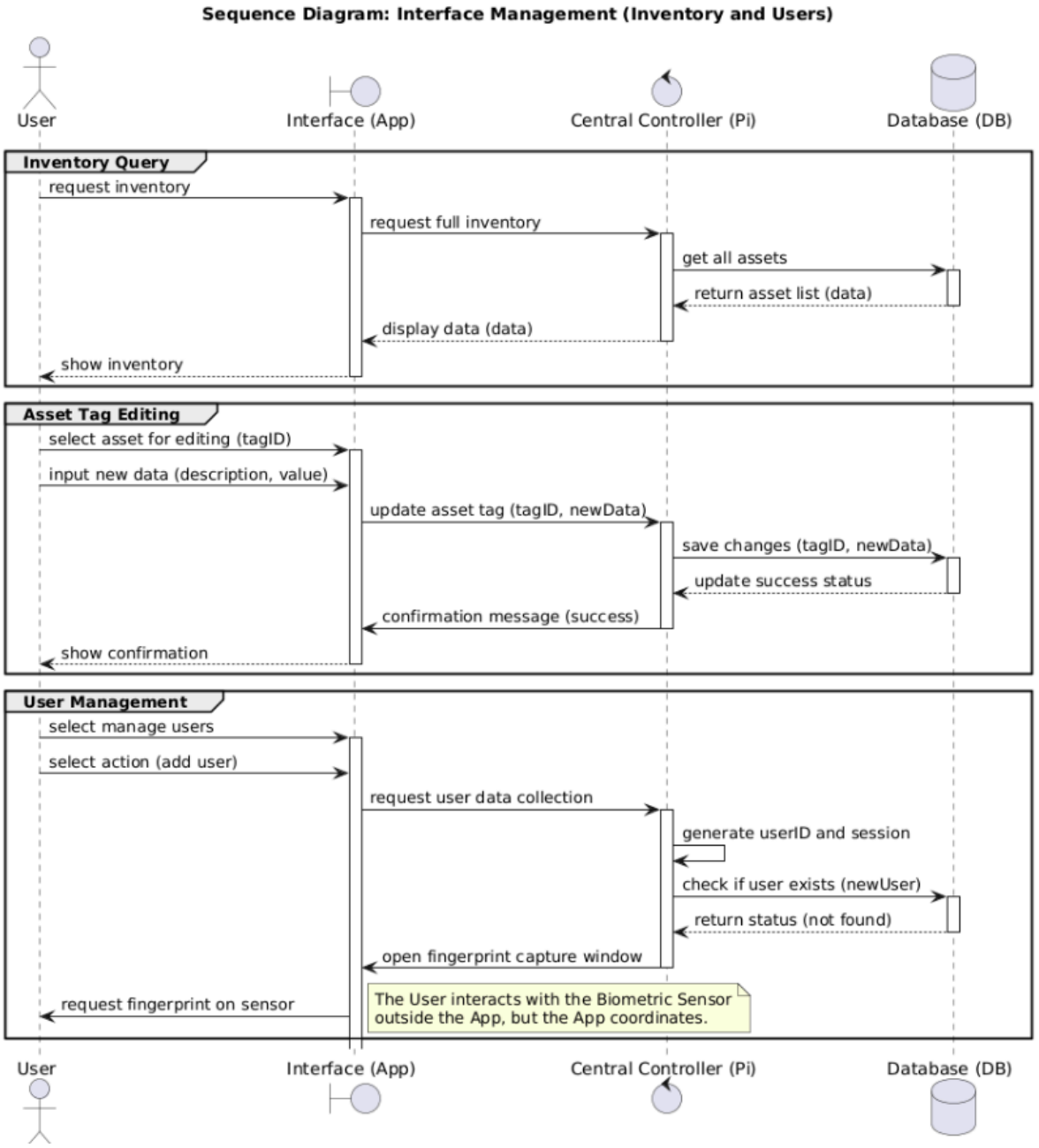


Figure 17. Sequence diagram of interface.

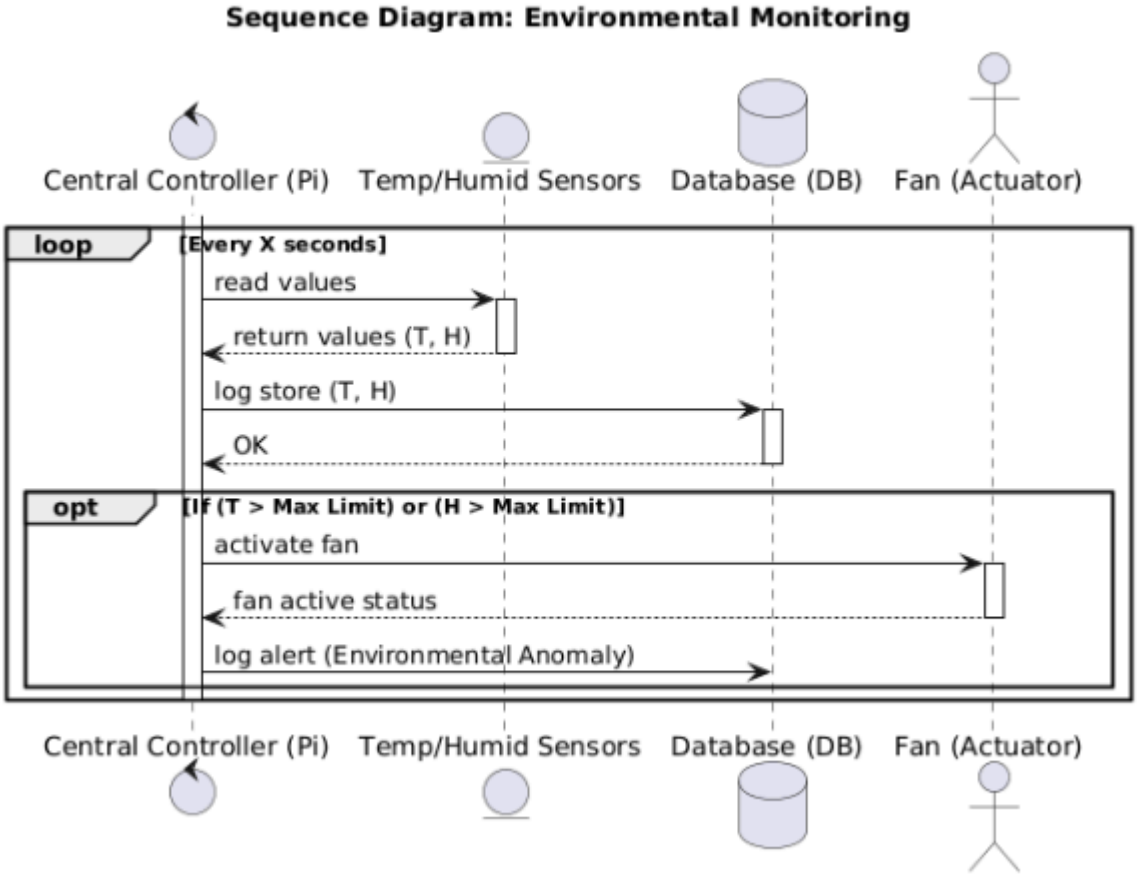


Figure 18. Sequence diagram of the environmental Monitoring.

5.9. Estimated Budget

Table 2. Estimated Project Costs

Item	Quantity	Unit price / €
Raspberry Pi 4 Model B	1	70
Temperature and humidity sensor	1	10
FAN	1	5
PIR sensor	1	5
Buzzer	1	1,5
Led	1	1,5
Servos	2	14
Reed switch	2	5
Fingerprint sensor	1	30
RFID sensors	3	50
Other Components	x	10
TOTAL	13	202

6. Design

After the analysis phase is completed and the goals of the Secure Asset Guard system are clearly defined, the project moves to the implementation phase. In this stage, the ideas and requirements previously studied are turned into a working solution. To avoid problems later, it is important to clearly understand what needs to be built and how each part of the system will function.

This chapter starts by identifying the hardware components used in the project and explaining how they connect and communicate with each other. After that, the software part of the system is described, including the main modules and how they process data and interact with the hardware.

A short explanation of the main technologies and components is also included to justify why they were chosen for this project. With all elements defined, diagrams and schematics are presented to illustrate how the Secure Asset Guard operates and how its various components work together.

6.1. Hardware Specification

6.1.1. Raspberry Pi 4 Model B

The Raspberry Pi 4 Model B (Figure 19) was selected as the development board and central processing unit for this system, fulfilling one of the project's technical constraints.

Figure 19. Raspberry Pi 4 Model B.

Specifications:

Processor: Broadcom BCM2711, quad-core Cortex-A72 (ARM v8) 64-bit SoC @1.5GHz;

- Memory: 2GB LPDDR4
- Connectivity:
  - Wireless 2.4GHz and 5.0 GHz;
  - LAN, Bluetooth 5.0, BLE;

  - Gigabit Ethernet;
  - 2x ports USB 3.O;
  - 2x ports usb 2.0;
- GPIO:
  - 40 pins;
  - Output voltage: 3.3V;
  - Output current: 16mA (maximum drive strength);
- Video:
  - 2x micro-HDMI ports (support 4k/60fps);
  - 2-lane MIPI DSI display port;
  - Output current: 16mA (maximum drive strength);
- SD memory card: supports micro-SD card to load the operating system and store data;
- Power:
  - 5V DC with USB-C;
  - 5V DC with GPIO header;
  - Power over Ethernet;
  - Minimum 3A;
- Operating temperature: 0oC to 50oC

6.1.2. Waveshare UART Capacitive Fingerprint Sensor (Type D)

The Waveshare UART Capacitive Fingerprint Sensor (Type D) is a compact and efficient biometric sensor module designed for secure fingerprint recognition and storage. Featuring a powerful Cortex processor, it provides fast, accurate, and omni-directional 360° fingerprint verification. With onboard storage for up to 150 fingerprints, low power consumption, and a simple UART interface, this module enables quick integration with embedded platforms like Raspberry Pi for secure access control of the safe.

Figure 20. Waveshare UART Capacitive Fingerprint Sensor.

Table 3. Specifications

Parameter	Value
Sensor	capacitive touch sensor
Resolution	508 DPI
Image pixels	160×160
Image greyscale	8
Sensor dimension	R15.5 mm
Fingerprint capacity	150
Matching time	<500ms (1:N, and N≤100)
False acceptance rate	< 0.001%
False rejection rate	< 0.1%
Operating voltage	2.7~3.3V
Operating current	< 50mA
Sleep current	< 16uA
Communication port	UART
Baudrate	19200 bps
Operating environment	Temperature: -20°C~70°C; Humidity: 40%RH~85%RH (no condensation)

6.1.3. RFID module (RDM6300)

This project uses the RDM6300 125kHz RFID module (Figure 21) for secure access control (entry and exist). The RFID reader authenticates users by reading unique ID tags, enabling authorized entry to the secure room. The module transmits the tag's unique identification code via UART, allowing seamless integration with the microcontroller for reliable door unlocking

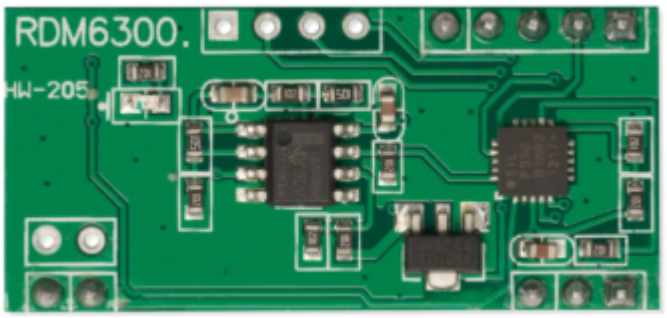


Figure 21. RDM6300 125kHz RFID module.

Specifications:

- Operating Frequency: 125 kHz.
- Baud Rate: 9600. 8 bits, stop bit, no parity ;

- Interface: TTL level RS232 format.
- Working Voltage: 5Vdc(+/-5%).
- Working Current: <50mA.
- Receive Distance: 20~50mm.
- Working Temperature: -10°C~+70°C.
- Humidity: 0~95%.
- Loop Antenna: 46x33x3mm.
- Reader PCB size: 38x18 mm

6.1.4. MG996R Metal Gear Servo Motor

The MG996R servo motor is selected for opening the door of the secure room and safe due to its high torque of 11 kgf·cm at 6V, providing sufficient force to operate locking mechanisms reliably. Its durable metal gears and fast response ensure precise and consistent control, making it ideal for secure access applications.

Figure 22. MG996R servo motor.

Specifications:

• Stall torque: 9.4 kgf·cm (4.8V), 11 kgf·cm (6 V)
• Operating speed: 0.17 s/60º (4.8 V), 0.14 s/60º (6 V)
• Operating voltage: 4.8V a 7.2V
• Running Current: 500mA.
• Stall Current: 2.5A (6V).
• Dead band width: 5µs
• Weight: 55g. Dimension: 40.7 x 19.7 x 42.9 mm approx

6.1.5. Reed Switch Sensor

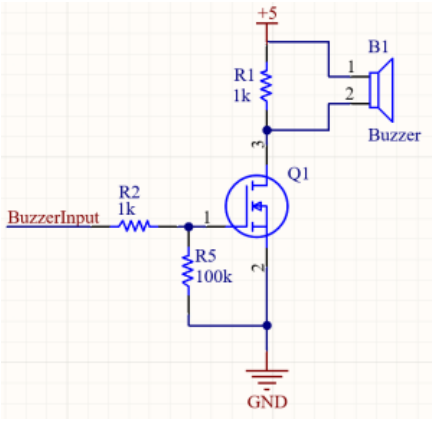
To detect whether the secure room door or the safe is open or closed, a digital magnetic sensor and a magnet will be used. The sensor acts as a closed circuit when within 11 to 16 mm of the magnet and as an open circuit when further away.

Figure 23. Reed Switcht.

6.1.6. Buzzer

To notify about intruders an active buzzer is a practical choice. This buzzer only requires a power supply to emit a sound, unlike passive buzzers that need a modulated input signal. The selected buzzer operates between 3.3 and 5 volts, drawing about 25mA of current and generating a tone near 2.3 kHz

Figure 24. Buzzer.



Because the buzzer consumes more current than a typical microcontroller pin can provide safely, a simple driver circuit with an N-channel MOSFET will be used. When the MOSFET gate receives a control signal above its threshold, it switches on and powers the buzzer. Resistors in the driver circuit protect the MOSFET and ensure stable operation by limiting current and preventing unwanted switching.

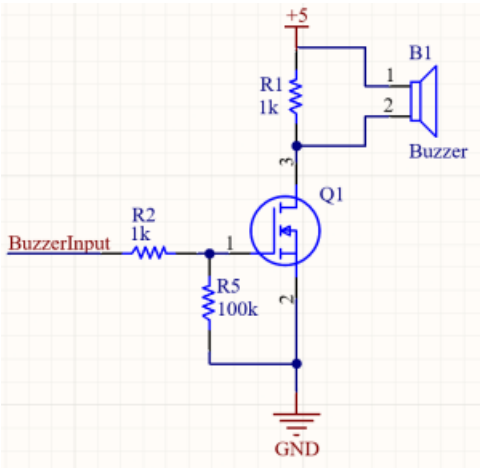
This setup allows the buzzer to alert the user effectively while maintaining safe electrical interfacing with the control system.

6.1.7. Electric fan

The fan will activate when the temperature exceeds a preset thresholdF

Figure 25. Eletric fan.

Since the fan requires more current than a typical microcontroller output can provide, a driver circuit is necessary. This driver consists primarily of an N-channel MOSFET acting as an electronic switch. The MOSFET gate receives an ON/OFF signal from the controller: when the gate voltage surpasses the threshold, the MOSFET switches on, powering the fan. Additional components such as gate resistors and protection resistors are included to ensure stable operation, protect the MOSFET from voltage spikes, and limit current during switching transitions.



This driver circuit enables safe and effective ON/OFFFA control of the fan, ensuring reliable operation and proper current handling.

6.1.8. HC-SR501 PIR motion sensor

The HC-SR501 PIR motion sensor detects movement by sensing infrared radiation from moving objects such as people This sensor will be used to monitor motion inside the protected room, serving as an intrusion detection component in the system.

Figure 26. HC-SR501 PIR motion sensor

Specifications:

- Input Voltage: DC 4.5-20V
- Static current: 50uA
- Output signal: 0,3V or 5V (Output high when motion detected)
- Sentry Angle: 110 degree
- Sentry Distance: max 7 m
- Shunt for setting overide trigger: H - Yes, L - No
- Operation Level Digital 5V
- Power Supply External 5V

6.1.9. YRM1001 UHF RFID Reader

For inventory control inside the safe, an RFID scanner module will be used to detect and read RFID tags on items. This scanner operates via radio frequency to identify tags quickly and reliably, allowing real-time inventory monitoring without manual handling. It is well suited for confined spaces like safes due to its compact form factor and efficient tag reading range, enhancing security and automated tracking. The RFID technology enables fast scanning of multiple items for accurate inventory management.

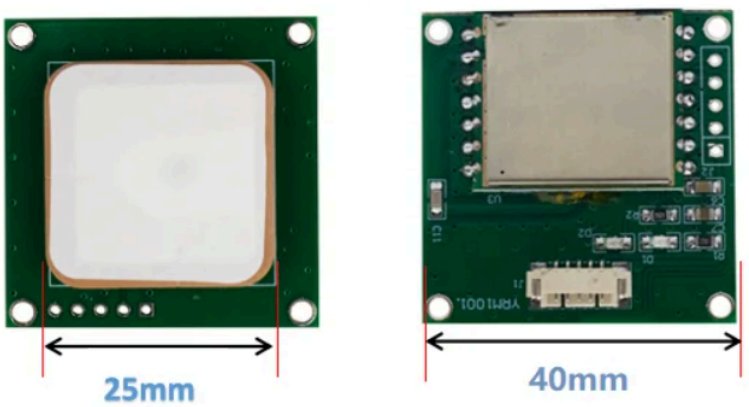


Figure 27. YRM1001 UHF RFID Reader.

Specifications:

- Frequency range from 840MHz to 960MHz which allows it to comply with local regulations.
- UART Command interface.
- Reading range up to 40 meter.
- Supply Voltage from 3.3v to 5v.
- Max current draw of 180mA.

6.1.10. Humidity and Temperature Sensor

The SHT30 sensor measures temperature and humidity and communicates digitally via the I2C protocol. It will be used to monitor environmental conditions inside the safe, providing critical data for maintaining proper storage conditions and triggering alerts if necessary.

Figure 28. SHT30 sensor.

Specifications:

- Operating voltage: 2.15 – 5.5 V
- Output: I2C interface output (up to 1 MHz)
- I2C addresses: 0x44 / 0x45
- Humidity Operating Range: 0 – 100 %RH
- Temperature Operating Range: -40 ℃ to 125 ℃
- Humidity accuracy: ±2 %RH (typical, SHT30)
- Temperature accuracy: ±0.2 °C (typical, 0 °C to 65 °C)
- Humidity response time: 8 seconds (τ63%)

6.1.11. Led

An LED will be used to visually alert when an intruder is detected. Upon intrusion detection, the LED will illuminate to immediately notify users of the security breach.

Figure 29. LED

6.1.12. 8-channel bidirectional logic level converter

An 8-channel bidirectional logic level converter will be used to safely interface 5V peripherals with the Raspberry Pi's 3.3V GPIO. The module automatically translates signal levels in both directions without requiring control lines, enabling reliable UART communication between the Pi and devices such as the RFID reader. This prevents damage and ensures correct signal interpretation throughout the system.

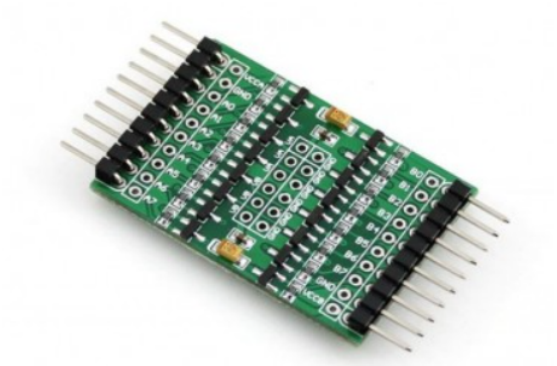


Figure 30. 8-channel bidirectional logic level converter.

Features

- 8 channels bidirectional voltage translation between different logic level, translate between Ax and Bx automatically
- 4 pairs of power supply interfaces, supports more situations
- Use tantalum capacitors for power filter, provides more stability
- Voltage level: 1.8V-6V
- Dimension: 31.6mm x 25.4mm

How to Use

In the case of translating between 3.3V and 5V system:

- VCCA/VA connects to 3.3V power supply

- VCCB/VB connects to 5V power supply
- GND connects to power negative pole respectively, the two power supply should be common-grounded with each other
- When Ax has TTL 3.3V input, Bx will get TTL 5V output
- When Bx has TTL 5V input, Ax will get TTL 3.3V output
- NO direction control required

6.1.13. Power Supply for raspberry Pi

This USB-C power adapter provides a stable 5V output at up to 3A (15W), designed to supply reliable power to devices requiring USB-C input, such as the Raspberry Pi 4. It accepts a wide input voltage range (100–240V AC, 50/60Hz) for global compatibility. The adapter ensures continuous and sufficient power delivery even when connected peripherals increase the load, making it ideal for maintaining stable system operation.

Figure 31. USB-C power supply.

Specifications:

- Input: 100–240V AC, 50/60Hz
- Output: 5V DC, 3.0A (15W)
- Output connector: USB Type-C

6.1.14. Power supply

A 12V DC 6A switched-mode power supply will be used to provide power to all system components except the Raspberry Pi. Designed for global compatibility, it accepts AC input from 100 to 240V and features protections against short circuit, overload, and overvoltage. This unit ensures a stable and reliable 12V output for connected modules, such as actuators, sensors, and supplementary controllers, allowing safe and continuous system operation.

Figure 32. 12V DC 6A power supply.

6.1.15. Step down

A DC-DC step-down converter will be integrated to supply 3.3V power from the system's main 12V rail. This module is intended for sensors and components that require a regulated 3.3V input. Featuring an onboard power indicator LED and output enable button, the converter helps ensure safe and stable operation for sensitive devices in the project.

Figure 33. DC-DC step-down converter 3.3 V.

Specifications:

- Input 12 V
- Output 3.3V
- Max current = 800 mA

A DC-DC step-down converter will be integrated to supply 5V power from the system's main 12V rail. This module is intended for sensors and components that require a regulated 5V input, and it is used to ensure that the outputs directed to the Raspberry Pi from the sensors are also at 5V. Featuring an onboard power indicator LED and output enable button, the converter helps ensure safe and stable operation for sensitive devices in the project.

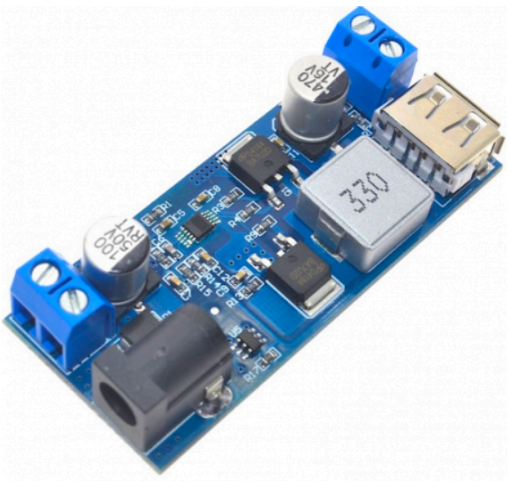


Figure 34. DC-DC step-down converter 5V.

Specifications:

- Input Voltage: 9 ~ 36VDC.
- Output voltage: 5VDC (USB female).
- Output current: 6A.
- Efficiency: up to 96%.
- Switching frequency: 500KHz.
- Operating temperature: -40°C ~ 85°C.

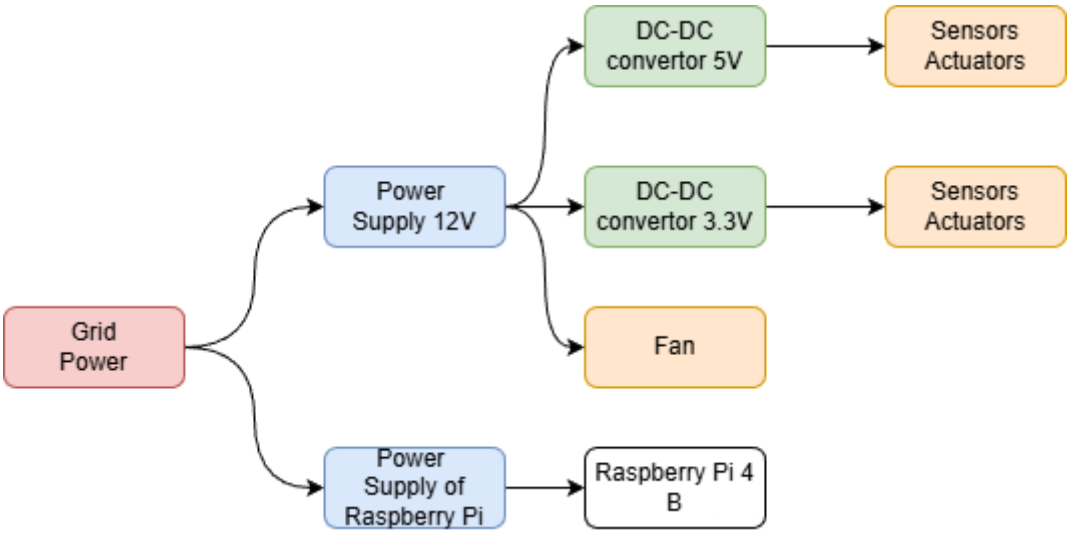


Figure 35 System Power Distribution Flow.

6.1.16. Deployment Diagram

Having specified the individual components, the central processing unit (Raspberry Pi 4), the dual power supply architecture, specialized sensors, and actuators, the following Deployment diagram (Figure 36) consolidates the complete hardware architecture.

This diagram illustrates how the Central Control unit interfaces with all peripherals. It differentiates between direct 3.3V logic connections (I2C, GPIO, and some UARTs), 5V peripherals requiring logic level conversion (via the Level Shifter, and high-power components (Servos, Fan) that are controlled via dedicated drivers (MOSFETs) and external power sources. This diagram visually integrates all components detailed in the hardware specification into a single, cohesive system.

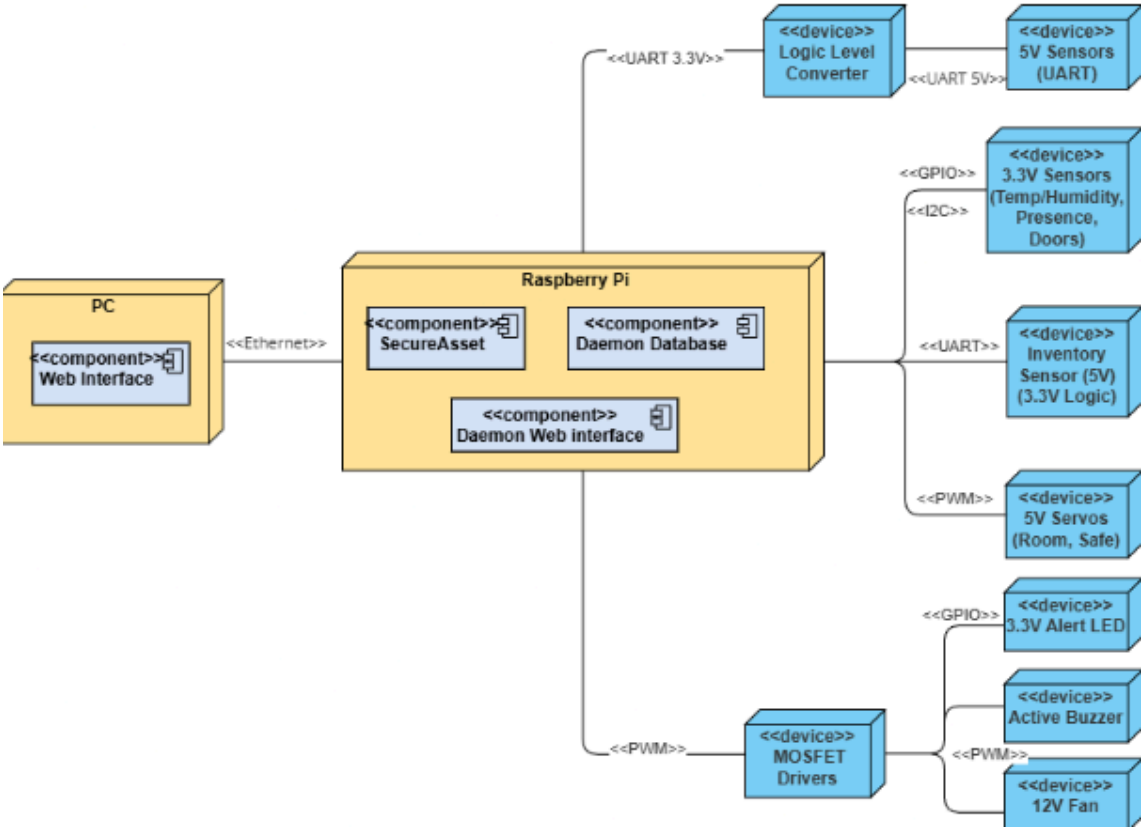


Figure 36. Deployment Diagram.

GPIO PINOUT

GPIO	Description
GPIO 2	I2C Data (SDA) for Temp/Humidity Sensor
GPIO 3	I2C Clock (SCL) for Temp/Humidity Sensor
GPIO 14	UART0 TX (Transmit) to Fingerprint Sensor
GPIO 15	UART0 RX (Receive) from Fingerprint Sensor
GPIO 0	UART2 TX (Transmit) to RFID Entry Sensor
GPIO 1	UART2 RX (Receive) from RFID Entry Sensor
GPIO 4	UART3 TX (Transmit) to RFID Exit Sensor
GPIO 5	UART3 RX (Receive) from RFID Exit Sensor
GPIO 8	UART4 TX (Transmit) to YMR1001 Inventory
GPIO 9	UART4 RX (Receive) from YMR1001 Inventory
GPIO 12	PWM (Hardware) Signal for Servo 1 (Room)
GPIO 13	PWM (Hardware) Signal for Servo 2 (Safe)
GPIO 18	OUTPUT control for 12V Fan
GPIO 17	INPUT from PIR (Presence) Sensor
GPIO 27	INPUT from Reed Switch (Room Door)
GPIO 22	INPUT from Reed Switch (Safe Door)
GPIO 6	INPUT from Fingerprint Sensor (WAKE pin)
GPIO 23	OUTPUT Control for Active Buzzer
GPIO 24	OUTPUT Control for Alert LED
GPIO 25	OUTPUT Enable (EN) for YMR1001 Inventory
GPIO 26	OUTPUT Sleep of fingerprint sensor

6.2. Software Components

6.2.1. Tool and Cots

The development of the "Secure Asset Guard" project relies on a specific set of software tools for design, development, and analysis, as well as key Commercial-Off-The-Shelf (COTS) components that form the software's foundation.

- **Buildroot:** A tool that makes it easy to generate custom Linux distributions and will be used to generate the image running on the Raspberry Pi.
- **Draw.io (diagrams.net):** Digramming tool used during the analysis and design phase to create all system diagrams, including system architecture, Use Case diagrams, UML diagrams (Class, Sequence), and the GUI sitemap.

- **Figma**: Web-based UI/UX design tool used to create the visual mockups and prototypes for the system's graphical user interface, which will be served by the Mongoose web server.
- **Clion**: The Primary Integrated Development Environment (IDE) used for writing, debugging, and managing the main C++ application code for the embedded system.
- **Cppcheck**: Static analysis tool for C/C++ code. It will be used to detect bugs, undefined behavior, and potential security vulnerabilities in the source code before compilation and execution.
- **Valgrind:** Dynamic analysis tool suite used for memory debugging and profiling. It will be essential for detecting memory leaks and threading errors during runtime, ensuring long-term stability.
- **Gprof**: A performance analysis (profiling) tool. It will be used to analyze the application's execution time, helping to identify and optimize performance bottlenecks within the C++ code.
- **Wireshark**: A network protocol analyzer. It will be used to debug the local TCP/IP communication between the Mongoose server and the web GUI, ensuring the internal API is functioning correctly.
- **Pthreads**: A standardized library defined by the POSIX standard that enables multithreading in C and C++ programs. It allows a program to execute multiple tasks concurrently by creating and managing threads, improving performance and responsiveness. The library provides functions for thread creation, synchronization, and communication between threads, making it suitable for systems that need to perform several operations at the same time.
- **Mongoose**: A lightweight embedded web server library that allows a device to host a web interface and manage HTTPS communication without relying on an external web server.
- **Sqlite:** A lightweight relational database management system that stores all data in a single file. It is serverless, meaning it runs directly within the application without requiring a separate database server. SQLite is widely used in embedded systems, mobile applications, and environments where simplicity, low resource usage, and fast local data access are important.
- **Visual Studio Code** (VS Code): Free, lightweight, and powerful source code editor widely used for web development with HTML, CSS, and JavaScript

6.3. Database

To give a clear view of the database structure, an entity-relationship diagram is presented in Figure 37 The Secure Asset Guard database is composed of six main entities:

- **Users:**
  This entity stores the users authorized to operate the system. Each user record includes a name, a stored RFID card identifier used for room access, and a fingerprint identifier used for vault access. These authentication methods allow the system to validate access attempts and log which person performed each action.
- **Sensors**:
  This entity stores all the sensors connected to the system. Each sensor is identified by an ID and described by its type (e.g., temperature) and the last measured value.
- **Actuators:**
  This entity stores all actuator devices controlled by the system. For each actuator, the database stores its type and current state.
- **Assets:**
  This entity represents the items stored inside the vault. For each asset, the database stores its name, RFID tag and current state. The state indicates whether the asset is currently inside the vault or has been removed. Additionally, the database keeps the timestamp of the last RFID reading, allowing the system to know when the asset was last detected, and the date when the asset was first registered in the system.
- **Logs:**
  This entity stores all system events. Each log entry contains a timestamp, a description of what occurred, the type of event (user, sensor, actuator or asset), and the identifier of the entity involved. This allows the system to keep a complete history of actions and access attempts, making it possible to trace who did what and when.
- **Main system:**
  This entity establishes a one-to-many relationship with all of the other entities. This means that a single Secure Asset Guard unit (running on the Raspberry Pi) can manage multiple users, sensors, actuators, assets and log entries simultaneously. In addition, this entity is responsible for storing the configurable system parameters, such as the temperature threshold, the sampling time for sensor readings, and the maximum number of failed access attempts allowed.

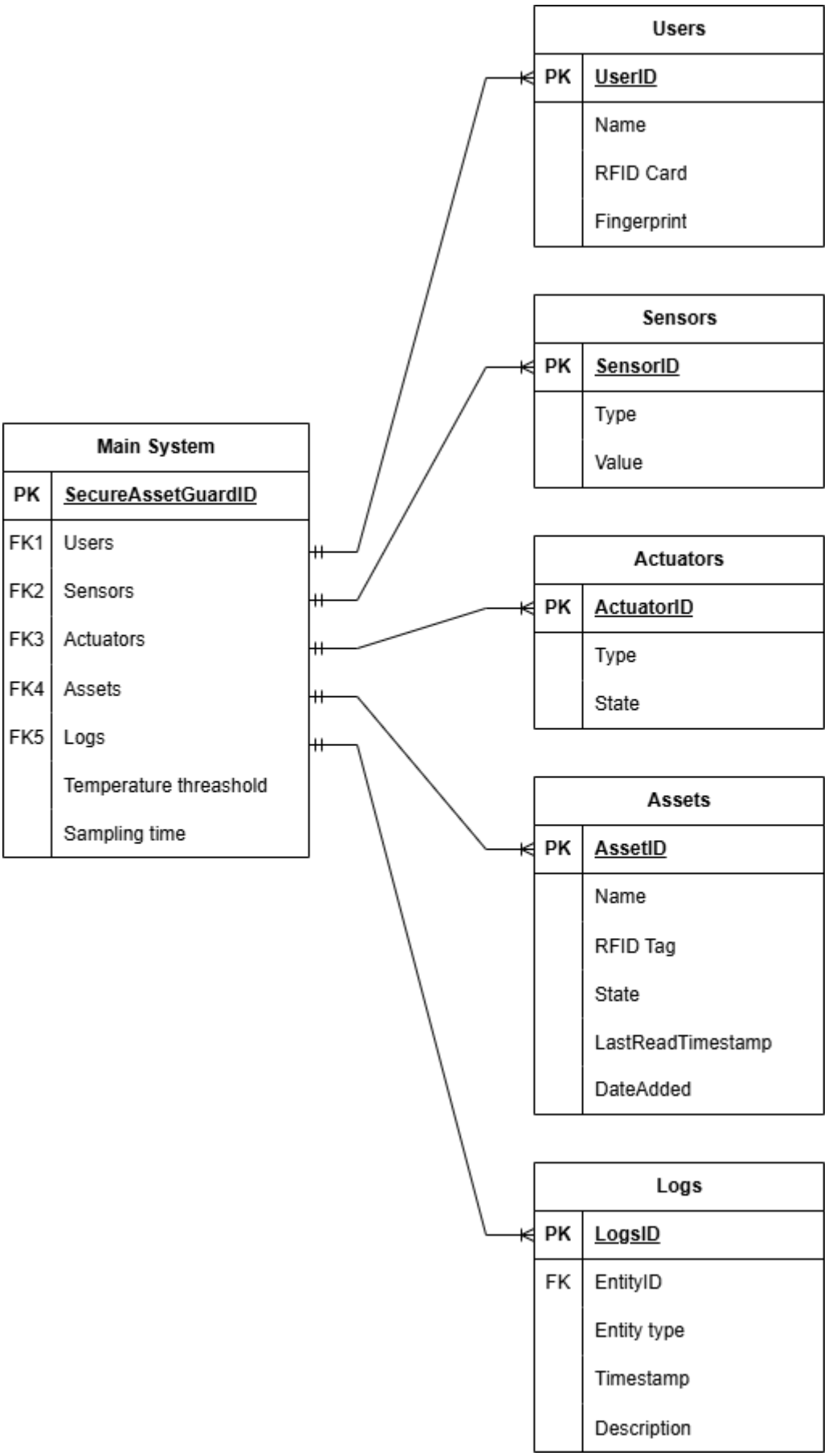


Figure 37. Entity Relationship Diagram.

6.3.1. Database security

In this system, all data is stored locally on the Raspberry Pi. The Raspberry Pi communicates only with a single PC over a private and isolated network, meaning the system does not expose data to external networks or remote servers. Because communication occurs only in a controlled environment, the main security concern is not data transmission, but the protection of the stored data. If someone obtains physical access to the Raspberry Pi or copies the SD card, they could attempt to read the information stored in the database. For this reason, securing data at rest is essential.

To guarantee confidentiality of stored information, the system uses SQLite together with SQLCipher, which applies AES-256 encryption to the entire database file. With this approach, the database stored on the SD card remains unreadable without the correct encryption key. Even if the storage is physically removed and analyzed on another device, the encrypted file appears as random data, and no meaningful information can be extracted.

To protect the encryption key, it is not stored in a plain-text file. Instead, the key is embedded within the compiled application in an obfuscated form, meaning the key is deliberately hidden and broken into parts so it cannot be easily read or extracted from the program. When the program starts, it temporarily reconstructs the key in memory to unlock the encrypted database, then immediately erases it from RAM. This prevents the key from being retrieved from the SD card or extracted using simple static analysis tools like the strings command.

6.4. System Security Architecture

While the *Secure Asset Guard* system is designed for a private, isolated network , a multi-layered security model is essential. This architecture addresses two primary concerns: the security of data in transit during user authentication, and the protection of user credentials stored within the database.

6.4.1. Authentication and Secure Transport

The communication between the client's web browser and the embedded Mongoose server is established via the HTTP protocol. To ensure that system data and controls are only accessed by authorized personnel, the system implements a credential-based authentication layer.

Since the HTTP protocol is inherently "stateless," a mechanism is required to maintain the user's authenticated state across different requests after the initial login. To address this, the system utilizes a session-based authentication model.

Upon a successful login—where the username and password are validated against the database—the server generates a unique, 32-character hexadecimal session token. This token is sent back to the client's browser and stored as an HTTP cookie with the HttpOnly

flag enabled, which serves as a security measure to prevent client-side scripts (such as JavaScript) from accessing the session data.

For all subsequent interactions, the browser automatically includes this session cookie in the HTTP headers. The server's application logic validates the token against its active session map on every request, confirming the user's identity and their respective access level (e.g., "Viewer" or "Admin") without requiring the user to re-submit their credentials for every action.

6.4.2. Credential Security (Password Hashing)

While the database *file* is protected by SQLCipher , the user passwords *within* that database require their own layer of cryptographic protection. Storing user passwords in plain text is a critical security flaw. For saving sensitive user data securely, using strong password hashing is a must.

The most robust methods for this are **Key Derivation Functions (KDFs)**, which are mathematical algorithms designed to be slow and resource-intensive, making them resistant to brute-force attacks.

Although several algorithms are available, such as Scrypt or Bcrypt, this project selected **Argon2** . Argon2 is the modern industry standard, designed to be highly resistant to both GPU and ASIC-based cracking attempts by being "memory-hard." The Raspberry Pi 4's processor has sufficient performance to handle the computational cost during login, prioritizing security.

To mitigate "rainbow table" attacks, where identical passwords would produce identical hashes, **password salting** is employed. Salting is the process of adding a unique, randomly generated string (the "salt") to each user's password *before* hashing . This salt is stored in the database alongside the hash. The Argon2 algorithm handles this process automatically, ensuring every hash is unique, even if the passwords are the same .

6.4.3. GUI Interface

The graphical user interface (GUI) for the Secure Asset Guard system is implemented as a web interface, designed to provide an intuitive, user-friendly experience while displaying data, system logs, and enabling administrative tasks. It fulfills the non-functional requirement for simplicity and clarity, as well as functional needs for monitoring and management. The interface is structured around core system entities (Sensors, Actuators, Logs, Users, Assets) with a focus on visual consistency, hierarchical navigation, and role-based access—administrative actions are restricted to authorized users. The design prioritizes minimalism, color-coded feedback, and modular layouts to support efficient workflows in security monitoring scenarios. This interface will be done with html, CSS and JavaScript

The interface is organized around a main navigation menu that provides access to all system functionalities. The following diagram summarizes the navigation logic and the available screens:

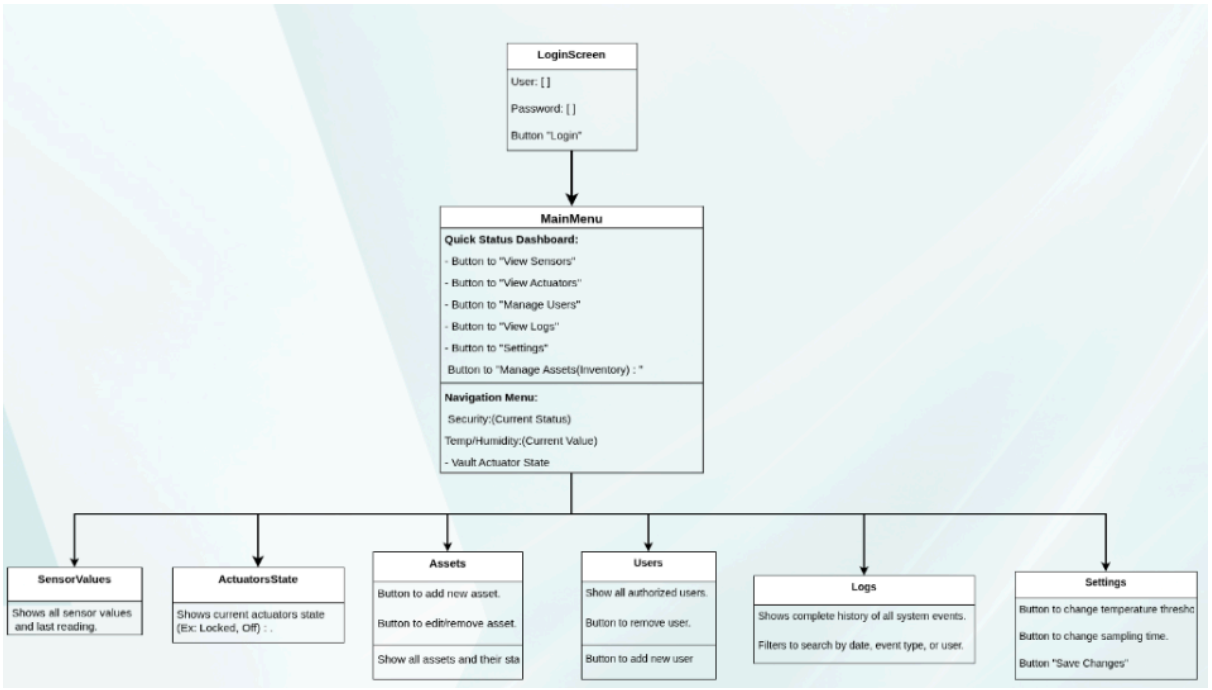
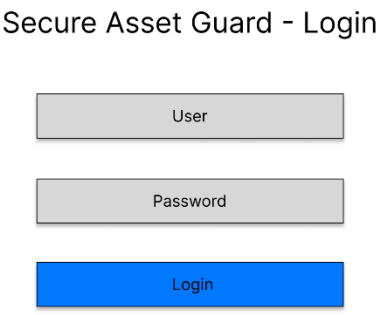


Figure 38 . Navigation Flowchart

6.4.4. Authentication Screen (Login)

The login screen is the entry point to the system, ensuring secure access to confidential data and functions through username and password verification in the database.

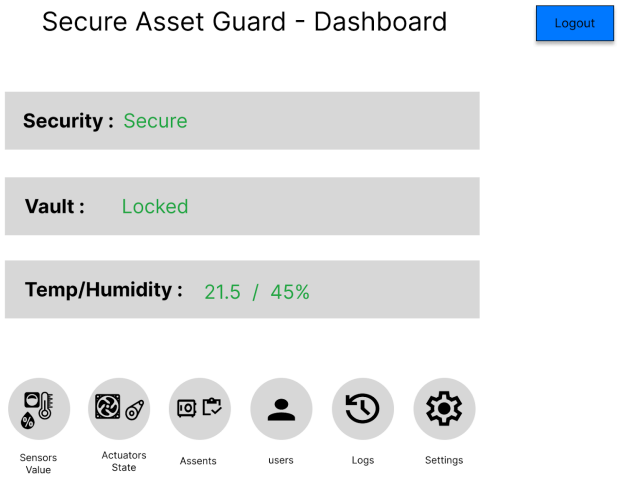
- **Functionality:** Users input credentials, which are authenticated to grant access to the main dashboard. Upon success, the interface redirects to the main menu, initiating data updates.
- **Design Rationale:** A minimalist layout reduces visual clutter, with clearly labeled input fields and a prominent "Login" button (highlighted in primary color, e.g., blue) to guide user focus.



6.4.5. Main Menu and Quick Status Dashboard

This serves as the central hub, offering an immediate overview and navigation to all modules providing both an immediate status overview and the primary navigation point for all management and monitoring modules.

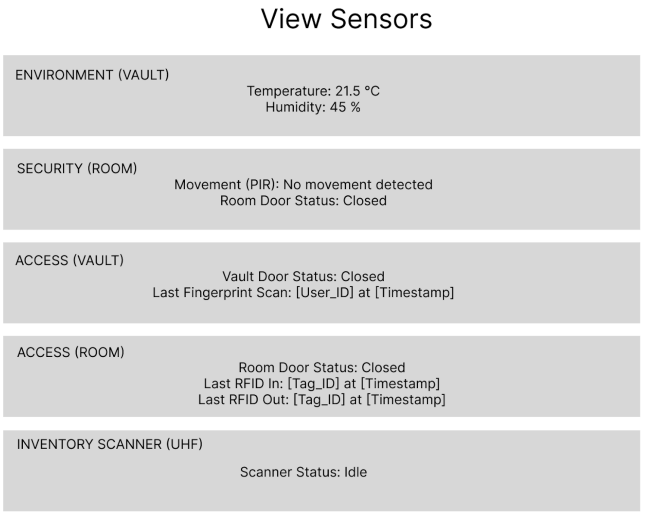
- **Composition**: The top section features a quick status panel with color-coded indicators for security status (e.g., SAFE/ALERT in green/red), vault lock state (LOCKED/UNLOCKED), and environmental readings (temperature/humidity).
- **Navigation**: The lower area uses card-style buttons with icons and labels for direct access to modules, providing a visual map of system functions.



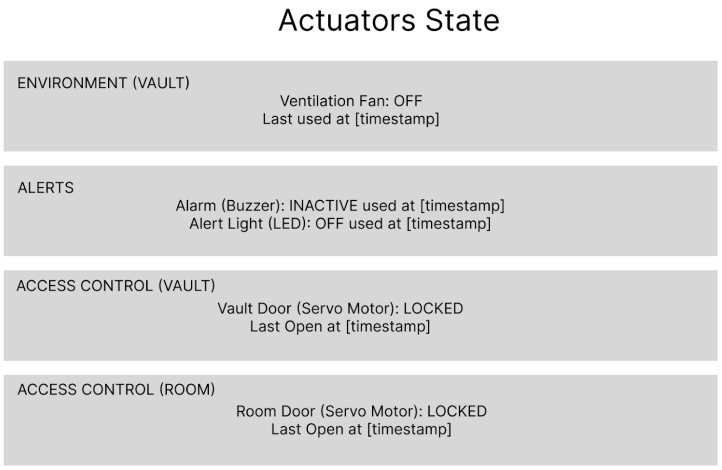
6.4.6. Monitoring Modules (View-Only)

These read-only screens are accessible to all authenticated users (operators and administrators) and provide monitoring of sensors, actuators, and system logs. No modifications are allowed; actuator control is fully autonomous via the Central Controller (Raspberry Pi) based on database thresholds. The layout uses categorized sections, color-coded status (green: normal, yellow: warning, red: alert), and clear timestamps for rapid assessment.

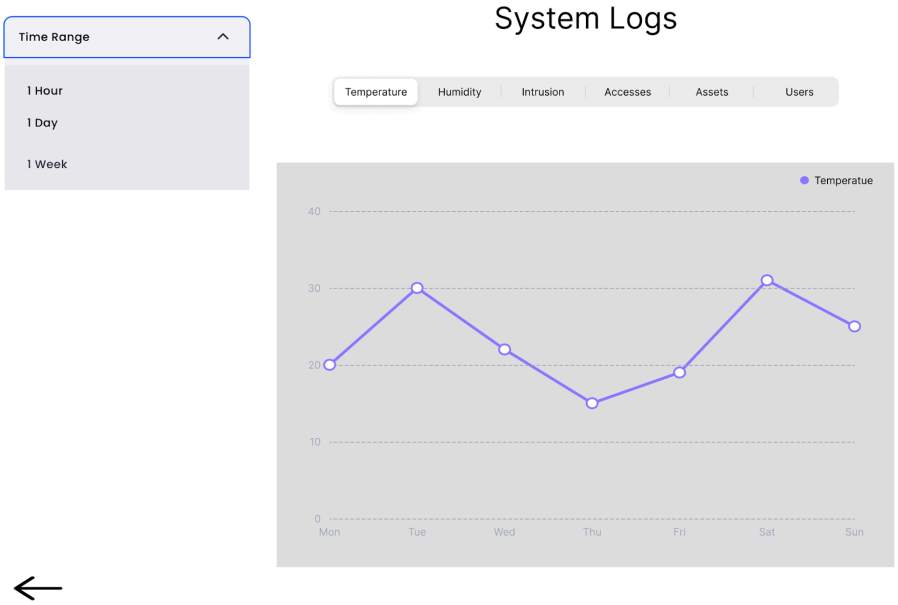
- **VIEW SENSORS**: Grouped into **Environment** (temperature, humidity), **Security** (door status, intrusion), and **Access** (door status, RFID events). Displays sensor name, current value, unit, last reading timestamp, and status badge for quick hazard detection and integrity checks.



- **VIEW ACTUATORS**: Organized by **Access Control** (servo: LOCKED/UNLOCKED), **Alerts** (buzzer/LED: ON/OFF), and **Environment** (fan: ON/OFF). Shows current state, activation time (if active), and response indicator to confirm automated actions (e.g., fan on due to heat).



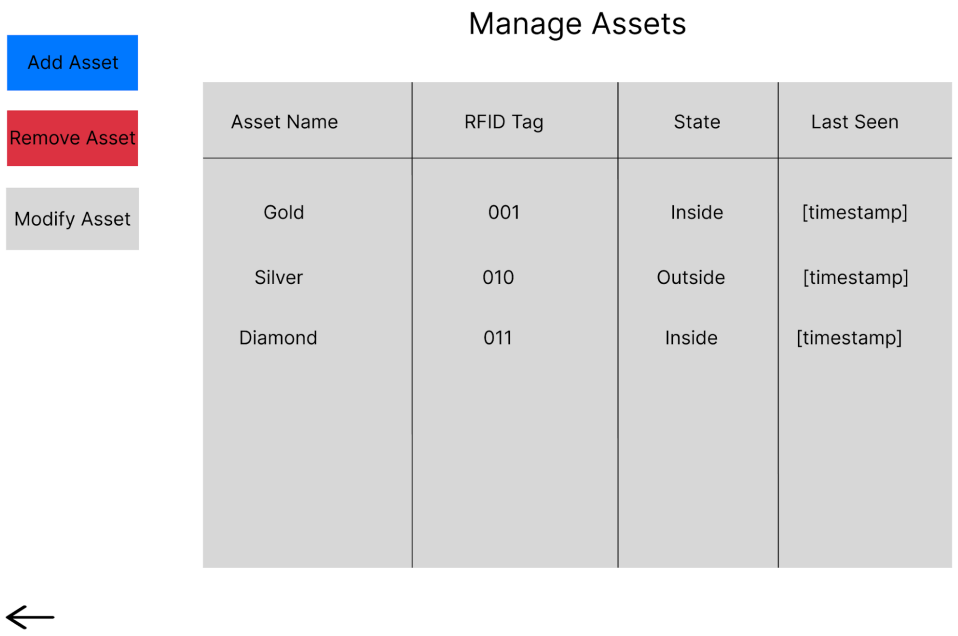
- **VIEW LOGS**: Displays full event history in chronological order: access attempts, threshold breaches, actuator triggers, RFID/fingerprint events, and asset movements. Includes filters (date, type, severity) and visual cards (**Security & Alerts**, **Access History**, **Environmental Trends** with line graphs). Supports audit and trend analysis without data alteration.



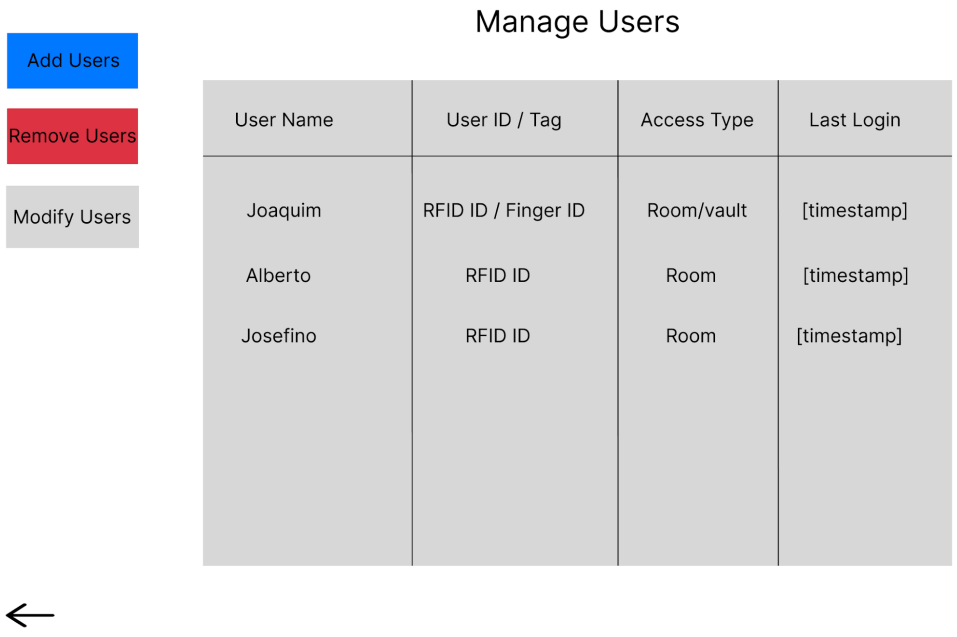
6.4.7. Management Modules (Interactive Tables)

These interactive screens provide administrators with full capabilities over the system's core entities—Assets, Users, and Settings—using intuitive tabular and form-based interfaces. Access is strictly restricted to users with administrative privileges, ensuring secure configuration of inventory, access control, and operational parameters. The design employs standard UI patterns: sortable tables, modal forms, inline editing, and color-coded action buttons (blue for **Add**, gray for **Modify**, red for **Remove**) to reduce errors. All changes are instantly saved to the database, logged, and applied system-wide (e.g., new RFID registration, threshold updates).

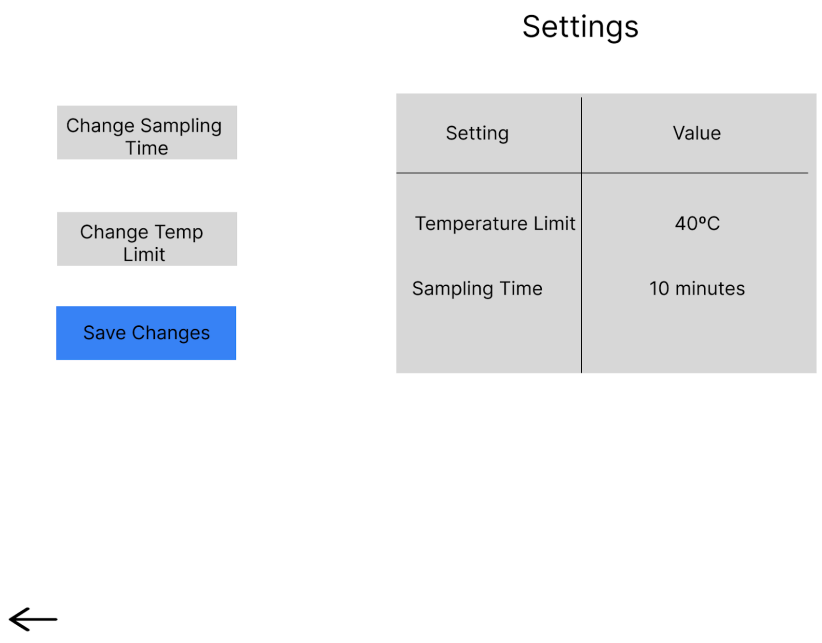
- **MANAGE ASSETS**: Displays vault inventory in a table with **Asset Name**, **RFID Tag**, **State** (Inside/Removed, color-coded), and **Last Seen**. Admins can:
    - **Add** assets via modal (name + RFID).
    - **Modify** name or RFID.
    - **Remove** with confirmation.
    - **View** status and movement history via log links.



- **MANAGE USERS**: Controls access permissions via table mapping **User Name**, **RFID Card ID**, **Fingerprint ID**, **Role** (Admin/Operator), and **Last Login**. Admins can:
    - **Add** users with biometric/RFID credentials.
    - **Modify** details, roles, or access methods.
    - **Remove** or lock accounts.
    - **View** authentication history.



- **SETTINGS**: Provides input fields to configure autonomous behavior:
    - **Temperature Threshold** (triggers fan).
    - **Sensor Sampling Interval**.



6.5. Threads Overview

To ensure the system operates efficiently and that critical events are handled without delays, the threads are assigned to different priority levels.

The highest priority thread is tAct, since it controls the actuators. Whenever a critical event occurs, such as an intruder being detected or the temperature exceeding a defined threshold the system must react immediately. This thread cannot be delayed because actions like triggering the alarm or activating the fan must occur as fast as possible.

The medium priority threads are tVerifyRoomAccess, tVerifyVaultAccess, and tCheckMovement and they are responsible for detecting events, such as validating access or confirming movement inside the room. These tasks are important and need to be processed quickly, but they do not require the same level of urgency as tAct.

Finally, tInventoryScan and tReadEnvSensor are assigned low priority, as they perform readings where small delays do not compromise system operation, making them suitable for a lower priority group.

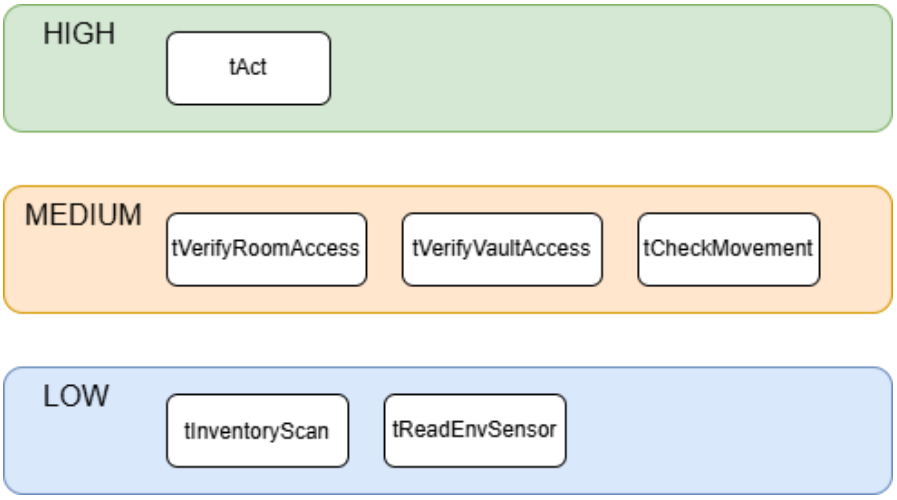


Figure 39. threads priority

6.6. Thread Behavior

6.6.1. tReadEnvSensor

The thread **tReadEnvSensor** waits for the condition variable cReadEnvSensor to be triggered. When activated, it reads the temperature and humidity sensor value.

If the temperature value is above a defined threshold, a command is sent to the message queue associated with thread tAct, so the actuators can be activated, more precisely the fan. After, the measured value is sent to the database through the message queue mqToDatabase so it can be stored.

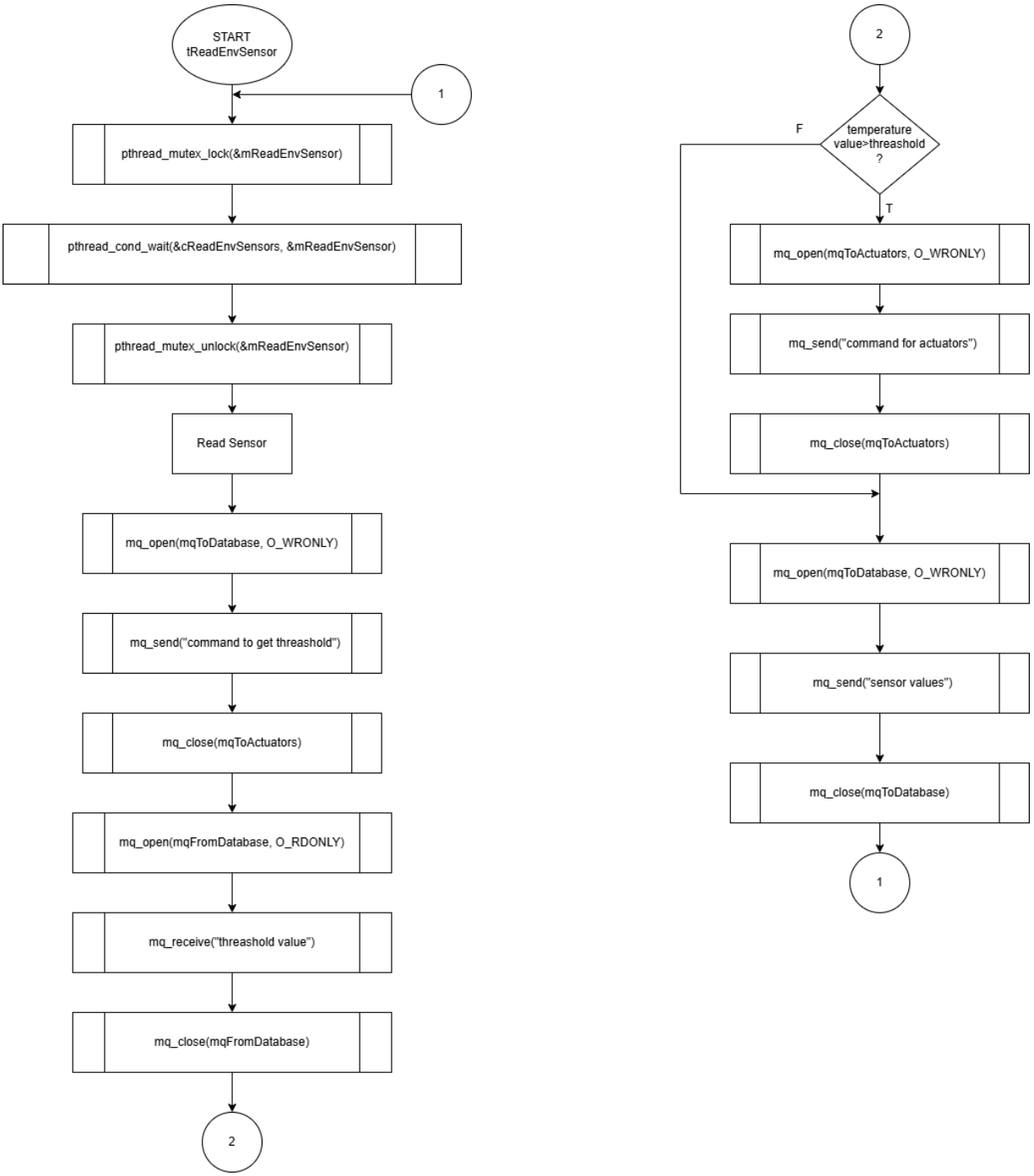


Figure 40. Flowchart of tReadEnvSensor

6.6.2. tVerifyRoomAccess

The thread **tVerifyRoomAccess** waits for the condition variable cVerifyRoomAccess to be triggered. When activated, it checks if the room door is already open.
If the door is closed, the thread reads the RFID card and sends the received RFID identifier to the database through the message queue mqToDatabase, requesting authentication.

After receiving the response from the database through the message queue mqFromDatabase, if the authentication is successful, a command is sent to the message queue associated with the thread **tAct**, which will unlock the room door. If authentication fails, the number of failed attempts is incremented and, when the maximum number of attempts is reached, a command is sent to trigger the alarm through the same actuator thread. At the end, an access log is sent to the database.

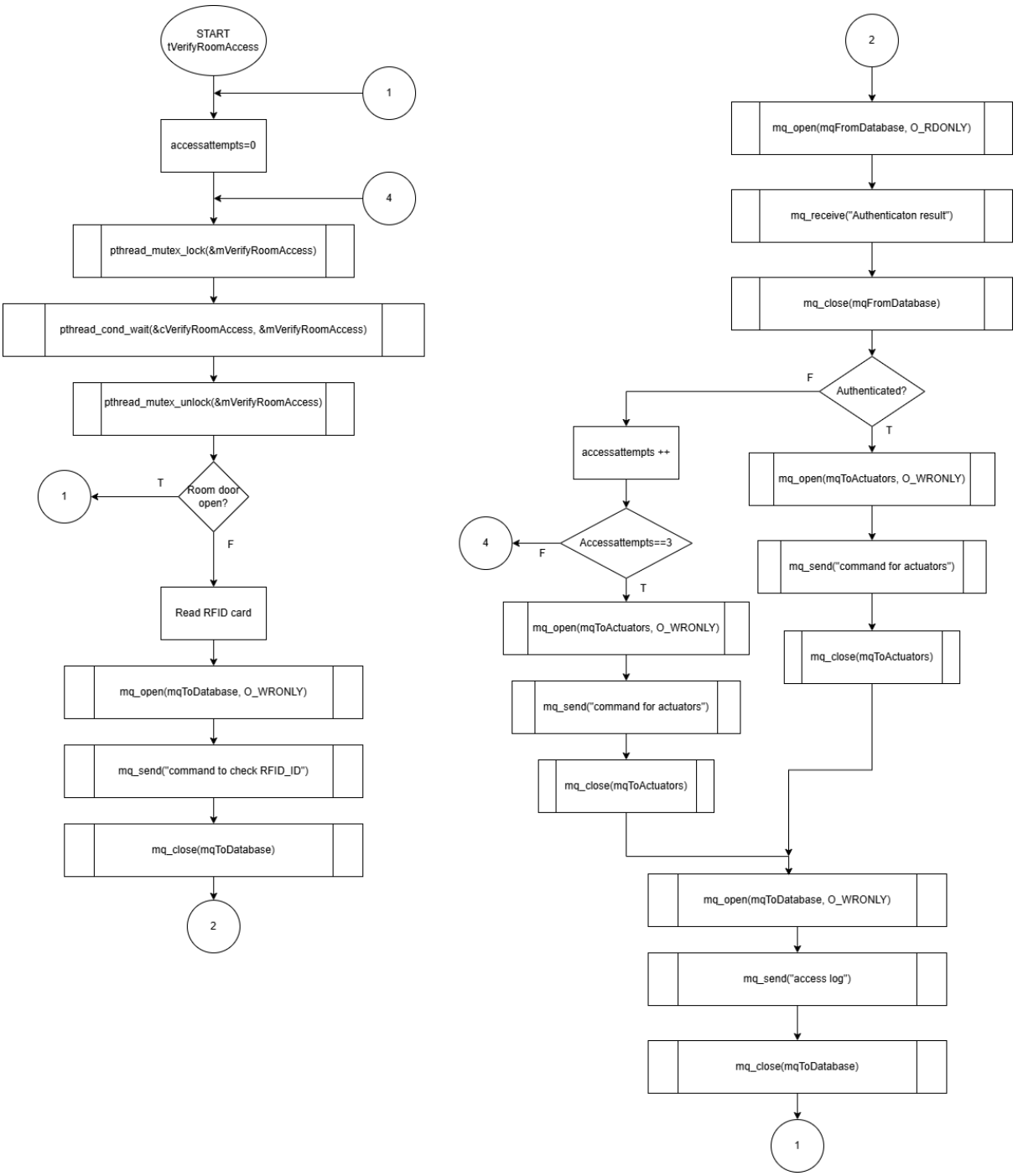


Figure 41. Flowchart of tVerifyRoomAccess

6.6.3. tAct

The thread **tAct** is responsible for handling all actuator commands. It continuously waits for a message on the message queue mqToActuators. Whenever a command is received such as locking or unlocking a door, activating the alarm, or turning on the fan the thread interprets the command and performs the corresponding action on the actuators.

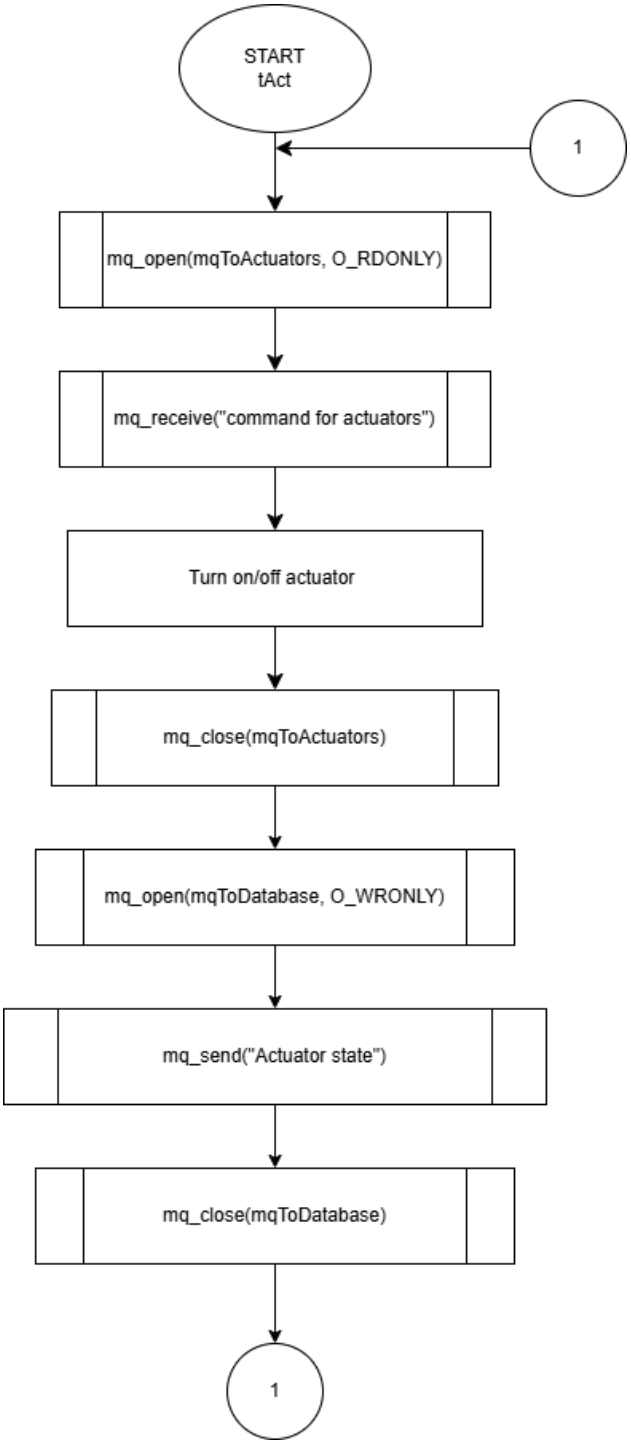


Figure 42. flowchart of tAct

6.6.4. tInventoryScan

The thread **tInventoryScan** waits for the condition variable cInventoryScan to be triggered (this happens when the vault door is opened or closed). Once activated, it reads the RFID tag inside the vault to identify the detected asset. The thread sends the scanned RFID identifier to the database through the mqToDatabase message queue, requesting verification of whether the asset exists in the system. After receiving the response from the mqFromDatabase message queue if the asset exists in the database, its status (inside or outside the vault) is updated by sending a message to the database. If the asset is not recognized, a message is sent indicating that a new or unauthorized asset was detected.

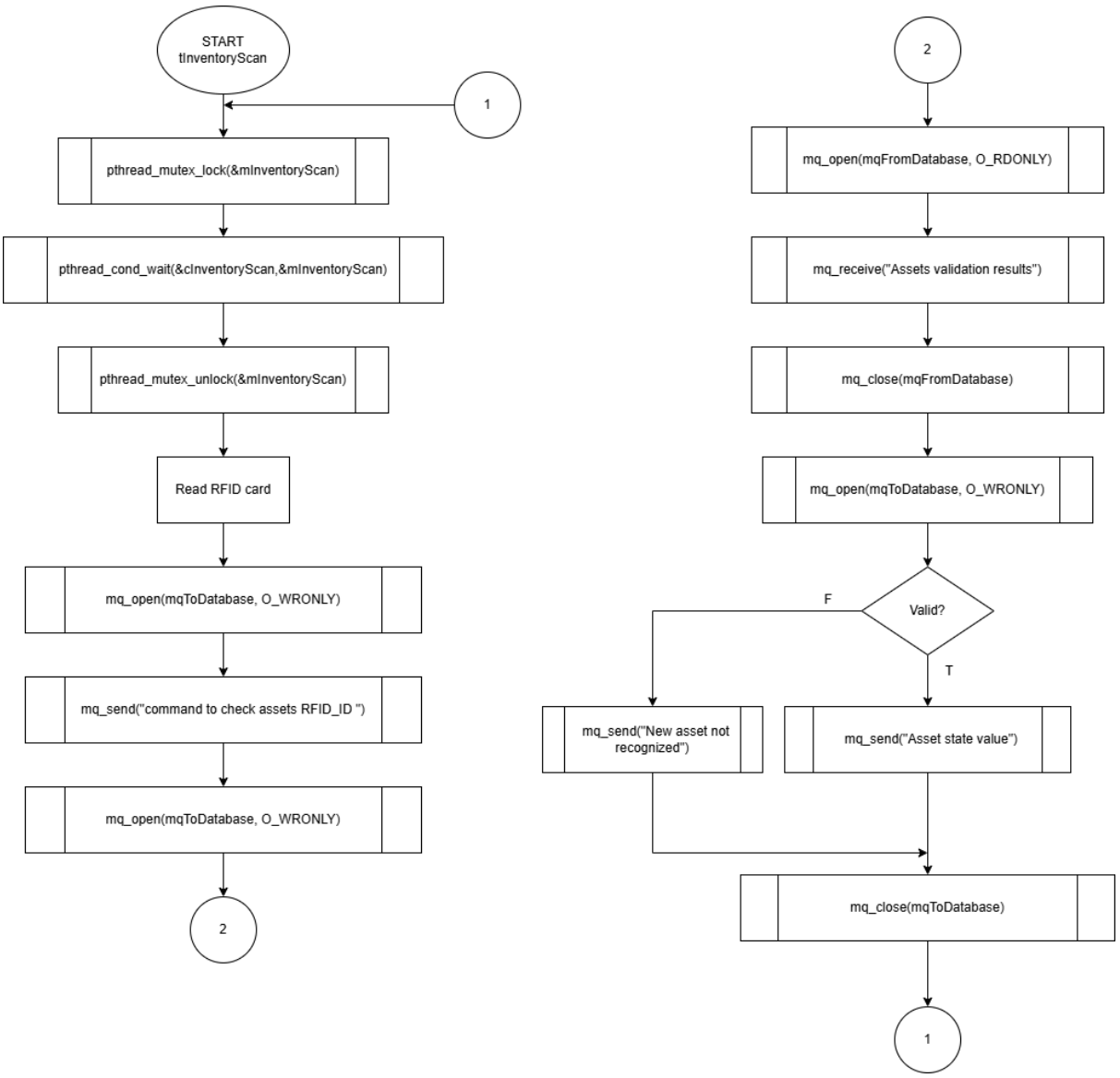


Figure 43. tInventoryScan

6.6.5. tVerifyVaultAccess

The thread **tVerifyVaultAccess** waits for the condition variable cVerifyVaultAccess to be triggered (this occurs when a fingerprint is detected on the vault biometric module). The fingerprint sensor performs the authentication internally, meaning the system immediately knows whether the fingerprint is authorized without needing to send a request to the database.

If the authentication is successful, a command is sent to the mqToActuators message queue so the vault door can be unlocked. Regardless of the authentication result, the access attempt is logged by sending a message to the database through the mqToDatabase message queue.

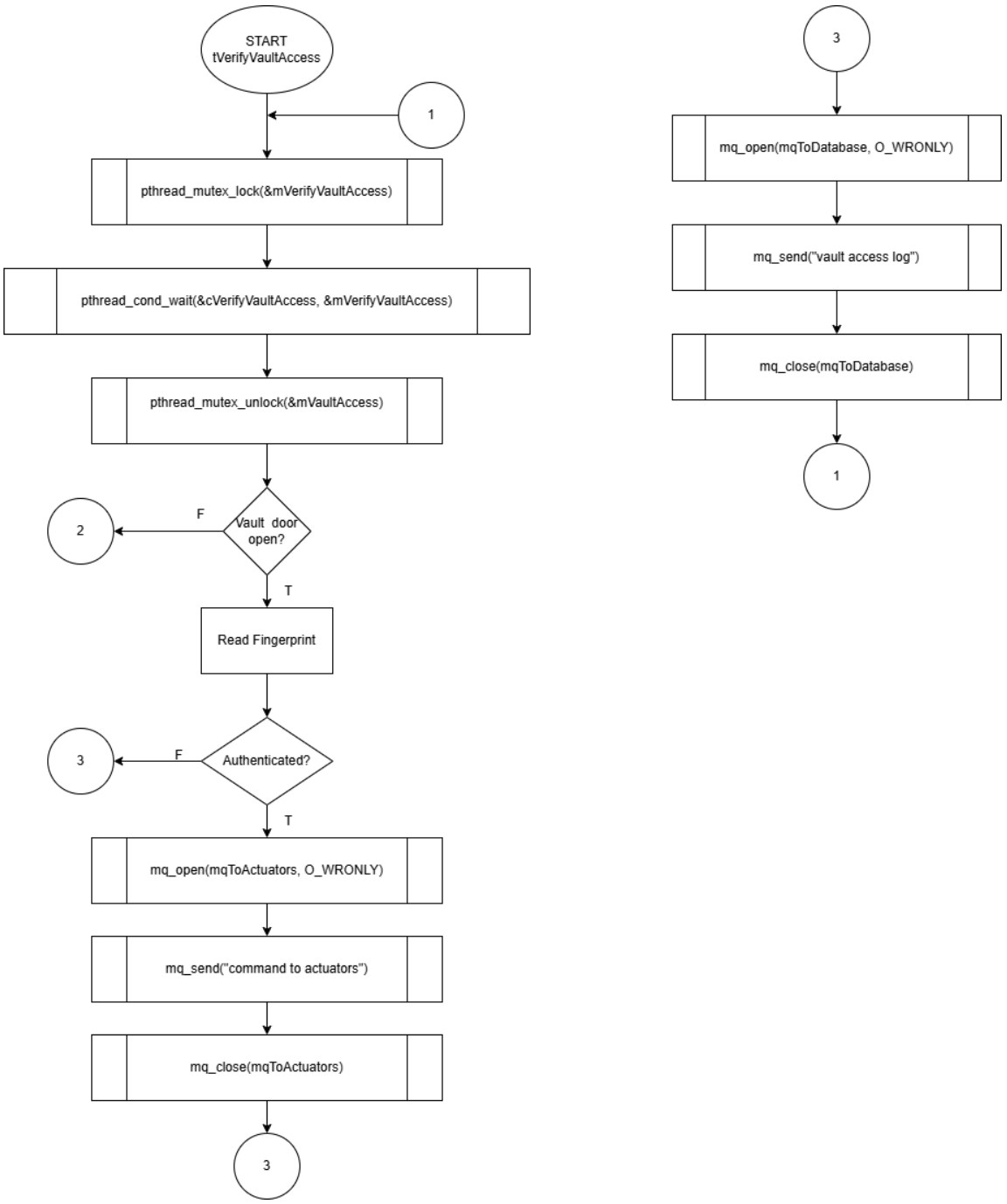


Figure 44.tVerifyVaultAccess

6.6.6. tCheckMovement

The thread *tCheckMovement* waits for the condition variable cCheckMovement to be triggered, meaning that movement was detected by the PIR sensor. When activated, the thread sends a request through the mqToDatabase message queue asking if there is any user registered as currently inside the room. The thread then waits for the response in the mqFromDatabase queue.

If the database indicates that no authorized user is present in the room, a command is sent through the mqToActuators queue to activate the alarm system, and an “invalid access”

log is stored in the database. If the response confirms that someone is properly registered inside the room, no action is taken and the system returns to standby.

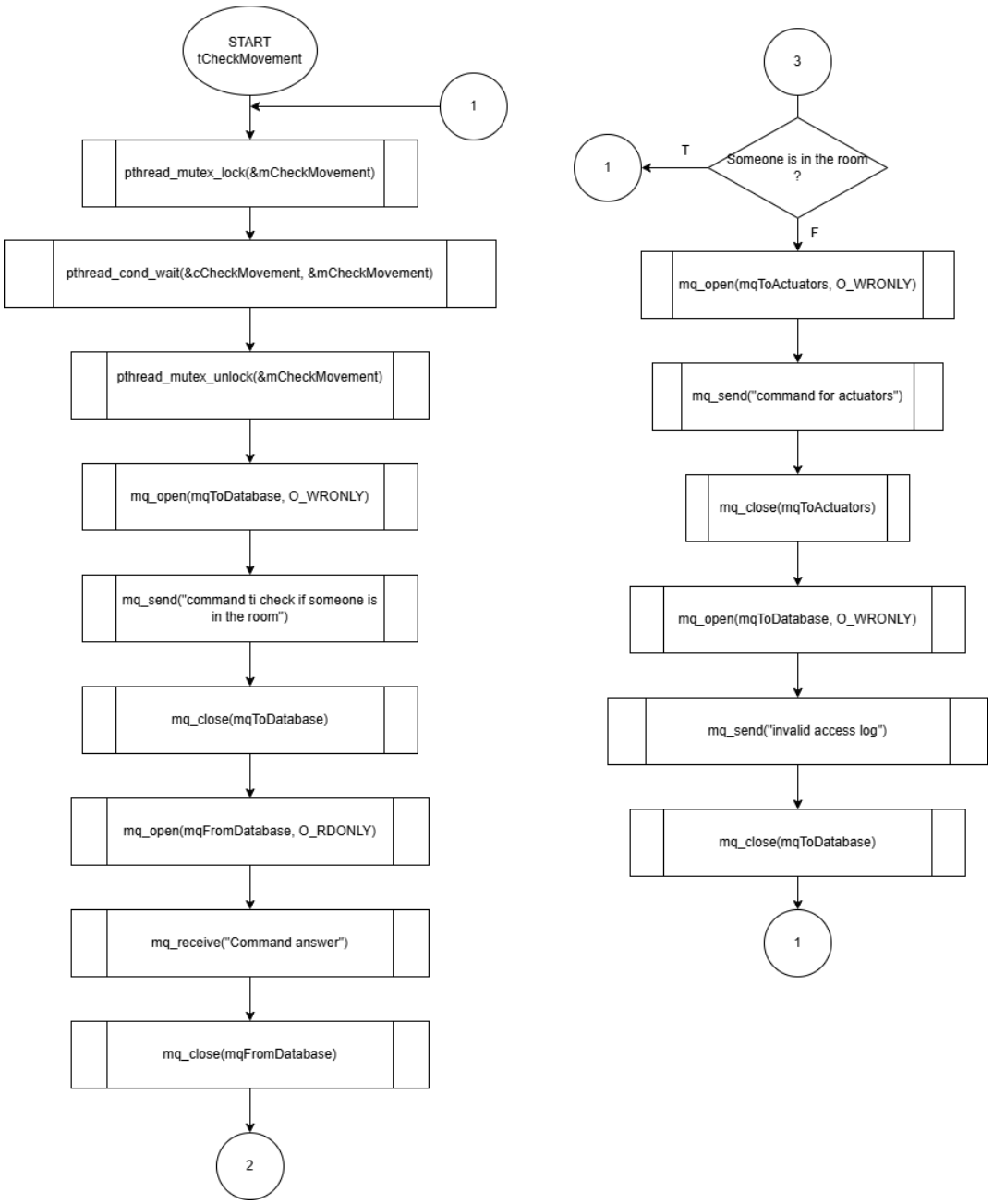


Figure 45 tCheckMovement

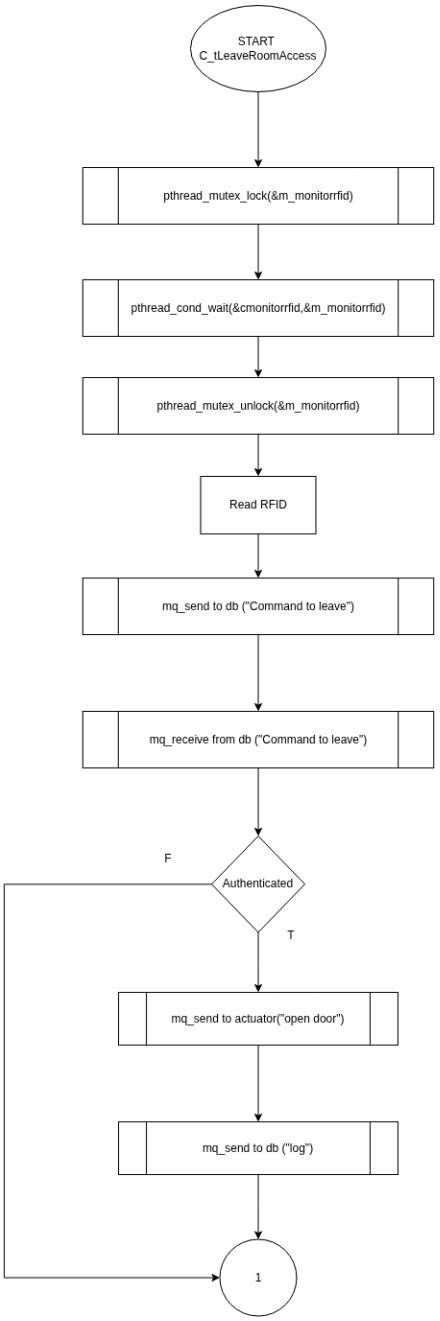
6.6.7 tLeaveRoomAccess

The thread tLeaveRoomAccess waits for the condition variable cLeaveRoomAccess to be triggered. When activated, it checks if the room door is already open.

If the door is closed, the thread reads the RFID card at the exit and sends the received RFID identifier to the database through the message queue mqToDatabase, requesting the user to be registered as leaving the room (updating the “inside room” state).

After receiving the response from the database through the message queue mqFromDatabase, if the authentication is successful, a command is sent to the message queue associated with the thread tAct, which will unlock the room door to allow the exit. At the end, an exit access log is sent to the database.

Unlike tVerifyRoomAccess, this thread does not use a maximum number of failed attempts, since it only manages the exit procedure and updates the user presence state.



6.7. Daemons

A daemon is a background process that runs continuously without direct user interaction. In this project, two daemons were implemented

6.7.1. dWebServer

The daemon dWebServer handles communication with the web interface and forwards all requests to the database. It uses the Mongoose web server library, waiting for HTTP requests from the user. When a request is received, the daemon processes the received data and sends a corresponding command to the database daemon through the message queue *mqToDatabase*. After sending the request, it waits for the database response via *mqFromDatabase*. Once the response is received, the daemon returns the appropriate HTTP reply to the web interface.

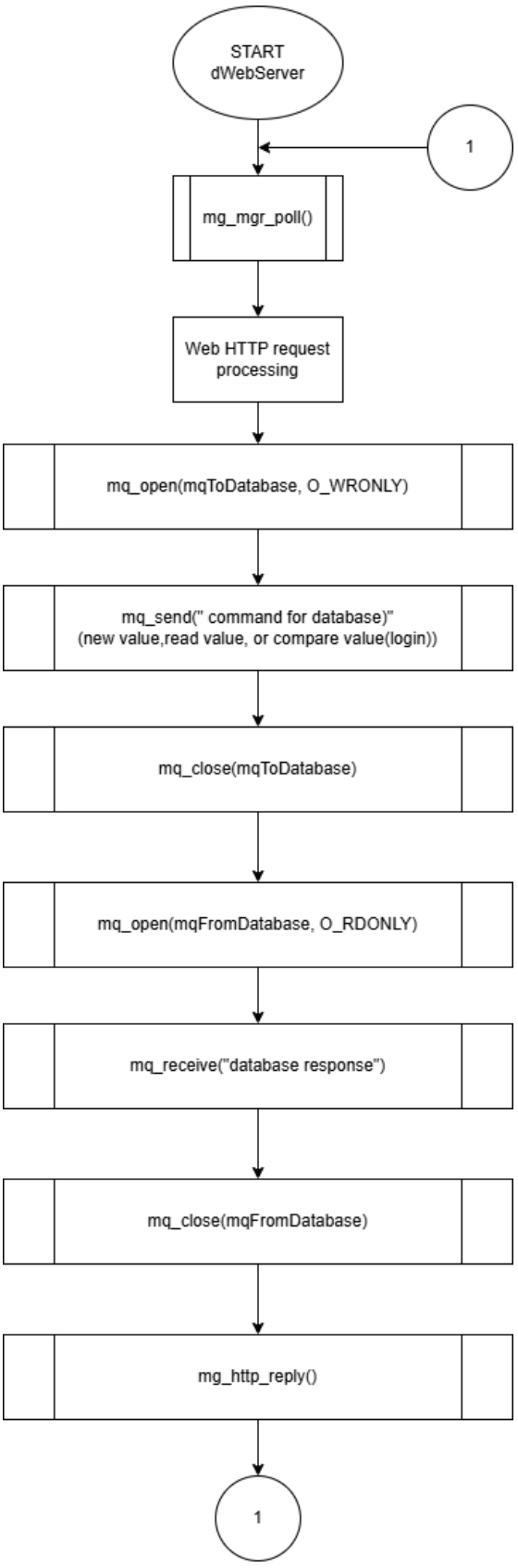


Figure 46. dWebServer

6.7.2. dDatabase

The daemon **dDatabase** is responsible for managing all communication with the database. It constantly waits for incoming messages from the mqToDatabase message queue, each representing a database operation requested by another thread or daemon. After receiving a message, it processes the command and executes the corresponding action on the database. Once the operation is completed, if needed the daemon sends a response back through the mqFromDatabase message queue.

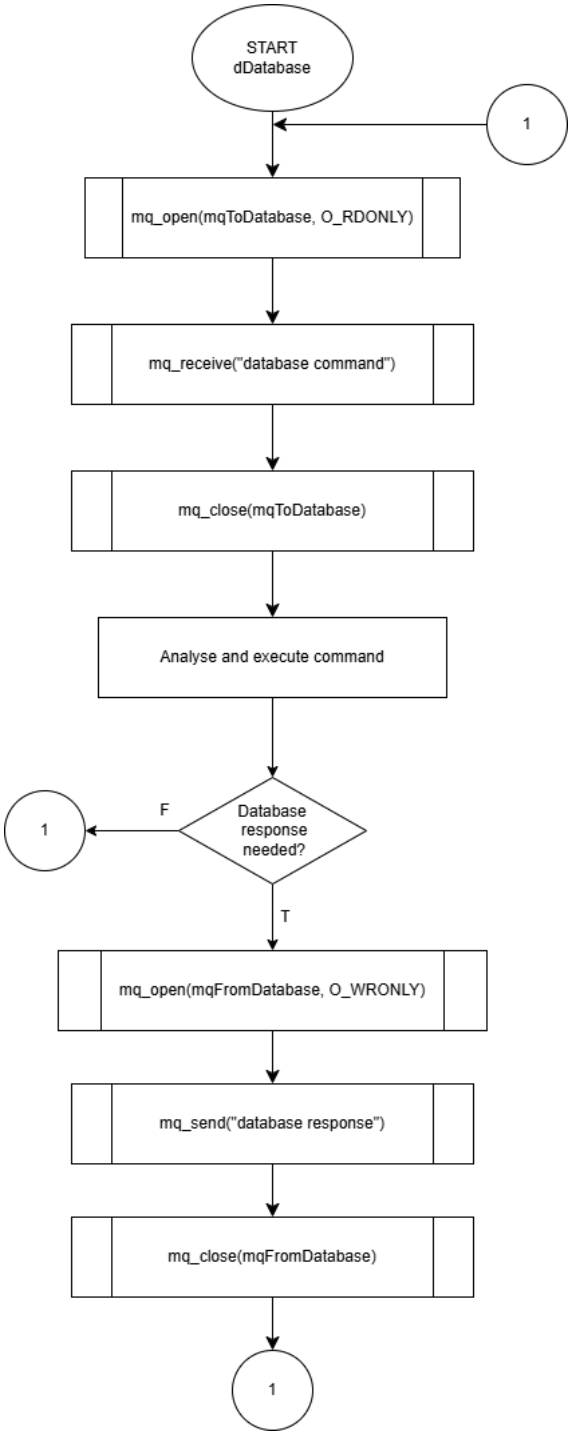


Figure 47. dDatabase

6.8. Inter-process communications and thread synchronization

The following diagram shows all communication flows between processes (via message queues) and between threads (via condition variables and mutexes), as well as the hardware/software signals used to trigger thread execution.

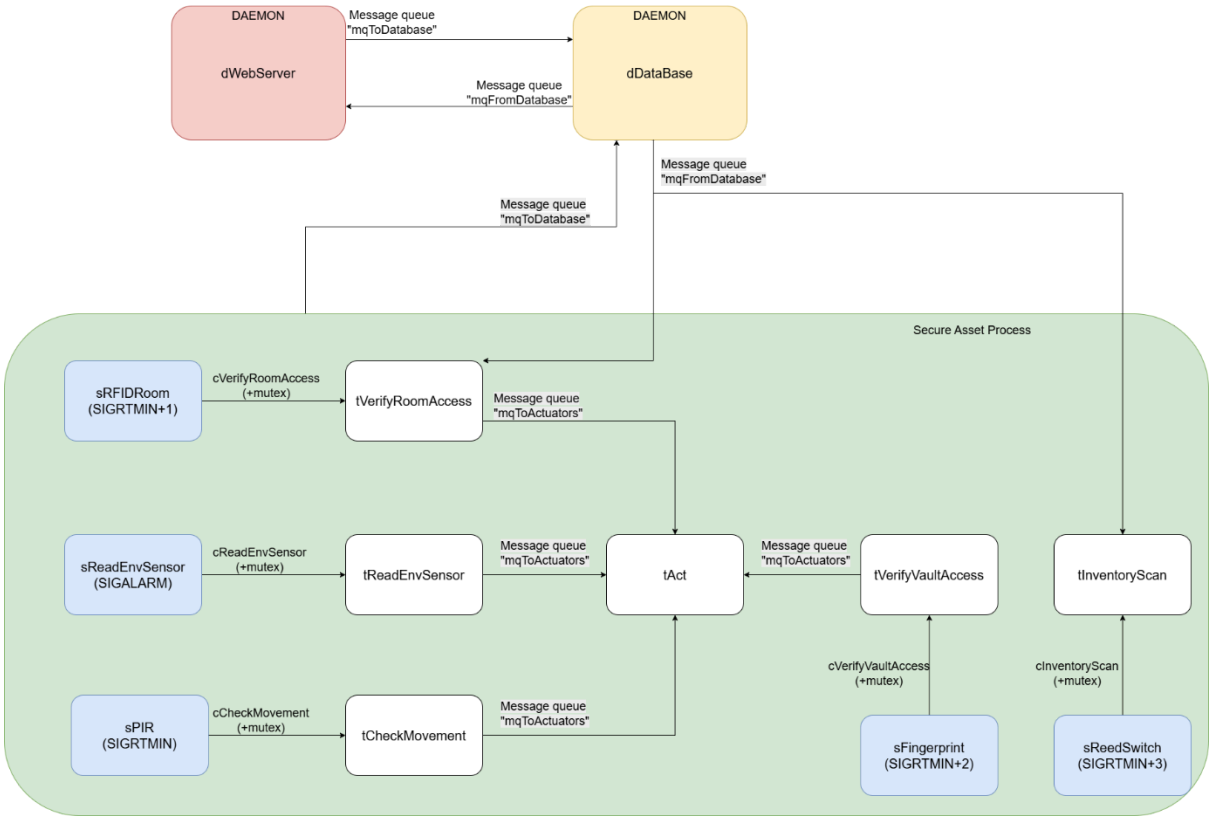
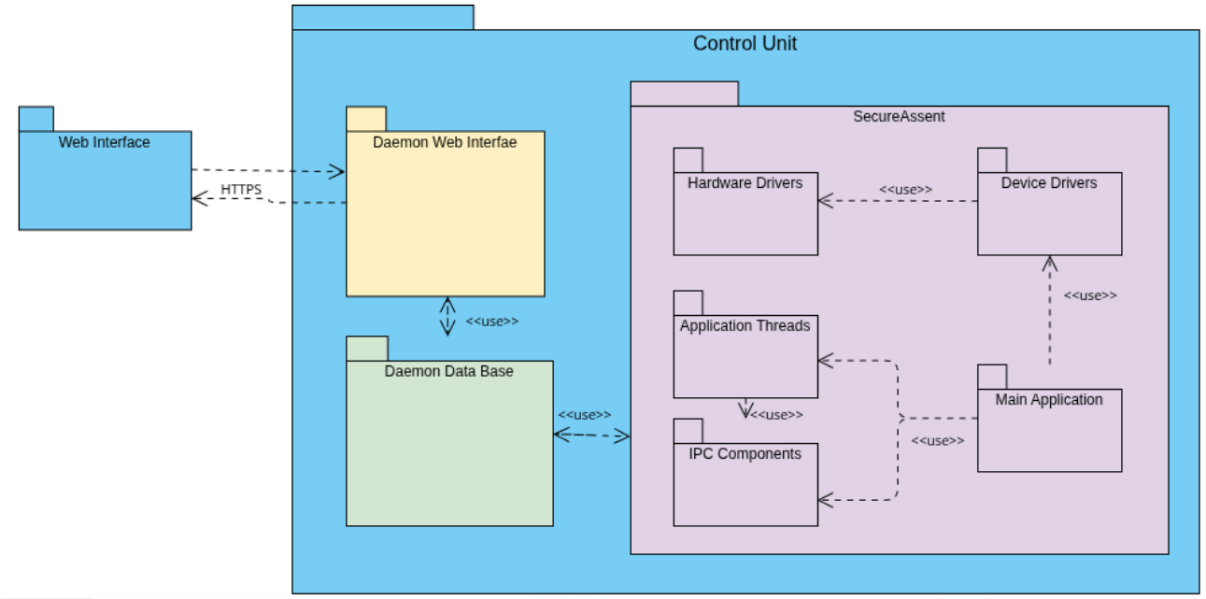


Figure 48. Thread interaction diagram.

Package Diagram



The Package Diagram illustrates the high-level organization of the system's source code, grouping related classes and components into logical namespaces. This structure promotes modularity and clearly separates the user interface from the embedded control logic.

The system is divided into two primary operational domains: the **Web Interface** (client-side) and the **Control Unit** (server-side/embedded).

1. Control Unit Components the Control Unit is the Raspberry Pi, that havef three independent processes that communicate via Inter-Process Communication (IPC):

- **Daemon Web Interface:** Manages external HTTPS requests from the user and acts as a bridge to the database.
- **Daemon Data Base:** A dedicated process for handling all persistent storage operations, ensuring data integrity and serving requests from both the web interface and the main application.
- **SecureAsset:** This is the core embedded application containing the system's business logic.

2. SecureAsset Internal Structure The *SecureAsset* package is further stratified into layers to decouple hardware details from application logic:

- **Hardware & Device Drivers:** The bottom layer consists of the *Hardware Drivers* (abstracting low-level protocols like I2C, UART, PWM) and *Device Drivers* (specific implementations for sensors and actuators).
- **IPC Components:** Provides the necessary wrappers for message queues and mutexes/condition variables, facilitating thread synchronization.

- **Application Threads:** Contains the specific logic for system tasks (e.g., monitoring environment, verifying access), which rely on the drivers for hardware interaction and IPC components for coordination.
- **Main Application:** The entry point that initializes the system, instantiates the drivers, and launches the application threads.

6.9. Class Diagram

Knowing that the software will be implemented in an object-oriented approach, there is a need to specify the classes that are used and their relations. To achieve this, a class diagram was designed.

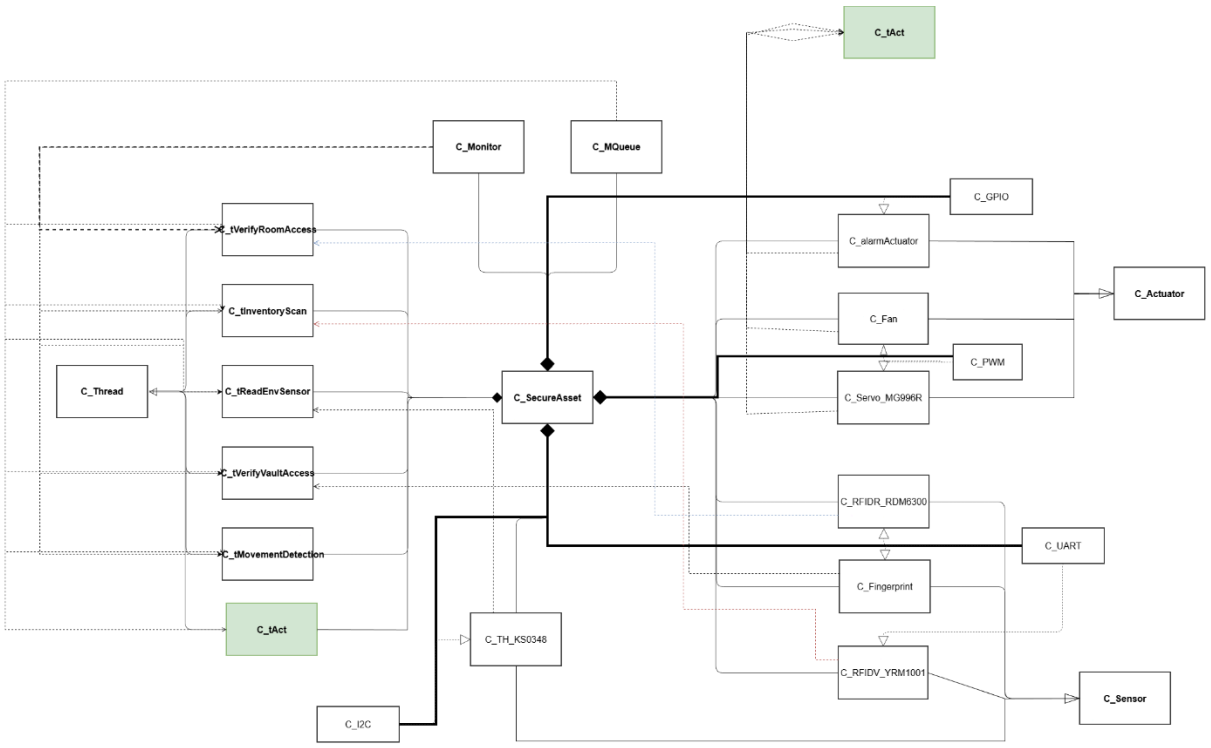


Figure 49. Class diagram.

In this project several groups of classes were implemented, each one with a specific function in the system. The handler classes (C_GPIOHandler, C_I2C, C_PWM, C_UART) abstract the hardware access (GPIO, I2C, PWM and UART). These handlers are used as dependencies by the device classes.

On top of the handlers we have the sensor classes (C_TH_KS0348, C_RFIDR_RDM6300, C_RFIDV_YRM1001, C_Fingerprint) and the actuator classes (C_LED, C_alarmActuator, C_Servo_MG996R).

All actuator classes inherit from the abstract base class C_Actuator, and all sensor classes inherit from C_Sensor.

The mechanism classes C_Monitor (thread synchronization) and C_MQueue (message queues) are used by the threads to communicate with each other.

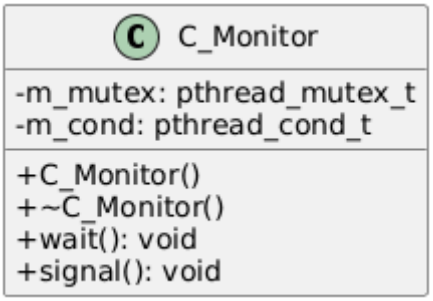
The system logic is executed by the thread classes (C_VerifyRoomAccess, C_InventoryScan, C_ReadEnvSensor, C_VerifyVaultAccess, C_MovementDetection, C_tAct), which all inherit from the abstract base class C_Thread.

Finally, the main class C_SecureAsset has a composition relationship with all concrete classes (sensors, actuators, handlers, mechanism classes), because it creates and owns them.

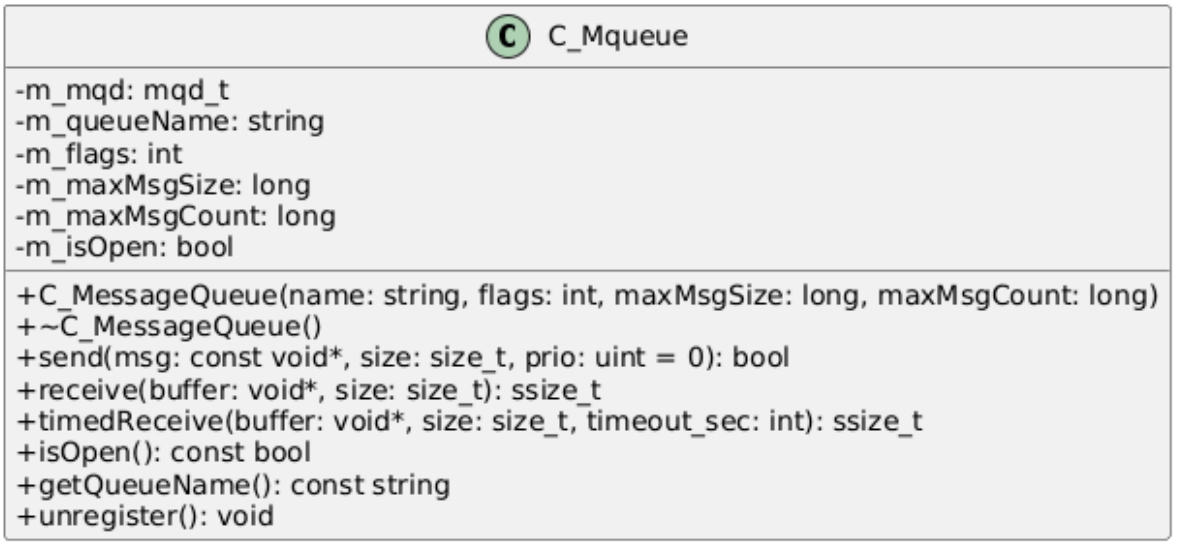
C_Sensor, C_Actuator and C_Thread are not composed, they are base abstract classes, used only to provide inheritance and virtual interfaces to their derived classes.

To give a more detailed look on the clases, they were designed the specif UML diagrams:

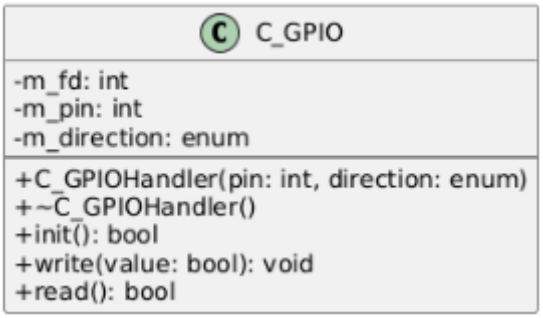
6.9.1. C_Monitor



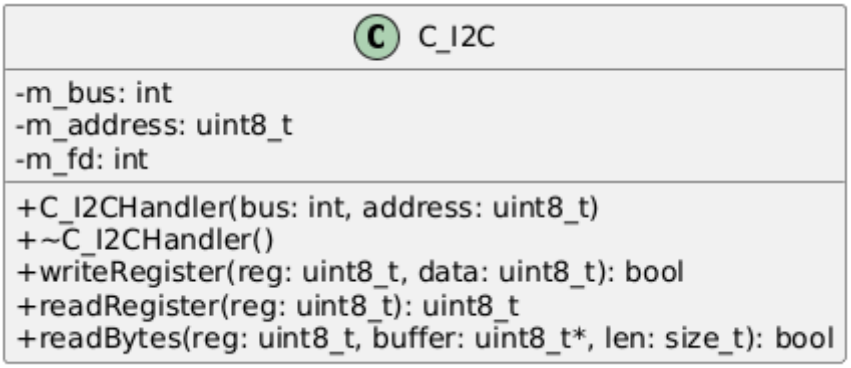
6.9.2. C_Mqueue



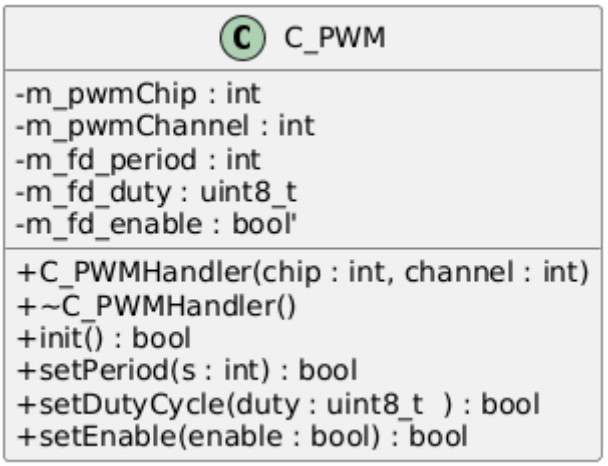
6.9.3. C_GPIO



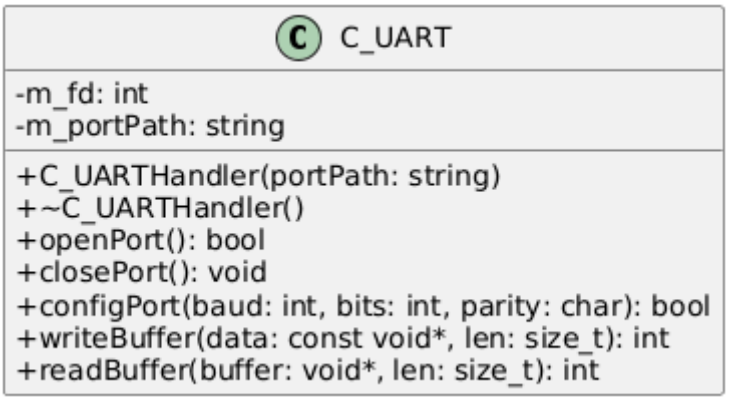
6.9.4. C_I2C



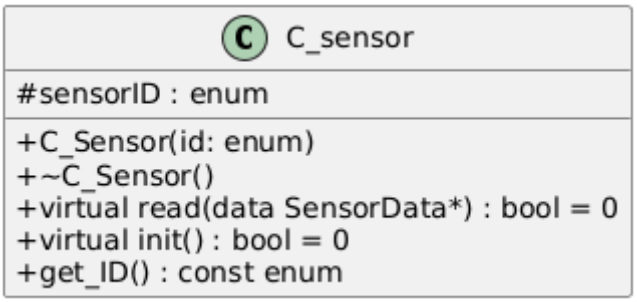
6.9.5. C_PWM

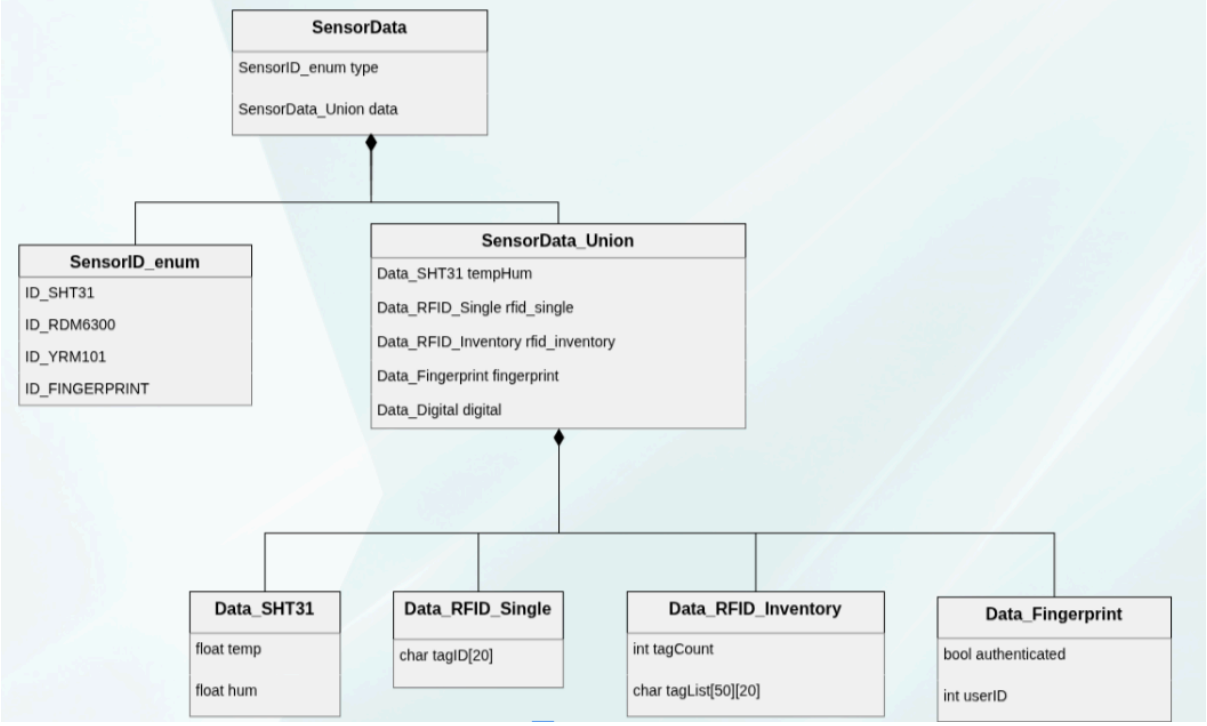


6.9.6. C_UART

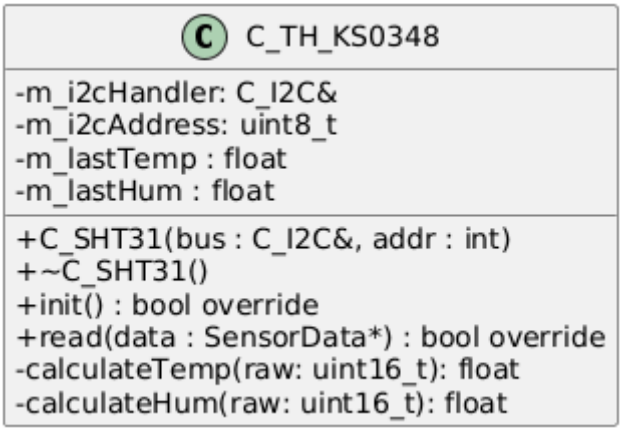


6.9.7. C_SENSOR

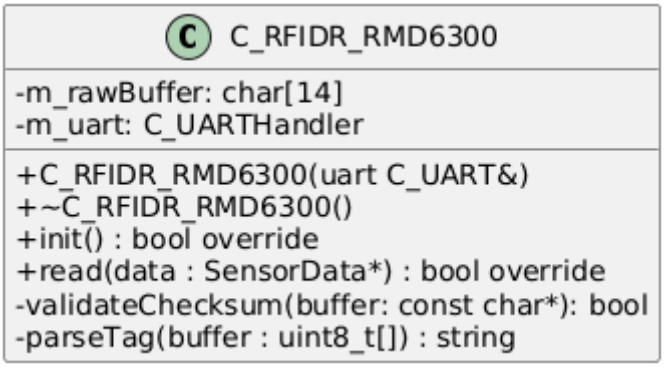




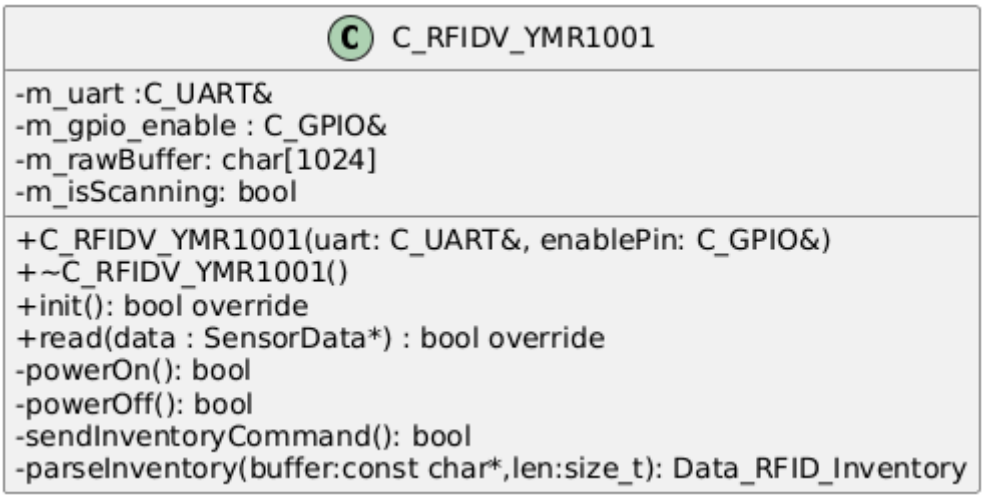
6.9.8. C_TH_KS0348



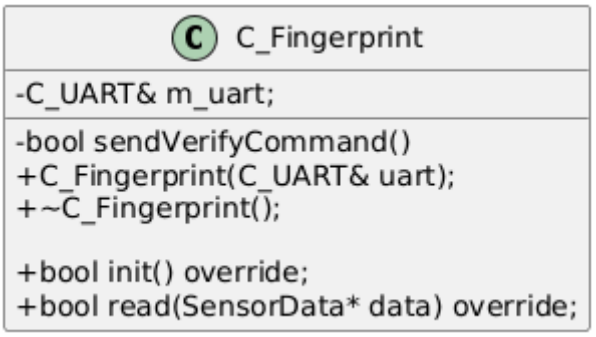
6.9.9. C_RFIDR_RMD6300



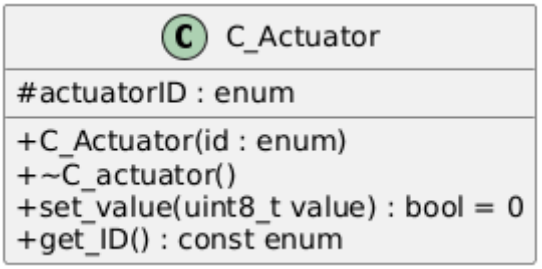
6.9.10. C_RFIDV_YMR1001



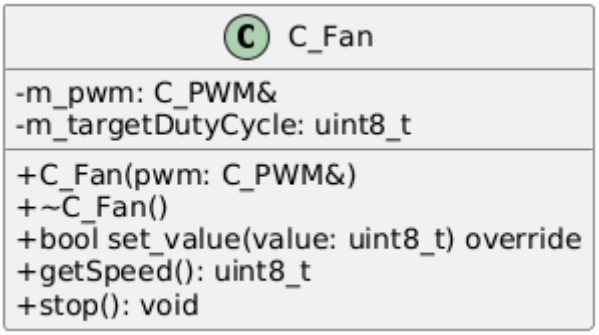
6.9.11. C_Fingerprint



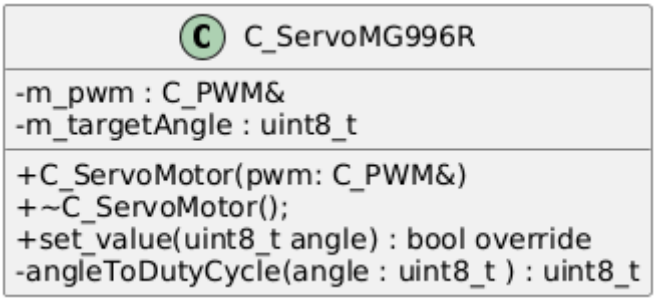
6.9.12. C_Actuator



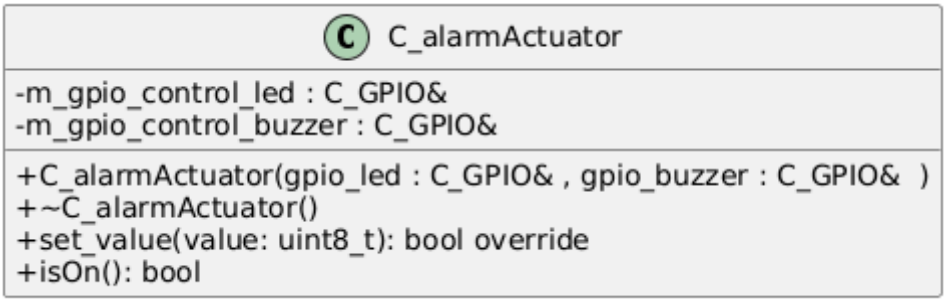
6.9.13. C_Fan



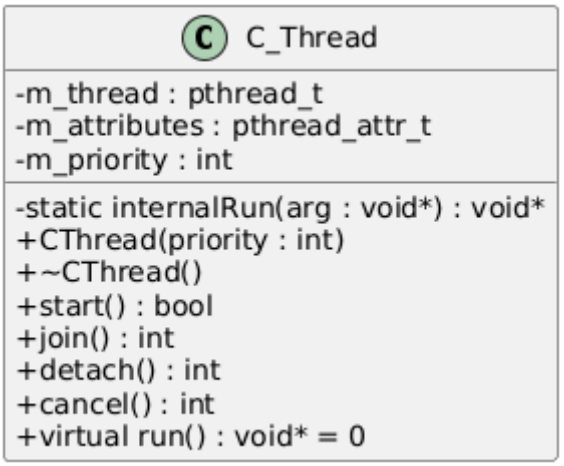
6.9.14. C_ServoMG996R



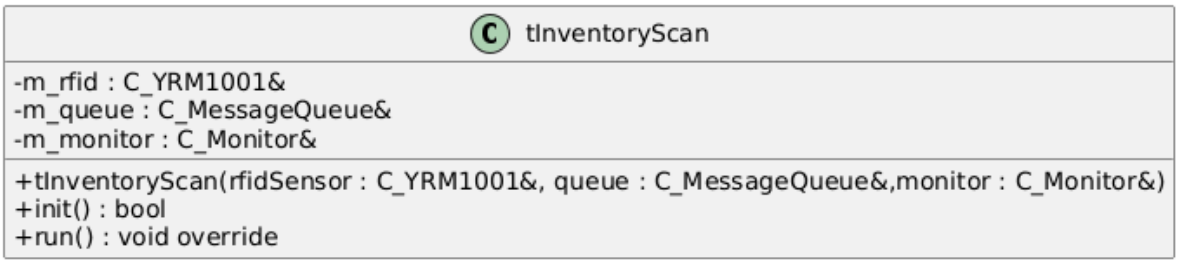
6.9.15. C_alarmActuator



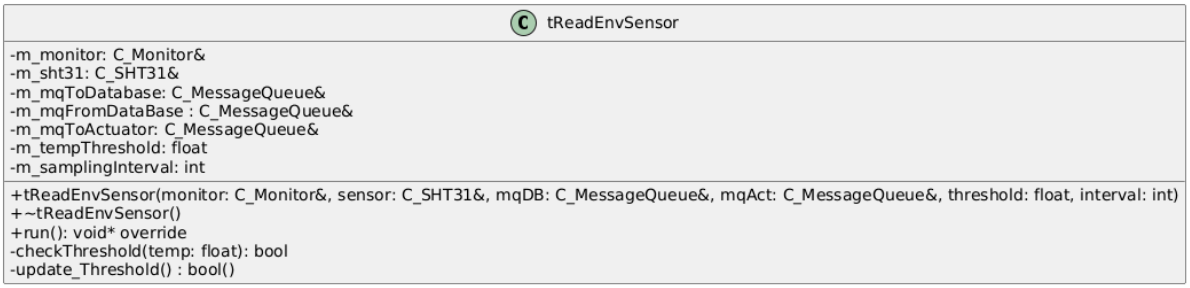
6.9.16. C_Thread



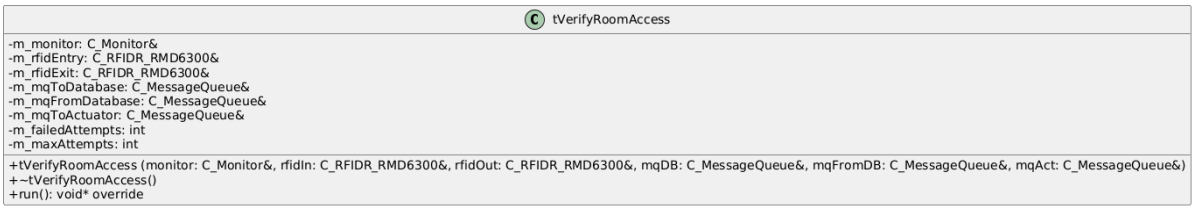
6.9.16. C_tInventoryScan



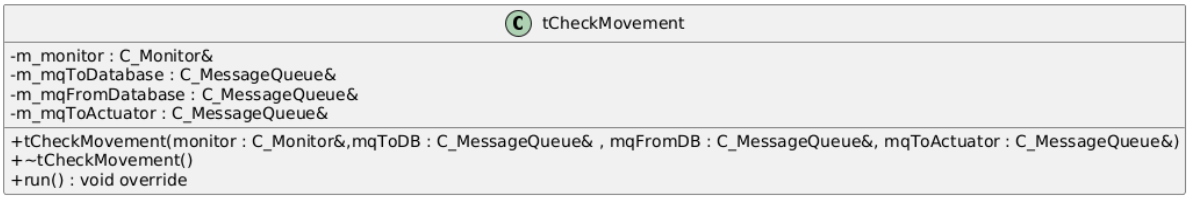
6.9.17. C_tReadEnvSensor



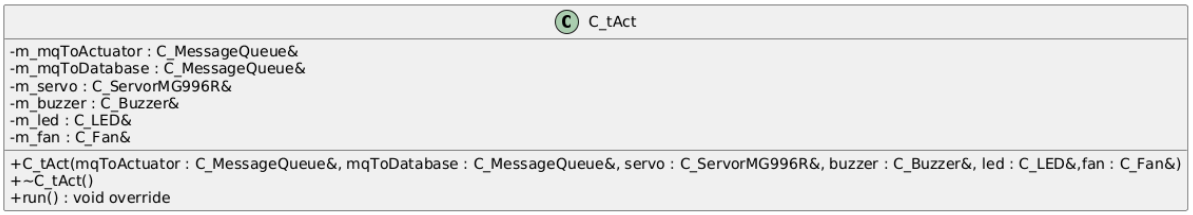
6.9.18. C_tVerifyRoomAccess



6.9.19. C_tCheckMovement

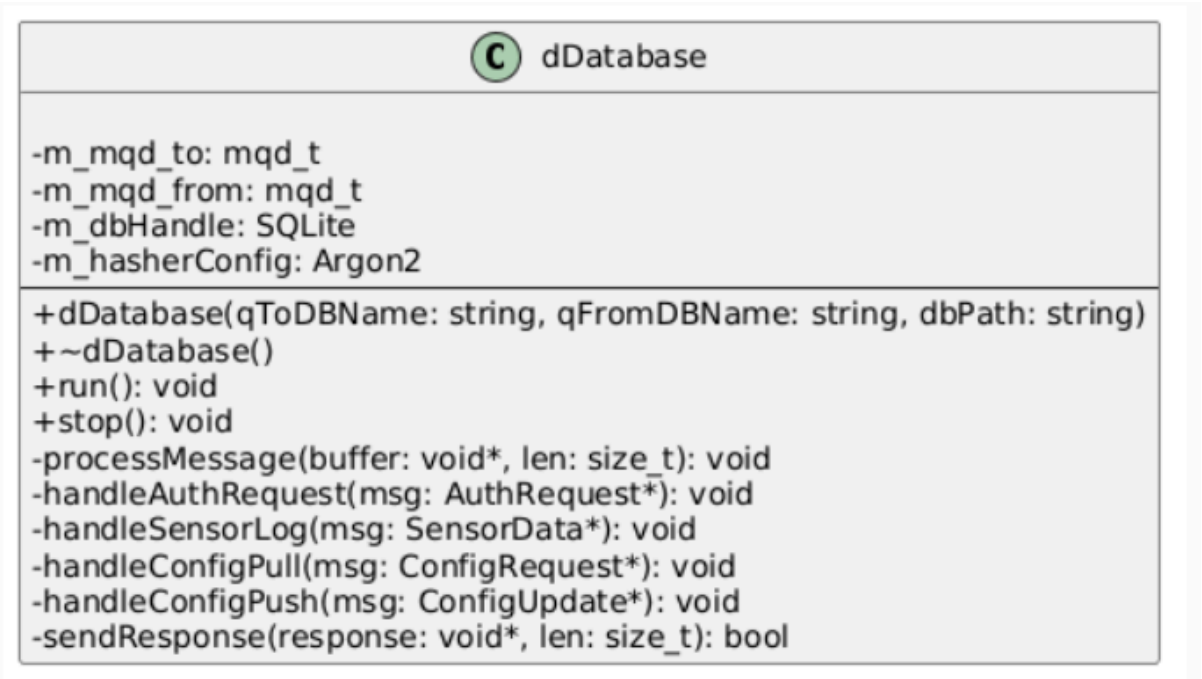


6.9.20. C_tAct



6.9.21. C_dWebServer

6.9.22. C_dDatabase



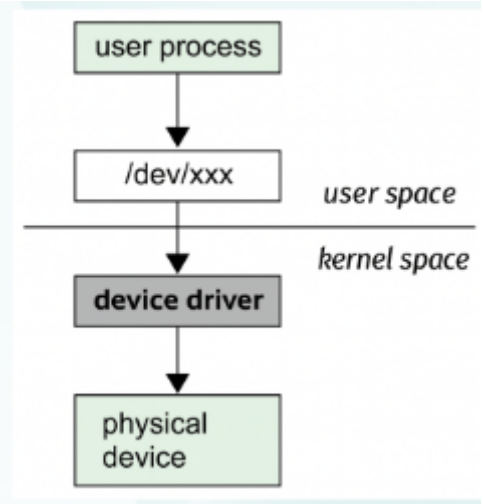
6.9.23. C_SecureAsset

6.10Linux Device Driver

Device drivers are software components that establish communication between the operating system and the hardware devices. They initialize and configure hardware, ensuring it is ready to be used by the operating system and applications.

In the Secure Asset Guard project, we are implementing a custom character-type device driver for the Raspberry Pi, specifically designed to handle **hardware interrupts**. Unlike a standard driver used for simple output control, this driver acts as an event-driven monitor for critical security inputs, such as the PIR motion sensors, Reed switches and activation pins for both RFID and fingerprint sensors.

Functionally, the driver is designed to detect state transitions on specific GPIO pins. When a sensor is triggered (e.g., a door is opened or movement is detected), the driver intercepts the hardware interrupt and immediately notifies the user-space application using **Real-Time Signals**. This design choice is fundamental for a security-focused system: it eliminates the need for the main application to constantly poll the sensors, significantly reducing CPU usage while ensuring a near-instantaneous response to any security breach. By shifting the detection logic to the kernel level, the system can "wake up" the necessary threads only when a relevant physical event occurs, ensuring high efficiency and reliability.



6.11 Startup and Shutdown Processes

During the design phase, the startup and shutdown processes were defined to ensure that the system transitions between states in a controlled and predictable way. A structured lifecycle reduces the risk of inconsistent runtime states, avoids leaving communication resources allocated after termination, and supports clean restarts.

Startup was designed to be coordinated by a single launcher process. The launcher is responsible for preparing the execution environment and then starting the system services in a defined order. The services run as background daemons, each dedicated to a specific subsystem: database management, web/API interface, and core system logic. The design assumes that a service should only be treated as available after it has completed its own internal initialization, so startup is sequential and validated rather than relying on timing assumptions.

Shutdown was designed to be ordered and bounded. When termination is requested, the launcher initiates a controlled shutdown sequence that stops the services in a dependency-aware order. The goal is to prevent the core logic from generating new work during shutdown, stop external interactions through the web interface, and terminate the database service last to minimize the risk of incomplete operations. After services terminate, the launcher completes cleanup to ensure no runtime artifacts remain that could interfere with the next execution.

This design choice leads naturally to a supervisor-plus-daemons structure: the launcher provides centralized lifecycle control, while each daemon remains focused on its subsystem responsibilities and runs without direct user interaction.

6.11.1 Daemons

The system services were designed as independent daemons to keep responsibilities separated and to allow each subsystem to run continuously without user interaction. Three daemons are planned: a database daemon to manage persistent data and answer requests, a web server daemon to provide the external interface, and a core daemon to execute the main system logic. This separation improves modularity because each daemon can be started, monitored, and stopped as an independent service, while the launcher coordinates the overall lifecycle and ensures the system behaves consistently during startup and shutdown.

6.12 Test Cases

The test strategy provides a framework for assessing the system, ensuring alignment with its quality and performance goals. This section outlines test cases that validate individual components by defining expected outcomes for various scenarios. These cases are critical during the design phase, enabling thorough component evaluation and simplifying issue resolution during implementation.

6. 12 .1 Power Supply Testing

Table 4. Power Supply Testing

Test case	Description	Expected Result	Real Result
Power Supply voltage	Apply mains power to the 12V 6A PSU. Measure the output voltage with a multimeter.	Output voltage is stable and within ±5% of 12V DC.	
3.3V Converter Test	Measure DC-DC converter output voltage for sensors of 3.3V	Voltage stable at 3.3V±5%.	
5V Converter Test	Measure DC-DC converter output voltage for sensors of 5V	Voltage stable at 5V±5%	
Logic Level Comm (LLC)	Check if communication between 3.3V and 5V devices function via the converter.	Bidirectional communication established.	

6.12.2 Sensors

Table 5. Sensors testing.

Test case	Description	Expected Result	Real Result
Read Temp/Hum	Check if temperature and humidity values are obtained.	Valid temperature and humidity values.	
Read RFID (125KHz - Entry)	Check if the tag ID is obtained correctly on the designated UART interface.	Tag ID read correctly.	
Read RFID (125KHz - Exit)	Check if the tag ID is obtained correctly on the designated UART interface.	Tag ID read correctly.	
Detect Motion (PIR)	Check if the PIR sensor detects motion.	Motion signal detected.	
Door Status (Reed Switch)	Check if the door's open/closed state is detected.	Door state read correctly.	
Read Fingerprint (Enroll)	Check if the fingerprint enrollment process functions.	Successful enrollment.	
Read Fingerprint (Verify)	Check if an enrolled fingerprint is recognized.	Successful verification.	

Read RFID (UHF)	Check if the UHF tag ID is obtained correctly on the designated UART interface.	UHF tag ID read correctly.	
Control RFID (UHF - Enable)	Check if the UHF module can be enabled/disabled via GPIO.	Module state control functions.	

6.12.3 Actuators

Table 6 Actuators testing.

Test case	Description	Expected Result	Real Result
Actuate Servo (Room)	Check if the room door servo actuates correctly.	Servos move to correct positions. Hold against manual force.	
Actuate Servo (Safe)	Check if the Safe door servo actuates correctly.	Servos move to correct positions. Hold against manual force.	
Actuate Fan	Verify ON/OFF control and speed regulation with PWM	Fan activates on command. Speed varies with PWM if implemented	
Actuate Buzzer	Check if the buzzer emits sound when activated.	Buzzer emits sound.	
Actuate LED	Check if the LED turns on/off when activated/deactivated.	LED turns on and off.	

6.12.4 Database and GUI

Table 7. Database and GUI testing.

Test case	Description	Expected Result	Real Result
Database Connection	Test the system's ability to establish and terminate a connection to the database.	Connection successfully established and terminated.	
SQLCipher Encryption	Test encrypted database with SQLCipher	Database encrypted with AES-256, requires key to open	

Write Operations	Insert access logs, sensor data, and inventory records	Data written correctly with proper timestamps and foreign keys
Read Operations	Test if the user interface can request and display logs from the database.	The interface displays the event logs correctly.
Update Operations	Modify user permissions and inventory status	Records updated correctly. Changes reflected immediately
Web Communication (Interface)	Test the connection from the user interface to the Mongoose server on the Pi.	Web page loads correctly showing system status
Interface Authentication	Attempt to authenticate on the user interface with valid and invalid credentials.	Access is granted for valid credentials and denied for invalid ones.
Data Synchronization (UI)	Verify that an event generated by the hardware (e.g., intrusion) is reflected in the user interface.	The user interface displays the intrusion alert in near real-time.

6.12.5 Integration Tests

Table 9. Integration tests.

Test case	Description	Expected Result	Real Result
Room Access Authentication	Test room access using both a valid and an invalid 125KHz RFID tag on the entry reader.	Door opens for valid tag and access is logged. Door remains closed for invalid tag and access is logged as denied.	
Vault Access & Inventory	Open the vault using a valid fingerprint. Close the vault door (triggering the Reed switch).	Vault door opens and access is logged. Upon closing, the UHF inventory scan runs and the database is updated.	
Intrusion Detection System	With the room door closed (Reed switch),and no authorized person inside trigger the PIR motion sensor.	Alarms (Buzzer/LED) activate. Intrusion is logged in the database and sent to the GUI.	
Environmental Control System	Increase the temperature near the SHT31 sensor above the defined threshold.	The Fan is activated automatically. The environmental alert is logged.	

Remote User Management	Use the remote interface to add/remove a new user's RFID tag and fingerprint.	The new user is added, successfully authenticate at the room and vault.

Dry Run

A dry run is an essential analytical method used to validate a system's logic by simulating its behavior without live execution. This technique allows designers to manually trace an algorithm's execution path, step-by-step, to examine its data flow, logical branches, and resulting outputs.

The primary purpose is to identify potential design flaws, confirm that system requirements are met, and validate the algorithm's correctness before implementation begins. This process is crucial for minimizing subsequent development errors and ensuring the system's final reliability.

In this case, the dry run will focus on the core access control algorithm within the tVerifyRoomAccess thread. This logic is critical, as it must correctly handle a combination of hardware inputs (Tag Validity, Door Status) and internal state (Failed Attempts) to determine whether to grant access or trigger an alarm. The following table will, therefore, demonstrate how the system reacts to these varied input scenarios, allowing for the observation of its state logic under different user interactions.

Line	Tag valid	Door Open	Attempts >= 3	Unlock Door	Trigger Alarm
1	0	0	0	0	0
2	1	0	0	1	0
3	0	0	1	0	1
4	1	0	1	1	0
5	1	1	0	0	0

7. Theoretical Introduction

7.1. Capacitive Biometric Sensing

For the vault's biometric authentication, the *Secure Asset Guard* system employs a **Capacitive Fingerprint Sensor (Waveshare UART Type D)**. This technology was deliberately chosen over common optical alternatives (like the AS608 module) due to its vast superiority in both security and system efficiency.

Unlike optical sensors, which capture a simple 2D image and are notoriously vulnerable to "spoofing" with high-resolution photographs or gelatin molds, a capacitive sensor measures the electrical properties of a living finger. The sensor is a complex array of micro-capacitors; the ridges of a fingerprint make direct contact, while the valleys create a tiny air gap, resulting in a unique, measurable electrical signature. This method implicitly detects "liveness," as artificial replicas lack the specific conductive properties of human tissue, making it the appropriate and secure choice for protecting high-value assets.

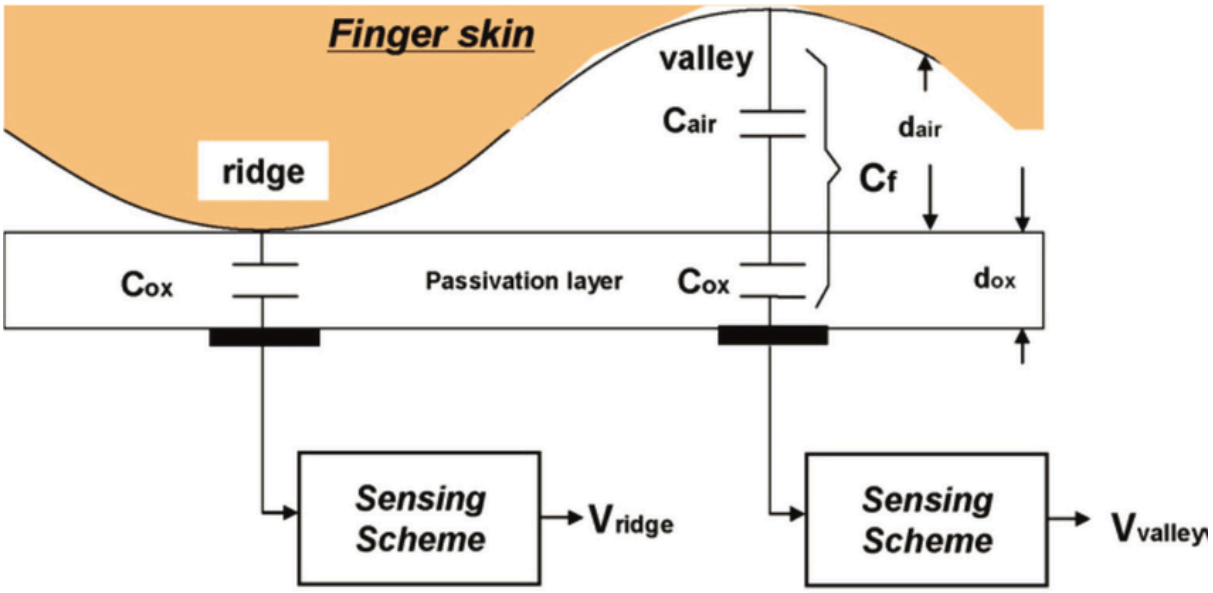


Figure 50 Fingerprint Ridge and Valley Detection.

Beyond its security, the module's design is highly efficient for an embedded system. It is not a simple sensor but a fully integrated "all-in-one" module, featuring its own embedded processor and commercial matching algorithm. This architecture is crucial as it frees the host Raspberry Pi from the CPU-intensive tasks of image processing and template matching.

This integrated intelligence also enables an"auto-wake" architecture. The module features an embedded human sensor, allowing it to rest in an ultra-low-power **sleep mode (drawing < 16uA)**. When a finger is detected, it wakes instantly and notifies the Raspberry

Pi via a dedicated **WAKE pin**. This feature allows the tAccessControl software thread to be entirely interrupt-driven, remaining dormant until a user initiates an action, rather than relying on an inefficient polling loop that wastes CPU cycles.

Communication with the Raspberry Pi is streamlined via a simple **UART protocol** (3.3V logic). The Pi sends high-level commands (e.g., VERIFY_FINGERPRINT, ENROLL_FINGERPRINT) and receives simple, definitive status codes (e.g., MATCH: user_id=5, ACK_NOUSER). In summary, the capacitive sensor's robust security, on-board processing, and power-efficient, interrupt-driven design make it the optimal technical solution for this project.

7.2. Radio Frequency Identification (RFID) Technology

Radio Frequency Identification (RFID) is a wireless technology used to identify objects using radio waves. All RFID systems consist of two main components: a reader (transceiver) and a tag (transponder). However, RFID is not a single technology; it operates across different frequency bands, each with distinct physical operating principles that define its capabilities. The Secure Asset Guard system deliberately employs two different RFID technologies to address two fundamentally different requirements: secure access and efficient inventory management.

Low-Frequency (LF) RFID operates via inductive coupling. For the secure room access control, the system uses the 125 kHz Low-Frequency (LF) RDM6300 module. The reader's antenna generates a short-range magnetic field (typically 1-10 cm). When a passive LF tag enters this field, the field's energy is captured by a coil in the tag, which powers up the tag's microchip. The tag then sends its ID back to the reader, not by transmitting its own radio signal, but by modulating the magnetic field it is drawing power from. For an access control application, this short-range, low-power mechanism is a critical security feature, not a limitation. It requires deliberate physical action from the user—bringing the card close to the reader—preventing accidental scans from a distance and ensuring only the person at the access point is authenticated.

Ultra-High Frequency (UHF) RFID operates via radio frequency backscatter and is fundamentally different in principle and capability. For the automated inventory scan inside the vault, the UHF YRM1001 module (840-960 MHz) is used. The reader transmits a continuous radio wave over a longer distance (several meters). The tag's antenna harvests energy from this wave to power its chip. To send its ID back, the tag does not generate its own radio signal. Instead, it changes its antenna's impedance to alternately reflect or absorb the reader's wave. The reader detects these reflections (the "backscatter") and decodes them as the tag's ID. This mechanism allows UHF systems to have much longer read range

and, crucially, to support a complex anti-collision protocol. This protocol enables the reader to "singulate" (identify and list) hundreds of tags in its field almost simultaneously, allowing the system to perform a complete inventory of all tagged assets inside the vault in seconds when the vault door closes.

In summary, this dual-technology approach leverages the specific physical strengths of each RFID band: the secure, proximity-based authentication of LF for access control, and the efficient, long-range, simultaneous multi-tag detection of UHF for rapid inventory management.

7.3. Theoretical Foundations: Embedded Web Communication Architecture

To provide a robust, modern, platform-independent Graphical User Interface (GUI), the *Secure Asset Guard* system leverages a local web communication architecture rather than a compiled, platform-specific application. This architecture is theoretically composed of three main components: the embedded web server, the Application Programming Interface (API), and the data interchange format.

The core of this architecture is the **embedded web server** (such as the Mongoose library). Unlike traditional standalone web servers, an embedded server is a lightweight software library compiled directly *into* the main C++ application. Its primary function is to enable the embedded device—the Raspberry Pi—to act as a self-contained web server, listening for Hypertext Transfer Protocol (HTTP) requests on a specific TCP/IP port on the local network (LAN). This allows any client device on the same network (such as a PC or smartphone) to access the system's interface using a standard web browser, requiring no custom software installation.

While the server's first role is to serve the static files (HTML, CSS, and JavaScript) that define the visual interface, its second and more dynamic role is to expose an **Application Programming Interface (API)**. This API is a formal set of defined endpoints (URLs) that the client's JavaScript can call to interact with the system's backend C++ logic. This design follows **REST (Representational State Transfer)** principles, using standard HTTP methods like GET to retrieve data (e.g., fetch logs or sensor status) and POST to send data or trigger actions (e.g., add a new user or asset). This client-server architecture cleanly separates the presentation layer (the frontend, running in the browser) from the core system logic (the backend, running in C++ on the Pi).

For this communication to be effective, the client and server must exchange data in a mutually understood, structured format. The standard for this is **JSON (JavaScript Object**

Notation). JSON is a lightweight, text-based data-interchange format that is both human-readable and easily parsed by machines. It serves as the standard for all data traveling through the API. The embedded C++ application formats data retrieved from the SQLite database into a JSON string before sending it in an HTTP response. Conversely, the browser's JavaScript bundles user input (e.g., a new user's credentials) into a JSON string, which is sent in the body of an HTTP POST request. The C++ application then parses this JSON to execute the requested action. This combination of an embedded server, a local API, and the JSON data format creates a flexible, modern, and efficient solution for the system's interface.

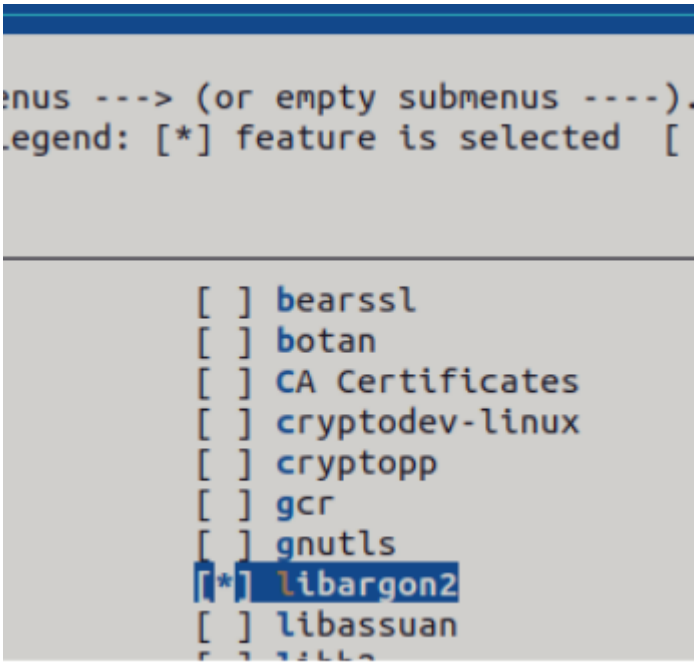
8. Implementation

8.1. Buildroot Image Configuration

To generate the operating system image dedicated to this project, Buildroot was used to create a customized and optimized Linux environment. Several packages were added to provide the necessary libraries and APIs.

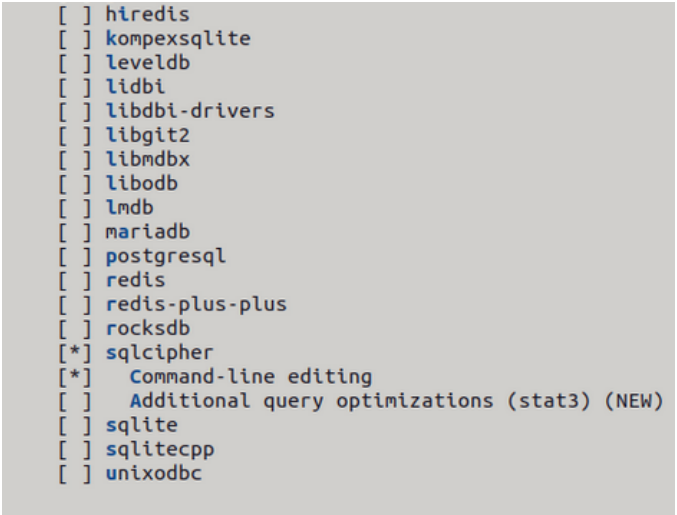
8.1.1. Password hashing

libargon2: Included to provide high-security password hashing using the **Argon2id** algorithm, ensuring that user credentials are not stored in plain text.



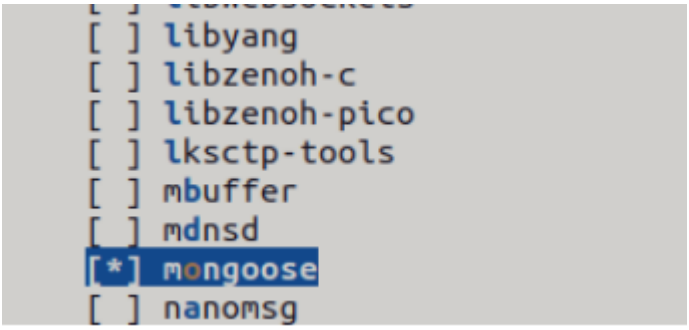
8.1.2. Database

- **sqlite**: The primary database engine implemented to handle the persistent storage of logs, user data, and system configurations.
- **sqlcipher**: An extension of SQLite included to provide AES-256 encryption for the database files, protecting sensitive data from unauthorized access.



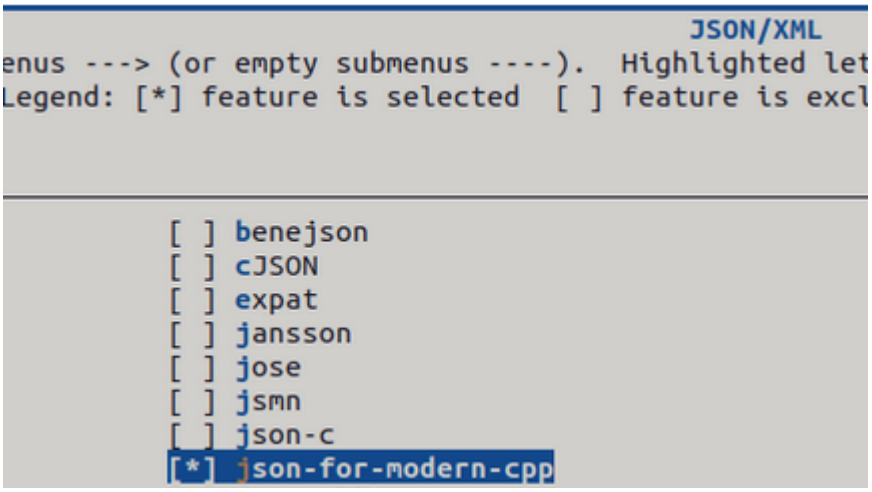
8.1.3. Embedded web server

- **mongoose**: An embedded networking library used to support the dWebServer daemon, enabling the REST API and the graphical dashboard for real-time monitoring.



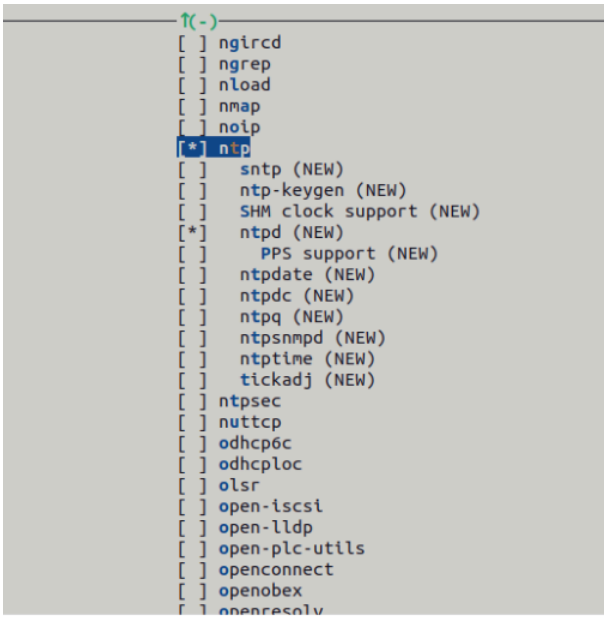
8.1.4. Data Serialization and Exchange (JSON)

- **json-for-modern-cpp**: This library (nlohmann/json) was enabled to facilitate efficient data serialization and communication between the system core and the web interface.



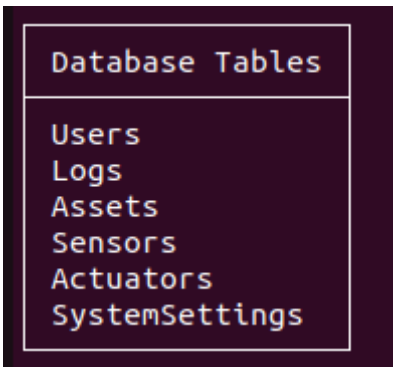
8.1.5. Network Protocols and Synchronization

- **ntp**: Includes components such as ntp and ntpd to synchronize the system clock via the network, ensuring that all log entries and database records have accurate timestamps.



8.2 DataBase

In this project, a database was used to store the values attained by the sensors, as well as the states and operating modes of the actuators and the system's configurations. To achieve this goal, and to make it as easy and intuitive as possible to interpret, retrieve and utilize the stored data, a relational structure was implemented using SQLite. This approach ensures data persistence across system reboots and provides a centralized management point for user permissions, security logs, and asset tracking.



The database is organized into the following tables:

- **Users**: Stores authorized personnel information, including RFID and biometric identifiers, password hashes for web access, privilege levels, and presence status within the room.

```
sqlite> PRAGMA table_info(Users);
```

cid	name	type	notnull	dflt_value	pk
0	UserID	INTEGER	0		1
1	Name	TEXT	0		0
2	RFID_Card	TEXT	0		0
3	FingerprintID	INTEGER	0		0
4	Password	TEXT	0		0
5	AccessLevel	INTEGER	0		0
6	IsInside	INTEGER	0	0	0

- **Logs**: Records all system events and alerts with detailed timestamps and descriptions.

```
sqlite> PRAGMA table_info(Logs);
```

cid	name	type	notnull	dflt_value	pk
0	LogsID	INTEGER	0		1
1	EntityID	INTEGER	0		0
2	Timestamp	INTEGER	0		0
3	LogType	INTEGER	0		0
4	Description	TEXT	0		0
5	Value	INTEGER	0		0
6	Value2	INTEGER	0	0	0

- **Assets**: Manages the inventory of items inside the vault, tracking their specific RFID tags, current location state (Inside/Outside), and the exact time they were last detected.

```
sqlite> PRAGMA table_info(Assets);
```

cid	name	type	notnull	dflt_value	pk
0	AssetID	INTEGER	0		1
1	Name	TEXT	0		0
2	RFID_Tag	TEXT	0		0
3	State	TEXT	0		0
4	LastRead	INTEGER	0		0

- **Sensors**: Maintains the most recent measurements from the environmental sensors along with their respective last update timestamps.

```
sqlite> PRAGMA table_info(Sensors);
```

cid	name	type	notnull	dflt_value	pk
0	SensorID	INTEGER	0		1
1	Type	TEXT	0		0
2	Value	INTEGER	0		0
3	LastUpdate	INTEGER	0	0	0

- **Actuators**: Tracks the current operating state and last activation time of hardware components, including servos, fans, and alarm systems.

```
sqlite> PRAGMA table_info(Actuators);
```

cid	name	type	notnull	dflt_value	pk
0	ActuatorID	INTEGER	0		1
1	Type	TEXT	0		0
2	State	INTEGER	0		0
3	LastUpdate	INTEGER	0	0	0

- **SystemSettings**: Stores global system parameters, specifically the temperature threshold for automatic cooling and the sampling intervals for sensor data acquisition.

```
sqlite> PRAGMA table_info(SystemSettings);
```

cid	name	type	notnull	dflt_value	pk
0	ID	INTEGER	0		1
1	TempThreshold	INTEGER	0	30	0
2	SamplingTime	INTEGER	0	600	0

8.2.1 Database Encryption (SQLCipher)

Because the database contains security-sensitive information (authorized users, access levels, audit logs, and asset tracking), an additional protection layer was implemented through **full database encryption**. For this purpose, the project uses **SQLCipher**, an SQLite-compatible library that provides **transparent AES-based encryption at rest**. As a result, even if the database file is copied from the device, it cannot be opened or interpreted with a standard SQLite tool without the correct encryption key.

During initialization, the system opens the database file normally and immediately applies the encryption key using the SQLCipher API (sqlite3_key). The key is represented as a 32-byte value (256 bits) and is reconstructed at runtime from two constant byte arrays using an XOR operation. This method does not change the key, but it avoids storing it as a readable plain-text string in the binary. After the key is provided to SQLCipher, the temporary key buffer is securely wiped from memory to reduce exposure during runtime.

To ensure compatibility and consistent encryption parameters across tools and deployments, the system relies on SQLCipher 4 defaults, including a 4096-byte page size and PBKDF2-HMAC-SHA512 for key derivation (KDF). With this configuration, the database remains unreadable to standard SQLite applications (which typically return “file is not a database”), while authorized software using the correct key can transparently access all tables and records.

At startup, dDatabase::open() performs sqlite3_open() and immediately applies the SQLCipher key with sqlite3_key(). If a key application fails, the database handle is closed and the system aborts initialization to avoid operating in an unencrypted or inconsistent state. The reconstructed key is stored only in a temporary buffer and is wiped from memory using secureZero() right after sqlite3_key() is called.

Finally, it is important to note that **user passwords are not stored in plaintext**. Web authentication credentials are stored as **Argon2id password hashes** with random salts, providing an additional security layer independent from database encryption. This ensures that even in the unlikely scenario of a key compromise, passwords remain protected against offline cracking attacks.

```
std::array<unsigned char, kDbKeyLength> buildDbKey() {
    std::array<unsigned char, kDbKeyLength> key{};
    for (size_t i = 0; i < key.size(); ++i) {
        key[i] = static_cast<unsigned char>(kDbKeyPart[i] ^
kDbKeyMask[i]);
    }
    return key;
}

void secureZero(void* data, size_t len) {
    volatile unsigned char* p = static_cast<volatile unsigned
char*>(data);
    while (len--) {
        *p++ = 0;
    }
}
```

```
bool dDatabase::open() {
    int result = sqlite3_open(m_dbPath.c_str(), &m_db);

    if (result != SQLITE_OK) {
        std::cerr << "ERRO: Não foi possível abrir a base de dados: "
                  << sqlite3_errmsg(m_db) << std::endl;
        return false;
    }

    std::array<unsigned char, kDbKeyLength> key = buildDbKey();

    if (sqlite3_key(m_db, key.data(), static_cast<int>(key.size())) !=
SQLITE_OK) {
        std::cerr << "ERRO: Falha ao definir chave SQLCipher: "
                  << sqlite3_errmsg(m_db) << std::endl;
```

```
        secureZero(key.data(), key.size());
        sqlite3_close(m_db);
        m_db = nullptr;
        return false;
    }

    secureZero(key.data(), key.size());

    std::cout << "SUCESSO: Ligado ao ficheiro: " << m_dbPath <<
std::endl;
    return true;
}
```

8.2.2 Database Access Layer

Database operations are encapsulated in the dDatabase class, which acts as the single access point to SQLite/SQLCipher. To avoid concurrent access issues and to keep the system modular, database requests are handled through a message-driven interface. The function processDbMessage() receives commands (DB_CMD_*) from other modules via message queues and dispatches them to dedicated handlers (e.g., authentication, logging, dashboard queries, sensor/actuator updates, and asset inventory updates). This design centralizes all persistent state changes and ensures consistent logging of security-relevant events.

```
void dDatabase::processDbMessage(const DatabaseMsg &msg) {
    switch (msg.command) {
        case DB_CMD_ENTER_ROOM_RFID:
            handleAccessRequest(msg.payload.rfid, true);
            break;
        case DB_CMD_LEAVE_ROOM_RFID:
            handleAccessRequest(msg.payload.rfid, false);
            break;
        case DB_CMD_UPDATE_ASSET:
            handleScanInventory(msg.payload.rfidInventory);
            break;
        case DB_CMD_WRITE_LOG:
            handleInsertLog(msg.payload.log);
            break;
        case DB_CMD_USER_IN_PIR:
            handleCheckUserInPir();
            break;
```

```
        case DB_CMD_LOGIN:
            handleLogin(msg.payload.login);
            break;
        case DB_CMD_GET_DASHBOARD:
            handleGetDashboard();
            break;
        case DB_CMD_GET_SENSORS:
            handleGetSensors();
            break;
        case DB_CMD_GET_ACTUATORS:
            handleGetActuators();
            break;
        case DB_CMD_REGISTER_USER:
            handleRegisterUser(msg.payload.user);
            break;
        case DB_CMD_GET_USERS:
            handleGetUsers();
            break;
        case DB_CMD_CREATE_USER:
            handleCreateUser(msg.payload.user);
            break;
        case DB_CMD_MODIFY_USER:
            handleModifyUser(msg.payload.user);
            break;
        case DB_CMD_REMOVE_USER:
            handleRemoveUser(msg.payload.userId);
            break;
        case DB_CMD_GET_ASSETS:
            handleGetAssets();
            break;
        case DB_CMD_CREATE_ASSET:
            handleCreateAsset(msg.payload.asset);
            break;
        case DB_CMD_MODIFY_ASSET:
            handleModifyAsset(msg.payload.asset);
            break;
        case DB_CMD_REMOVE_ASSET:
            handleRemoveAsset(msg.payload.asset.tag);
            break;
        case DB_CMD_GET_SETTINGS:
            handleGetSettings();
            break;
        case DB_CMD_GET_SETTINGS_THREAD:
            handleGetSettingsForThread();
            break;
        case DB_CMD_UPDATE_SETTINGS:
```

```
            handleUpdateSettings(msg.payload.settings);
            break;
        case DB_CMD_FILTER_LOGS:
            handleFilterLogs(msg.payload.logFilter);
            break;
        default: ;
    }
}
```

8.3 Web Interface Implementation

To provide a remote and intuitive way to monitor and manage the system, a custom web interface was developed. Unlike standard dashboard tools, this interface was built from the ground up to ensure full integration with the system's daemons and to provide specific administrative functionalities. The front end consists of a responsive web application served by the **dWebServer** daemon, utilizing HTML5, CSS3, and JavaScript.

```
bool dWebServer::start() {
    std::string addr = "http://10.42.0.163:" + std::to_string(m_port) + "/";

    if (mg_http_listen(&m_mgr, addr.c_str(), eventHandler, this) == nullptr) {
        std::cerr << "[WebServer] Failed to start on port " << m_port <<
std::endl;
        return false;
    }

    m_running = true;
    std::cout << "[WebServer] Listening on " << addr << std::endl;
    return true;
}

void dWebServer::stop() {
    m_running = false;
}

void dWebServer::run() {
    while (m_running) {
        mg_mgr_poll(&m_mgr, 1000);
        cleanExpiredSessions();
    }
}
```

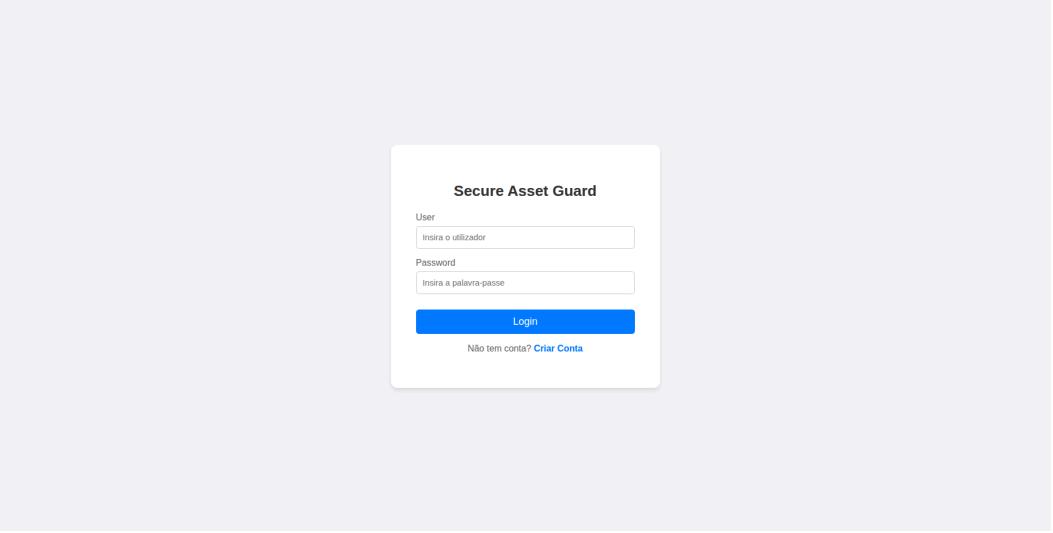
8.3.1 Web Server Architecture (dWebServer)

The communication between the user and the system is managed by a dedicated C++ web server daemon. This daemon acts as a bridge, handling HTTP requests from the browser and communicating with the **dDatabase** daemon via POSIX Message Queues to retrieve or update data. Data exchange between the front end and the back end is performed using **JSON** (JavaScript Object Notation), ensuring a lightweight and structured flow of information.

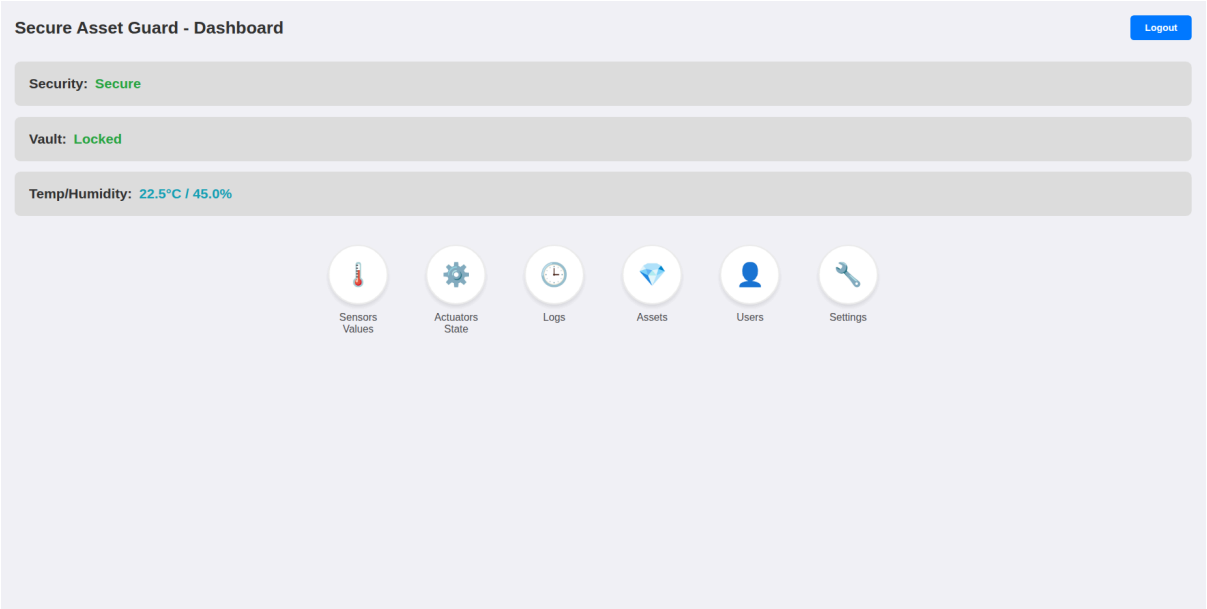
8.3.2 Interface Modules

The interface is divided into specialized modules, each represented by a dedicated page to ensure a clean and organized user experience:

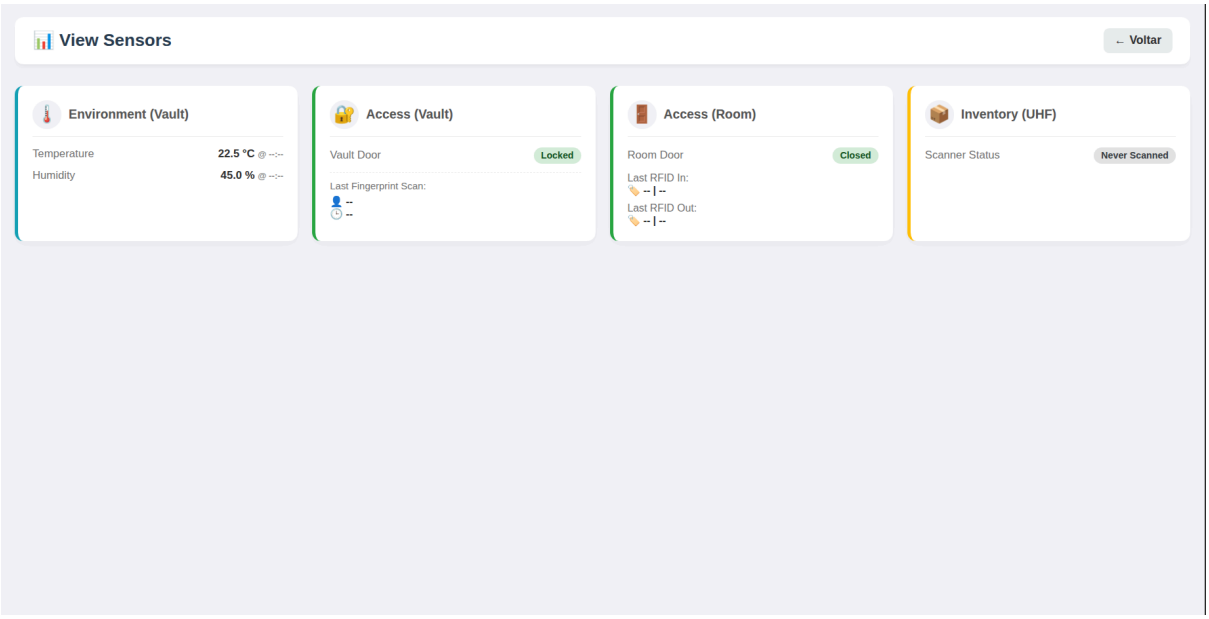
- **Authentication (login.html):** A secure entry point where users must provide credentials. The system validates the session by comparing password hashes using the **Argon2** algorithm in the database.



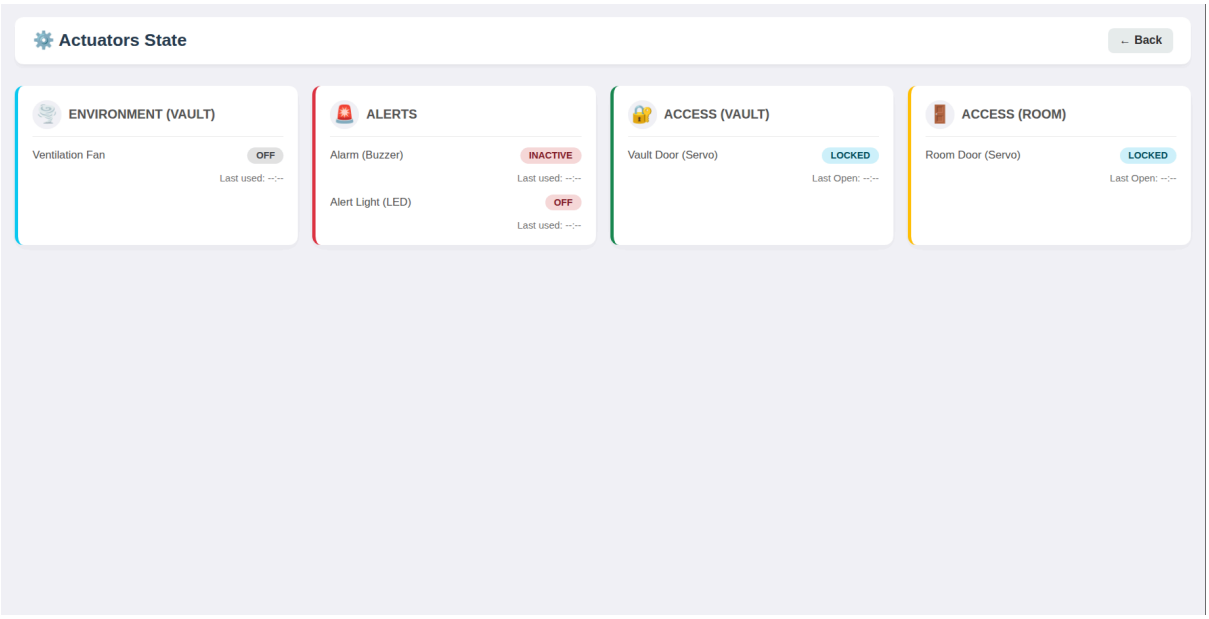
- **Dashboard (dashboard.html):** Provides a high-level overview of the system's status, including security alerts, vault lock state, and real-time environmental metrics (temperature and humidity).



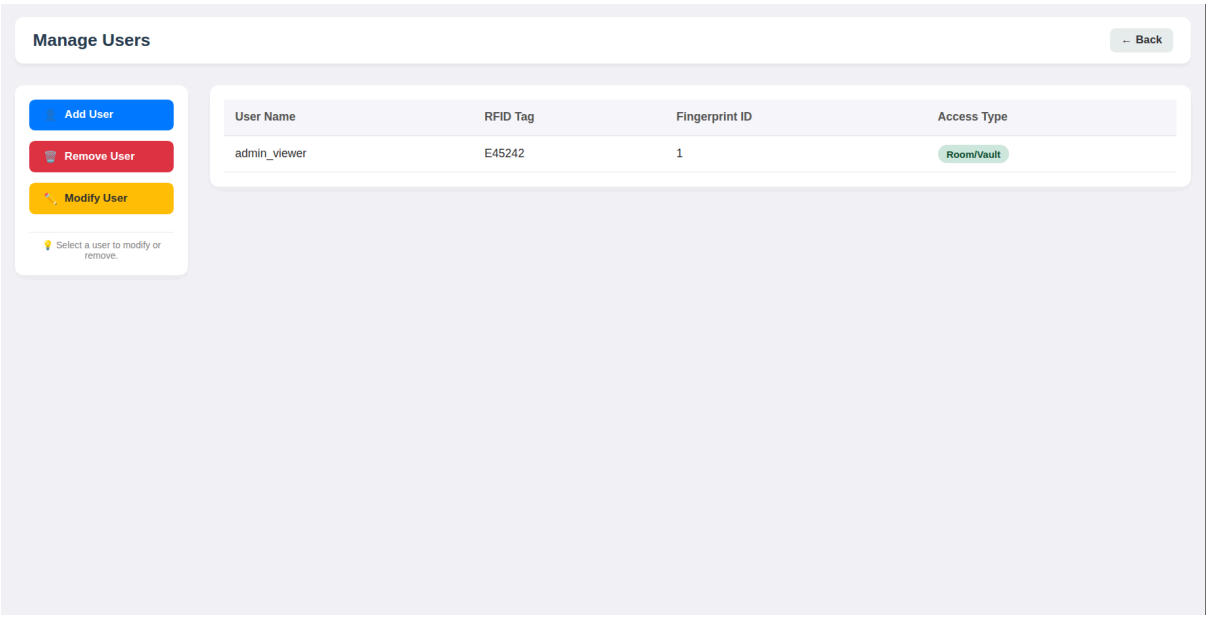
- **Sensor Monitoring (sensors.html):** Displays detailed data from all connected hardware, such as room occupancy, last vault access times, and environmental trends.



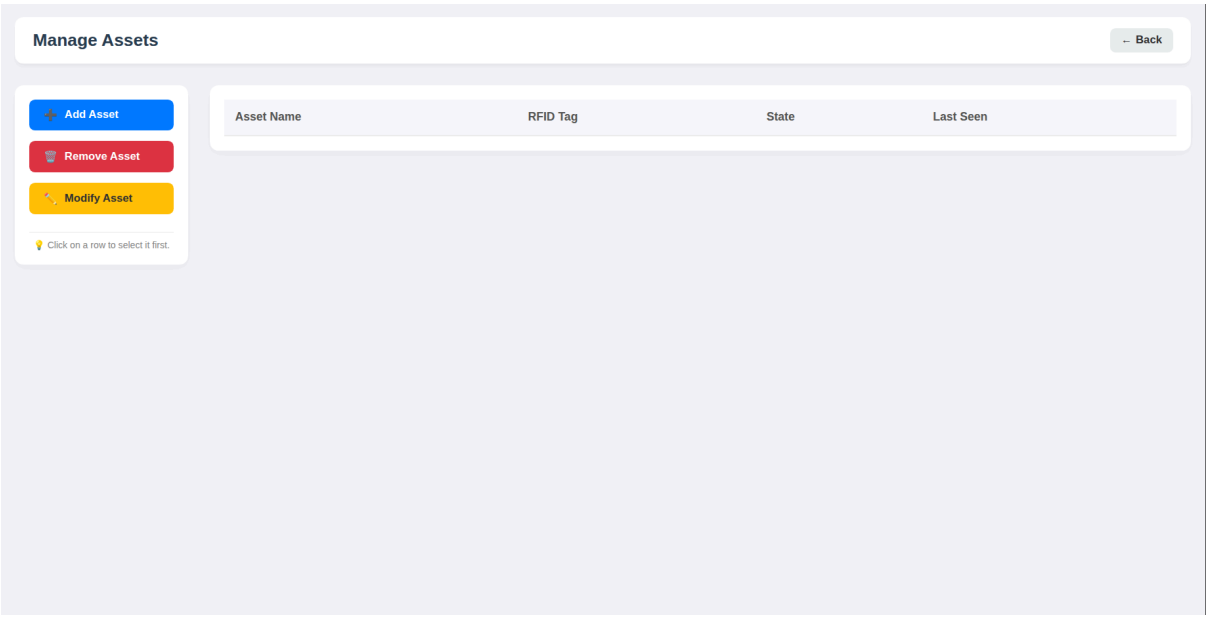
- **Actuator Control (actuators.html):** Allows authorized users to monitor and trigger system components, including the ventilation fan, the alarm system (LED and Buzzer), and the room/vault door servos.



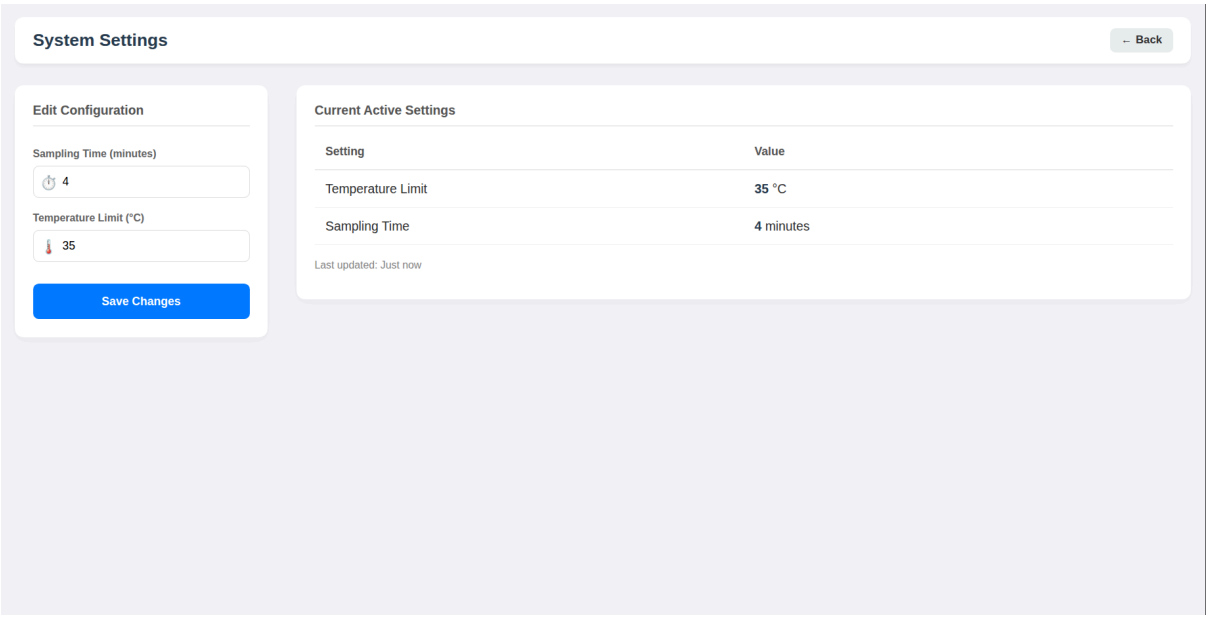
- **User Management (users.html):** An administrative module to list, create, modify, or remove authorized personnel, managing their RFID cards and biometric profiles.



- **Asset Tracking (assets.html):** Displays the current inventory of the vault, showing which items are "Inside" or "Outside" and the timestamp of their last detection by the UHF RFID scan.



- **System Settings (settings.html):** Provides a way to configure critical parameters, such as the temperature thresholds for the cooling system and the sampling intervals for sensor readings.



- **Logs (logs.html):** A centralized viewer for all system events, featuring filters by time range and event type (Intrusion, Access, or System) to facilitate security audits.

8.3.3 REST API Endpoints

The communication between the front-end application and the system's core is handled via a RESTful API implemented in the **dWebServer** daemon. Each administrative or

monitoring action triggers a specific HTTP request to the server, which then interacts with the database via Message Queues.

The table below describes the implemented endpoints and their respective HTTP methods:

Endpoint	Method	Description	Access Level
/api/login	POST	Authenticates credentials and starts a session.	Public
/api/dashboard	GET	Retrieves general system status and security state.	Viewer
/api/sensors	GET	Returns real-time environmental and occupancy data.	Viewer
/api/actuators	GET	Returns the current state of all hardware actuators.	Viewer
/api/logs/filter	POST	Retrieves history logs based on time and type filters.	Viewer
/api/users	GET / POST	Lists all users or registers a new hardware user.	Admin
/api/users/{id}	PUT / DELETE	Modifies or removes a specific user profile by ID.	Admin
/api/assets	GET / POST	Lists all inventory or creates a new asset entry.	Admin
/api/assets/{tag}	PUT / DELETE	Modifies or removes a specific asset using its RFID Tag.	Admin
/api/settings	GET / POST	Reads or updates system thresholds and intervals.	Admin

The following implementation of the eventHandler illustrates the routing logic, where each URI is mapped to its respective handler function:

```
void dWebServer::eventHandler(struct mg_connection* c, int ev, void* ev_data) {
   if (ev == MG_EV_HTTP_MSG) {
       struct mg_http_message* hm = (struct mg_http_message*)ev_data;
       dWebServer* self = (dWebServer*)c->fn_data;

       if (matchUri(&hm->uri, "/api/login")) {
           self->handleLogin(c, hm);
       }
       else if (matchUri(&hm->uri, "/api/register")) {
           self->handleRegister(c, hm);
       }
       else if (matchUri(&hm->uri, "/api/logout")) {
           self->handleLogout(c, hm);
       }
       else if (matchUri(&hm->uri, "/api/dashboard")) {
           self->handleDashboard(c, hm);
       }
       else if (matchUri(&hm->uri, "/api/sensors")) {
```

```
            self->handleSensors(c, hm);
        }
        else if (matchUri(&hm->uri, "/api/actuators")) {
            self->handleActuators(c, hm);
        }
        else if (matchUri(&hm->uri, "/api/logs/filter")) {
            self->handleLogsFilter(c, hm);
        }
        else if (matchPrefix(&hm->uri, "/api/assets/")) {
            self->handleAssetsById(c, hm);
        }
        else if (matchPrefix(&hm->uri, "/api/users/")) {
            self->handleUsersById(c, hm);
        }
        else if (matchUri(&hm->uri, "/api/users")) {
            self->handleUsers(c, hm);
        }
        else if (matchUri(&hm->uri, "/api/assets")) {
            self->handleAssets(c, hm);
        }
        else if (matchUri(&hm->uri, "/api/settings")) {
            self->handleSettings(c, hm);
        }
        else if (matchUri(&hm->uri, "/")) {
            mg_http_reply(c, 302, "Location: /login.html\r\n", "");
        }
        else {
            SessionData session;
            bool logado = self->validateSession(hm, session);
            if (matchUri(&hm->uri, "/users.html") ||
                matchUri(&hm->uri, "/settings.html") ||
                matchUri(&hm->uri, "/assets.html")) {
                if (!logado || session.accessLevel < 1) {
                    mg_http_reply(c, 302, "Location: /login.html\r\n", "");
                    return;
                }
            }
            struct mg_http_serve_opts opts;
            memset(&opts, 0, sizeof(opts));
            opts.root_dir = "/root/SecureAsset/web";
            mg_http_serve_dir(c, hm, &opts);
        }
    }
}
```

8.3.4 Security and Session Management

To ensure that only authorized personnel can access sensitive information or trigger actuators, the system implements a robust security layer:

- **Session Tokens:** Upon a successful login, the server generates a unique **32-character hexadecimal token**. This token is stored in a secure cookie and used to identify the user in every subsequent request.
- **Access Level Verification:** Each session is associated with an AccessLevel (0 for Viewer, 1 for Room/Admin). The server explicitly checks these permissions before processing administrative API calls or serving management-specific HTML files like users.html or settings.html.
- **Token Expiration:** Sessions are automatically invalidated after one hour of inactivity to prevent unauthorized reuse of tokens.

8.4 GPIO Interrupt Device Driver

As mentioned previously, the PIR sensors, Reed switches, and the activation pins for the RFID and fingerprint modules generate state transitions on the GPIO pins when triggered. To capture these events, a custom interrupt-driven device driver was developed to notify the application via Real-Time Signals, avoiding the need for constant polling.

8.4.1.GPIO

According to the raspberry pi datasheet, it is necessary to use the registers represented in figure x to to control the GPIO:

Offset	Name	Description
0x00	GPFSEL0	GPIO Function Select 0
0x04	GPFSEL1	GPIO Function Select 1
0x08	GPFSEL2	GPIO Function Select 2
0x0c	GPFSEL3	GPIO Function Select 3
0x10	GPFSEL4	GPIO Function Select 4
0x14	GPFSEL5	GPIO Function Select 5

0xe4	GPIO_PUP_PDN_CNTRL_REG0	GPIO Pull-up / Pull-down Register 0
0xe8	GPIO_PUP_PDN_CNTRL_REG1	GPIO Pull-up / Pull-down Register 1
0xec	GPIO_PUP_PDN_CNTRL_REG2	GPIO Pull-up / Pull-down Register 2
0xf0	GPIO_PUP_PDN_CNTRL_REG3	GPIO Pull-up / Pull-down Register 3

To assist in the development of these device drivers, the first step was developing the functions represented below to configure the pins and to define their state by changing the registers presented before.

```
/**
 * Configures GPIO function.
 * functionCode: 0=Input, 1=Output, 4=Alt0, 5=Alt1, 7=Alt3, 2=Alt4,
3=Alt5.
 */
void SetGPIOFunction(struct GpioRegisters *regs, int pin, int
functionCode) {
    int index = pin / 10;
    int bit = (pin % 10) * 3;

    unsigned oldValue = regs->GPFSEL[index];
    unsigned mask = 0b111 << bit;

    regs->GPFSEL[index] = (oldValue & ~mask) | ((functionCode << bit) &
mask);
}

/**
 * Configures internal Pull-Up/Pull-Down.
 * pud: 0=None (Floating), 1=Pull-Up, 2=Pull-Down.
 */
void SetGPIOPullUpDown(struct GpioRegisters *regs, int pin, int pud) {
    int index = pin / 16;
    int bit = (pin % 16) * 2;

    unsigned int oldValue = regs->PUP_PDN[index];
    unsigned int mask = 0b11 << bit;

    regs->PUP_PDN[index] = (oldValue & ~mask) | ((pud << bit) & mask);
}
```

To manage multiple inputs efficiently, a structure was implemented to map each GPIO pin to its specific hardware and software parameters. This includes the trigger type (rising/falling edge), a unique Real-Time Signal ID and a timestamp for software debouncing. This centralized approach provides an organized and scalable way to handle interrupts from the PIR, Reed switches, and biometric modules, ensuring that the user-space knows exactly which sensor triggered the event.

```
/*
   The SensorConfig structure encapsulates the hardware and software
properties of each security peripheral. It maps physical GPIO pins to
specific Real-Time Signals and maintains state variables for kernel-level
debouncing.
 */

struct SensorConfig {
    int gpio_pin;
    unsigned long trigger;
    int signal_id;
    char *name;
    bool enabled;
    unsigned long last_time;
};

/*
   Initialization of the my_sensors array. This table defines the
configuration for all security inputs, including Reed switches, PIR motion
sensors, and activation pins for the biometric and RFID modules.
 */
static struct SensorConfig my_sensors[] = {
    /
    {22, IRQF_TRIGGER_RISING,  43, "reed_switch_vault", true,0},
    {27, IRQF_TRIGGER_RISING,  44, "reed_switch_room", true,0},
    {17, IRQF_TRIGGER_RISING,  45, "pir_sensor",  true,0},
    {6, IRQF_TRIGGER_RISING,  46, "fingerprint", true,0},
    { 16, IRQF_TRIGGER_FALLING, 47, "rfid_entry", true,0},
    { 20, IRQF_TRIGGER_FALLING, 48, "rfid_left", true, 0}
};
```

8.4.2 Module init function

The init function performs the driver's registration and hardware initialization. It allocates system resources to create six device nodes (/dev/irq0-5) and maps the physical GPIO

registers into the kernel's virtual memory space. Each pin is configured as an input with specialized electrical settings: Pull-Down resistors for the PIR, Reed switches and Fingerprint, and Pull-Up resistors for the RFID modules. The process concludes by requesting IRQ lines and associating them with a dedicated handler.

```
/**
 * Driver initialization function.
 * Handles kernel registration, memory mapping, and hardware IRQ setup.
 */
static int __init irqModule_init(void) {
    int ret, i;
    struct device *dev_ret;

    /* ..........................................................................
       1. REGISTRATION: Allocation of major/minor numbers and device class.
       .......................................................................... */
   if ((ret = alloc_chrdev_region(&irqDevice_majorminor, 0, NUM_IRQS, DEVICE_NAME)) < 0) {
        return ret;
    }

    if (IS_ERR(irqDevice_class = class_create(CLASS_NAME))) {
        unregister_chrdev_region(irqDevice_majorminor, NUM_IRQS); // Cleanup on error
        return PTR_ERR(irqDevice_class);
    }

    /* ..........................................................................
       2. CDEV SETUP: Initializing the character device and adding it to the system.
       .......................................................................... */
    cdev_init(&c_dev, &irqDevice_fops);
    c_dev.owner = THIS_MODULE;

    if ((ret = cdev_add(&c_dev, irqDevice_majorminor, NUM_IRQS)) < 0) {
        class_destroy(irqDevice_class);
        unregister_chrdev_region(irqDevice_majorminor, NUM_IRQS);
        return ret;
    }

    /* ..........................................................................
       3. DEVICE NODES: Creation of /dev/irqX entries with full rollback logic.
       .......................................................................... */
    for (i = 0; i < NUM_IRQS; i++) {
        dev_ret = device_create(irqDevice_class, NULL,
                                MKDEV(MAJOR(irqDevice_majorminor), i),
                                NULL, "irq%d", i);
        if (IS_ERR(dev_ret)) {
            while (i > 0) { // Destroy previously created devices
                i--;
                device_destroy(irqDevice_class, MKDEV(MAJOR(irqDevice_majorminor), i));
            }
            cdev_del(&c_dev);
            class_destroy(irqDevice_class);
            unregister_chrdev_region(irqDevice_majorminor, NUM_IRQS);
            return PTR_ERR(dev_ret);
```

```
        }
    }

    /* ..........................................................................
       4. IO-REMAP: Mapping physical GPIO registers to kernel virtual address space.
       .......................................................................... */
   s_pGpioRegisters = (struct GpioRegisters *)ioremap(GPIO_BASE, sizeof(struct
GpioRegisters));
    if (!s_pGpioRegisters) {
        for (i = 0; i < NUM_IRQS; i++) {
            device_destroy(irqDevice_class, MKDEV(MAJOR(irqDevice_majorminor), i));
        }
        cdev_del(&c_dev);
        class_destroy(irqDevice_class);
        unregister_chrdev_region(irqDevice_majorminor, NUM_IRQS);
        return -ENOMEM;
    }

    /* ..........................................................................
       5. HARDWARE & IRQ CONFIG: Setting pin modes and requesting interrupt lines.
       .......................................................................... */
    for (i = 0; i < NUM_IRQS; i++) {
        struct SensorConfig *sensor = &my_sensors[i];
        int gpio_logical;

        // Configure pin as input mode
        SetGPIOFunction(s_pGpioRegisters, sensor->gpio_pin, 0);

        // Set Pull-Down (2) for sensors 0-3 and Pull-Up (1) for RFIDs 4-5
        if(i != 4 && i != 5) {
            SetGPIOPullUpDown(s_pGpioRegisters, sensor->gpio_pin, 2);
        } else {
            SetGPIOPullUpDown(s_pGpioRegisters, sensor->gpio_pin, 1);
        }

        // Map GPIO to IRQ number (BCM2711 offset: 512)
        gpio_logical = 512 + sensor->gpio_pin;
        irq_numbers[i] = gpio_to_irq(gpio_logical);

        if (irq_numbers[i] < 0) {
            pr_err("irqModule: gpio_to_irq failed for pin %d\n", sensor->gpio_pin);
            while (i > 0) {
                i--;
                free_irq(irq_numbers[i], (void *)&my_sensors[i]);
            }
            iounmap(s_pGpioRegisters);
            for (i = 0; i < NUM_IRQS; i++) device_destroy(irqDevice_class,
MKDEV(MAJOR(irqDevice_majorminor), i));
            cdev_del(&c_dev);
            class_destroy(irqDevice_class);
            unregister_chrdev_region(irqDevice_majorminor, NUM_IRQS);
            return irq_numbers[i];
        }
        pr_alert("irqModule: %s (Pin %d) -> IRQ %d\n", sensor->name, sensor->gpio_pin,
irq_numbers[i]);
        // Request IRQ and associate with the handler and sensor data
```

```
        ret = request_irq(irq_numbers[i], gpio_irq_handler, sensor->trigger, sensor->name,
(void *)sensor);

        if (ret) {
            pr_err("irqModule: request_irq failed for %s\n", sensor->name);
            while (i > 0) {
                i--;
                free_irq(irq_numbers[i], (void *)&my_sensors[i]);
            }
            iounmap(s_pGpioRegisters);
            for (i = 0; i < NUM_IRQS; i++) device_destroy(irqDevice_class,
MKDEV(MAJOR(irqDevice_majorminor), i));
            cdev_del(&c_dev);
            class_destroy(irqDevice_class);
            unregister_chrdev_region(irqDevice_majorminor, NUM_IRQS);
            return ret;
        }
        sensor->enabled = true; // Enable the sensor flag
    }
    return 0;
}
```

8.4.3 Interrupt handler function

The interrupt handler function is the core of the driver's event-driven system. When a sensor is triggered, the handler recovers its specific data through the dev_id pointer. To ensure reliability, it implements a 500ms software debounce using kernel jiffies to filter out mechanical noise. If the event is valid, the driver sends a Real-Time Signal directly to the application.

```
/**
 * Interrupt Handler: Executed when a hardware trigger occurs.
 * Validates the event and notifies user-space via Real-Time Signals.
 */
static irqreturn_t gpio_irq_handler(int irq, void *dev_id) {

    struct kernel_siginfo info;
    // Cast dev_id back to our sensor configuration structure
    struct SensorConfig *sensor_atual = (struct SensorConfig *)dev_id;

    int pino = sensor_atual->gpio_pin;
    int sinal = sensor_atual->signal_id;


/* ..............................................................
   DEBOUNCING LOGIC: Uses jiffies to ignore repetitive triggers
```

```
.....................................................................*/
    unsigned long now = jiffies;

    // Filter out mechanical bounce
    if ((now - sensor_atual->last_time) < msecs_to_jiffies(500)) {
        return IRQ_HANDLED;
    }

    // Update timestamp for the last valid trigger
    sensor_atual->last_time = now;

/*.....................................................................
 SIGNAL DELIVERY: Sends an asynchronous notification to the registered
       process, passing the physical pin number as a payload (si_int).
..................................................................... */
    if (task != NULL) {
        memset(&info, 0, sizeof(struct kernel_siginfo));

        info.si_signo = sinal;      // Signal number (43-48)
        info.si_code = SI_QUEUE;    // Signal sent via queue
        info.si_int = pino;         // Payload: the pin that triggered

        // Send signal to the application (task)
        if (send_sig_info(sinal, &info, task) < 0) {
            pr_err("irqModule: Error sending signal.\n");
        }
    }

    return IRQ_HANDLED; // Interrupt successfully handled
}
```

8.4.4 Module exit function

To safely remove the kernel module, the irqModule_exit function is executed. This function is responsible for performing a complete cleanup of the resources allocated during initialization. It begins by releasing the six interrupt lines, ensuring that the system is no longer listening for hardware triggers, and restores all used GPIO pins to their default input mode. Following this, the driver unmaps the GPIO physical address space, deletes the character device structure, and destroys the individual device nodes (irq0-5) and the device class. Finally, the major and minor numbers are deallocated, ensuring a clean removal from the kernel without leaving orphaned resources.

```c
/**
 *  Function to exit the device driver.
 * It is responsible for releasing all system resources allocated during init.
 */
static void __exit irqModule_exit(void) {
    int i;
    pr_alert("%s: called\n", __FUNCTION__);

    /* ..........................................................................
       Iterates through all sensors to release the IRQ
       lines. The dev_id pointer must match the one used during request_irq.
       .......................................................................... */
    for (i = 0; i < NUM_IRQS; i++) {

        // Release the IRQ line back to the system
        free_irq(irq_numbers[i], (void *)&my_sensors[i]);

        pr_alert("irqModule: IRQ %d released for sensor %s\n",
                 irq_numbers[i], my_sensors[i].name);

        // Reset the pin to Input mode (default state)
        SetGPIOFunction(s_pGpioRegisters, my_sensors[i].gpio_pin, 0);
    }

    /* ..........................................................................
      Unmapping virtual memory, deleting the
       cdev structure, and destroying device nodes from /dev/irqX.
       .......................................................................... */

    // Unmap the virtual memory allocated for GPIO registers
    if (s_pGpioRegisters) {
        iounmap(s_pGpioRegisters);
    }

    // Delete the character device structure
    cdev_del(&c_dev);

    // Destroy individual device nodes
    for (i = 0; i < NUM_IRQS; i++) {
        device_destroy(irqDevice_class, MKDEV(MAJOR(irqDevice_majorminor), i));
    }

    /* ..........................................................................
       Final step involves destroying the device class and
       releasing the major and minor number region.
       .......................................................................... */
```

```
    // Destroy the class and unregister the region
    class_destroy(irqDevice_class);
    unregister_chrdev_region(irqDevice_majorminor, NUM_IRQS);

    pr_alert("%s: Driver unloaded successfully.\n", __FUNCTION__);
}
```

8.4.5 Implemented system calls

The driver implements **open**, **write**, **ioctl**, and **close** operations, while the **read** function is not implemented. This design choice is based on the system's asynchronous nature: instead of requiring the application to constantly poll the driver for data, the driver proactively notifies the user-space via Real-Time Signals. The **write** function is used to toggle interrupt lines on or off, and the **ioctl** function registers the application's PID to enable this signal-based communication.

```
static struct file_operations irqDevice_fops = {
    .owner          = THIS_MODULE,
    .open           = irq_device_open,
    .release        = irq_device_close,
    .write          = irq_device_write,
    .read           = NULL,
    .unlocked_ioctl = irq_device_ioctl,
}
```

```
 /* ..................................................................
   Triggered when /dev/irqX is accessed. Stores the device index (minor)
   for use in subsequent write/ioctl calls.
  ..................................................................... */
static int irq_device_open(struct inode* p_inode, struct file *p_file) {
    int minor = iminor(p_inode); // Extract device minor number

    if (minor >= NUM_IRQS) return -ENODEV;

    // Store minor number in private_data for context tracking
```

```
    p_file->private_data = (void *)(unsigned long)minor;

    pr_alert("irqModule: Device irq%d opened.\n", minor);
    return 0;
}
```

```
/*
.....
.....
   Acts as a control switch. Writing '0' disables the interrupt line,
   and '1' enables it, allowing the app to "mute" specific sensors.

.....
..... */
static ssize_t irq_device_write(struct file *pfile, const char __user
*pbuff, size_t len, loff_t *off) {
    char command;
    unsigned long minor = (unsigned long)pfile->private_data;
    struct SensorConfig *sensor = &my_sensors[minor];

    if (copy_from_user(&command, pbuff, 1)) return -EFAULT;

    if (command == '0') {
        if (sensor->enabled == true) {
            disable_irq(irq_numbers[minor]); // Mask the interrupt
            sensor->enabled = false;
        }
    }
    else if (command == '1') {
        if (sensor->enabled == false) {
            enable_irq(irq_numbers[minor]); // Unmask the interrupt
            sensor->enabled = true;
        }
    }
    else {
        return -EINVAL;
    }
    return len;
}
```

```
/*
.....

   Custom command to register the PID of the monitoring process. This
   tells the driver exactly where to send Real-Time Signals.

..... */
static long irq_device_ioctl(struct file *file, unsigned int cmd,
unsigned long arg) {
    if (cmd == REGIST_PID) {
        task = get_current(); // Link the driver to the caller's task
structure
        pr_alert("irqModule: App registered! PID = %d\n", task->pid);
    }
    else {
        return -EINVAL;
    }
    return 0;
}
```

```
/*
.....
   Safety cleanup. Ensures the driver stops sending signals to a process
   that is no longer active.

..... */
static int irq_device_close(struct inode *p_inode, struct file * pfile)
{
    if (task != NULL && task == get_current()) {
        task = NULL; // Prevent signals from being sent to dead
processes
    }
```

```
    return 0;
}
```

8.5 Startup and Shutdown Process

This section explains how the system is started and stopped at runtime. The project uses a **supervisor + daemons** architecture: a single **launcher** process creates and owns the shared IPC resources (POSIX message queues), starts three daemon processes, checks that each daemon is actually ready, and later shuts everything down in a controlled way.

Two rules guide the implementation:

- **Centralized IPC ownership:** only the launcher creates and unlinks POSIX message queues; daemons only open queues that already exist.
- **Explicit readiness:** the launcher does not assume a daemon is ready after fork/exec; it only proceeds after the daemon confirms that its initialization finished successfully.

8.5.1 Launcher Responsibilities

The launcher (main.cpp) is the component responsible for managing the full lifecycle of the system. Its responsibilities are:

- **Create IPC resources:** create all POSIX message queues needed by the system and keep them available for the entire runtime.
- **Start daemons in order:** launch each daemon in the correct sequence and wait until it reports readiness.
- **Store daemon PIDs:** obtain the **real** PID of each daemon process (after daemonization) so the launcher can later terminate it reliably.
- **Handle termination signals:** react to SIGINT and SIGTERM and trigger the shutdown procedure.
- **Cleanup:** unlink message queues and delete PID files at the end.

With this design, daemons are simpler and safer: they do not attempt to create or remove message queues, so they cannot leave IPC objects behind or delete resources needed by other components.

8.5.2 Signal Handling in the Launcher

At startup, the launcher registers a signal handler for SIGINT and SIGTERM. The handler sets a global flag (g_stop). During normal operation the launcher blocks in pause(), waking up only when a signal is delivered.

This behavior has two practical effects:

- The launcher consumes almost no CPU while the system is running.
- Shutdown starts immediately after a termination signal, without needing periodic polling.

```
static volatile sig_atomic_t g_stop = 0;

static void signalHandler(int signum) {
   std::cout << "\n[Wrapper] Sinal " << signum << " recebido. A encerrar..." <<
std::endl;
   g_stop = 1;
}
// on main

while (!g_stop) {
        pause();
    }
```

8.5.3 IPC Initialization: POSIX Message Queue Creation

The launcher creates the required POSIX message queues before starting any daemon. This is done using the C_Mqueue wrapper with createNew=true. Queue creation is treated as a strict prerequisite: if any queue cannot be created, the launcher terminates and the system does not start.

The message queues created are:

- /mq_to_db (message type: DatabaseMsg)
- /mq_to_actuator (message type: ActuatorCmd)
- /mq_rfid_in (message type: AuthResponse)
- /mq_rfid_out (message type: AuthResponse)
- /mq_move (message type: AuthResponse)
- /mq_finger (message type: AuthResponse)
- /mq_db_to_env (message type: AuthResponse)
- /mq_db_to_web (message type: DbWebResponse)

All created C_Mqueue objects are stored in a container owned by the launcher. Keeping these handles allows the launcher to unregister (unlink) all queues during shutdown in a single place.

```
std::vector<std::unique_ptr<C_Mqueue>> mqs;
try {
   mqs.push_back(std::make_unique<C_Mqueue>("/mq_to_db", sizeof(DatabaseMsg), 20, true));
   mqs.push_back(std::make_unique<C_Mqueue>("/mq_to_actuator", sizeof(ActuatorCmd), 20,
true));
   mqs.push_back(std::make_unique<C_Mqueue>("/mq_rfid_in", sizeof(AuthResponse), 10,
true));
   mqs.push_back(std::make_unique<C_Mqueue>("/mq_rfid_out", sizeof(AuthResponse), 10,
true));
   mqs.push_back(std::make_unique<C_Mqueue>("/mq_move", sizeof(AuthResponse), 10, true));
   mqs.push_back(std::make_unique<C_Mqueue>("/mq_finger", sizeof(AuthResponse), 10,
true));
   mqs.push_back(std::make_unique<C_Mqueue>("/mq_db_to_env", sizeof(AuthResponse), 10,
true));
   mqs.push_back(std::make_unique<C_Mqueue>("/mq_db_to_web", sizeof(DbWebResponse), 10,
true));
   std::cout << "[Wrapper] Filas criadas com sucesso" << std::endl;
} catch (const std::exception& e) {
   std::cerr << "[Wrapper] ERRO ao criar filas: " << e.what() << std::endl;
   return EXIT_FAILURE;
}
```

8.5.4 Daemon Startup with a Readiness Handshake

Daemons are started through the launchDaemon(binName, binPath) function. This function does more than execute a binary: it also verifies that the daemon completed initialization successfully.

This verification is important because each daemon:

- daemonizes (double-fork), which changes the PID that the launcher initially created,
- opens existing message queues,
- initializes its own subsystem (database, web server, or core threads).

A daemon should only be considered ready after all of these steps are complete.

8.5.4.1 Handshake Mechanism

The launcher uses UNIX domain socketpair(AF_UNIX, SOCK_STREAM) to create private communication channels between the launcher and each daemon. Two independent channels are used: one for startup readiness and one for shutdown confirmation.

Startup channel (NOTIFY_FD).

During startup, the launcher creates a socket pair sv_startup[2]:

- sv_startup[0] remains in the launcher (read end).
- sv_startup[1] is passed to the daemon (write end).

The daemon receives the file descriptor number through the NOTIFY_FD environment variable. To ensure that the descriptor remains open across exec(), the launcher clears the FD_CLOEXEC flag on sv_startup[1]. Without this step, exec() would close the descriptor and the daemon would not be able to report readiness.

The startup sequence is:

1. Create sv_startup with socketpair().
2. Clear FD_CLOEXEC on sv_startup[1].
3. Export NOTIFY_FD=sv_startup[1] into the environment.
4. fork().
5. Child: closes sv_startup[0] and executes the daemon binary using exec().
6. Launcher: closes sv_startup[1], removes NOTIFY_FD from its environment, and calls waitpid(child, ...) to wait for the initial child process to terminate (the intermediate process that exits as part of the daemonization flow).
7. The launcher waits up to 5 seconds for readability on sv_startup[0] using poll().
8. The daemon writes either:
    - a positive PID (the final daemon PID, after daemonization) if initialization succeeded; or
    - -1 if initialization failed.
9. The launcher reads the PID from sv_startup[0] and proceeds only if it is valid.

waitpid() is used only to wait for the initial child created by the launcher to terminate, which avoids leaving intermediate child processes and aligns the launcher with the daemonization sequence. Readiness itself is confirmed exclusively by the poll() + NOTIFY_FD handshake, because the daemon writes to NOTIFY_FD only after completing subsystem initialization.

A bounded timeout is used so the launcher cannot block indefinitely if a daemon crashes, deadlocks, or never sends a readiness message.

Shutdown channel (SHUTDOWN_FD).

In addition to the startup channel, the launcher creates a second socket pair sv_shutdown[2] used only during termination:

- sv_shutdown[0] remains in the launcher and is stored for later shutdown.
- sv_shutdown[1] is passed to the daemon through the SHUTDOWN_FD environment variable (also with FD_CLOEXEC cleared).

The daemon keeps this descriptor open during runtime. When a termination signal is handled and cleanup finishes, the daemon writes an acknowledgement (e.g., "OK\n") and closes SHUTDOWN_FD. During shutdown, the launcher sends SIGTERM and blocks on poll() for sv_shutdown[0] (with a bounded timeout), treating either an explicit acknowledgement or an end-of-file (POLLHUP/EOF) as confirmation that the daemon has terminated. If no confirmation is received before the timeout, the launcher escalates to SIGKILL.

This mechanism removes the need for time-based polling (e.g., repeated kill(pid, 0) checks with sleeps) and provides an explicit termination confirmation path for daemons that are not direct children of the launcher due to double-fork daemonization.

```
static bool clearCloExec(int fd) {
    int flags = fcntl(fd, F_GETFD);
    if (flags < 0) return false;
    return (fcntl(fd, F_SETFD, flags & ~FD_CLOEXEC) == 0);
}

struct DaemonInfo {
    pid_t pid = -1;
    int shutdown_fd = -1;
    std::string name;
};

static DaemonInfo launchDaemon(const std::string& binName, const std::string& binPath) {
    DaemonInfo info;
    info.name = binName;

    int sv_startup[2];
    int sv_shutdown[2];

    if (socketpair(AF_UNIX, SOCK_STREAM, 0, sv_startup) < 0) {
        std::perror("socketpair startup");
        return info;
```

```
    }
    if (socketpair(AF_UNIX, SOCK_STREAM, 0, sv_shutdown) < 0) {
        std::perror("socketpair shutdown");
        close(sv_startup[0]);
        close(sv_startup[1]);
        return info;
    }

    // Daemon ends must survive exec()
    if (!clearCloExec(sv_startup[1]) || !clearCloExec(sv_shutdown[1])) {
        std::perror("fcntl FD_CLOEXEC");
        close(sv_startup[0]); close(sv_startup[1]);
        close(sv_shutdown[0]); close(sv_shutdown[1]);
        return info;
    }

    char fdStartupStr[16], fdShutdownStr[16];
    std::snprintf(fdStartupStr, sizeof(fdStartupStr), "%d", sv_startup[1]);
    std::snprintf(fdShutdownStr, sizeof(fdShutdownStr), "%d", sv_shutdown[1]);
    setenv("NOTIFY_FD", fdStartupStr, 1);
    setenv("SHUTDOWN_FD", fdShutdownStr, 1);

    pid_t child = fork();
    if (child < 0) {
        std::perror("fork");
        close(sv_startup[0]); close(sv_startup[1]);
        close(sv_shutdown[0]); close(sv_shutdown[1]);
        unsetenv("NOTIFY_FD");
        unsetenv("SHUTDOWN_FD");
        return info;
    }

    if (child == 0) {
        // Child: keep write ends, close read ends
        close(sv_startup[0]);
        close(sv_shutdown[0]);
        execl(binPath.c_str(), binName.c_str(), nullptr);
        std::perror("execl");
        std::_Exit(EXIT_FAILURE);
    }

    // Wrapper: close write ends, keep read ends
    close(sv_startup[1]);
    close(sv_shutdown[1]);
    unsetenv("NOTIFY_FD");
    unsetenv("SHUTDOWN_FD");
```

```
    // Wait intermediate child (exits after daemonize first fork)
    waitpid(child, nullptr, 0);

    // Wait PID from daemon (startup handshake), timeout 5s
    pollfd pfd{};
    pfd.fd = sv_startup[0];
    pfd.events = POLLIN | POLLHUP;

    int ret = poll(&pfd, 1, 5000);
    if (ret > 0 && (pfd.revents & (POLLIN | POLLHUP))) {
        info.pid = readPidFromFd(sv_startup[0]);
    } else {
        std::cerr << "[Wrapper] ERROR: startup timeout for " << binName << std::endl;
    }
    close(sv_startup[0]);

    if (info.pid <= 0) {
        // startup failed: close shutdown channel too
        close(sv_shutdown[0]);
        info.shutdown_fd = -1;
        info.pid = -1;
        return info;
    }

    // Keep shutdown read end for later
    info.shutdown_fd = sv_shutdown[0];
    return info;
}
```

8.5.4.2 Startup Order

Daemons are started in the following order:

1. dDatabase
2. dWebServer
3. SecureAssetCore

This order ensures that services which are likely dependencies (database and web interface) are available before starting the core subsystem.

8.5.4.3 Failure Handling During Startup

If a daemon does not report readiness (timeout) or reports failure (-1), the launcher stops the daemons that were already started by invoking the same bounded shutdown procedure (stopDaemon), which starts with SIGTERM and waits for shutdown confirmation and then exits with a failure status. This prevents the system from running in a partial configuration.

The failure model is therefore:

- **Queue creation fails:** abort startup immediately.
- **Daemon readiness fails:** abort startup and terminate previously started daemons.

8.5.5 Runtime Waiting State

After all daemons have reported readiness and their PIDs are stored, the launcher enters the operational waiting state.

The runtime waiting loop is:

- while (!g_stop) pause();

The launcher does not process system workload; it supervises and waits for termination signals. This keeps overhead low and centralizes the shutdown trigger.

8.5.6 Controlled Shutdown Sequence

When the launcher receives SIGINT or SIGTERM, it performs shutdown in a strict order:

1. SecureAssetCore
2. dWebServer
3. dDatabase

This ordering reduces dependency issues during termination. Stopping the core first prevents it from generating new requests during shutdown. Stopping the web server next stops external traffic. Stopping the database last increases the likelihood that requests already issued have time to complete.

8.5.6.1 stopDaemon Procedure

For each daemon, the launcher performs a bounded termination procedure that avoids time-based polling and does not rely on waitpid() for the final daemon PID (since daemons use double-fork daemonization and are not direct children of the launcher).

The shutdown procedure is:

- Send SIGTERM to request a graceful shutdown.

- Wait up to 5 seconds using poll() on the daemon's dedicated shutdown handshake descriptor (shutdown_fd, passed to the daemon via SHUTDOWN_FD).
    - The daemon confirms completion by writing an acknowledgement (e.g., "OK\n") and closing the descriptor.
    - The launcher also treats an end-of-file condition (POLLHUP / EOF) as confirmation that the daemon has terminated.
- Escalate to SIGKILL if no acknowledgement or EOF is received before the timeout.

This procedure provides a clean shutdown attempt first, while still enforcing an upper bound on shutdown time. It also removes the need for periodic polling with kill(pid, 0) and avoids incorrect waitpid() usage on daemon PIDs that are not children of the launcher.

```
static void stopDaemon(DaemonInfo& daemon) {
   if (daemon.pid <= 0) return;

   std::cout << "[Wrapper] Stopping " << daemon.name << " (PID " << daemon.pid <<
")..." << std::endl;

   // Request graceful termination
   if (kill(daemon.pid, SIGTERM) != 0 && errno == ESRCH) {
       if (daemon.shutdown_fd >= 0) { close(daemon.shutdown_fd);
daemon.shutdown_fd = -1; }
       return;
   }

   // Wait for ACK or EOF (POLLHUP) on shutdown_fd, no sleeps
   if (daemon.shutdown_fd >= 0) {
       pollfd pfd{};
       pfd.fd = daemon.shutdown_fd;
       pfd.events = POLLIN | POLLHUP;

       int ret = poll(&pfd, 1, 5000);
       if (ret > 0 && (pfd.revents & (POLLIN | POLLHUP))) {
           // Accept either explicit "OK" or just EOF
           char buf[16];
           (void)read(daemon.shutdown_fd, buf, sizeof(buf));
           close(daemon.shutdown_fd);
           daemon.shutdown_fd = -1;
           std::cout << "[Wrapper] " << daemon.name << " stopped (ACK/EOF)." <<
std::endl;
```

```
            return;
        }

        // Timeout or error
        close(daemon.shutdown_fd);
        daemon.shutdown_fd = -1;
    }

    std::cout << "[Wrapper] " << daemon.name << " did not respond. Forcing
SIGKILL." << std::endl;
    kill(daemon.pid, SIGKILL);
}
```

8.5.7 Resource Cleanup

After all daemons are terminated, the launcher cleans all runtime artifacts created during startup:

- **Unlink message queues:** call C_Mqueue::unregister() for each queue created by the launcher.
- **Remove PID files:** delete the PID files created by the daemons:
    - /var/run/SecureAssetCore.pid
    - /var/run/dWebServer.pid
    - /var/run/dDatabase.pid

Removing these resources avoids conflicts on the next run (for example, queue names that already exist or stale PID files).

8.5.8 Summary of Lifecycle Behavior

The implemented lifecycle management provides:

- Deterministic startup using a readiness handshake with a bounded timeout.
- Clear IPC ownership: message queues are created and removed only by the launcher.
- Correct PID handling: the launcher stores the final daemon PIDs after daemonization.
- Controlled shutdown: fixed termination order and bounded waiting time (graceful then forced).
- Complete cleanup: message queues and PID files are removed to support clean restarts.

8.6 Daemons

This section explains how the three daemon processes are implemented: dDatabase, dWebServer, and SecureAssetCore. Each daemon is built as a separate executable. The launcher starts these executables, and each one then detaches from the terminal and runs in the background as a daemon.

All three daemons are written using the same overall pattern:

- daemonize using a double-fork and setsid();
- open the POSIX message queues that the launcher already created;
- initialize the subsystem (database, web server, or core);
- confirm readiness to the launcher through NOTIFY_FD;
- run until a termination signal is received;
- shut down cleanly and remove the daemon's PID file.

The daemons are **not** responsible for IPC ownership. In the code, this is enforced by opening message queues with createNew=false and never calling unlink/unregister operations. Only the launcher creates and removes the queues.

8.6.1 Common Daemon Structure

Even though the subsystems are different, the daemons follow the same lifecycle pattern. The steps below describe the common structure implemented by each daemon:

- Read handshake descriptors (NOTIFY_FD, SHUTDOWN_FD): The daemon reads the environment variables NOTIFY_FD and SHUTDOWN_FD and converts them into integer file descriptors. NOTIFY_FD is used only for the startup readiness handshake. SHUTDOWN_FD is reserved for shutdown confirmation and remains open during runtime.
- Daemonize: The process performs double-fork daemonization, calls setsid(), and detaches from the controlling terminal.
- Configure runtime environment: The daemon redirects stdin to /dev/null and redirects stdout/stderr to a logfile. It also writes its PID file under /var/run/.
- Remove startup variables from the environment: After daemonization, the daemon removes NOTIFY_FD and SHUTDOWN_FD from the environment using unsetenv(). This prevents startup parameters from persisting in the long-running runtime environment.
- Open message queues: The daemon opens only the queues it requires, using createNew=false (queues are created by the launcher).

- Initialize subsystem: The daemon initializes its subsystem resources (e.g., opening the database and ensuring schema availability, starting the web server resources, or starting core threads).
- Notify readiness: The daemon writes a short textual message to NOTIFY_FD:
    - its PID (after daemonization) on success; or
    - -1 on failure.

      After writing, the daemon closes NOTIFY_FD, making the readiness handshake bounded in time.
- Install signal handlers: The daemon registers handlers for SIGTERM and SIGINT.
- Run: The daemon executes its main loop (or blocks in its service loop) until a shutdown condition occurs.
- Shutdown and cleanup: The daemon stops its internal resources, removes its PID file, and does not unlink message queues. After completing cleanup, the daemon writes a shutdown acknowledgement (e.g., "OK\n") to SHUTDOWN_FD and closes the descriptor. This acknowledgement is used by the launcher to confirm orderly termination without time-based polling.

This pattern ensures the launcher can reliably detect when a daemon is ready and can also confirm when it has completed shutdown, while each daemon remains focused on its own functionality.

```
static void sendShutdownAck() {
   if (g_shutdown_fd >= 0) {
       const char* ack = "OK\n";
       (void)write(g_shutdown_fd, ack, 3);
       close(g_shutdown_fd);
       g_shutdown_fd = -1;
   }
}
```

8.6.1.1 Readiness Notification via NOTIFY_FD

The readiness handshake is implemented using the file descriptor passed by the launcher through NOTIFY_FD.

- If initialization succeeds, the daemon writes its PID (after daemonization).
- If initialization fails, the daemon writes -1.
- After writing the result, the daemon closes the file descriptor.

Closing NOTIFY_FD makes the startup handshake clearly bounded in time and prevents leaving a writer endpoint open. The practical result is that the launcher receives the PID of the final daemon process (not the intermediate forked process) and proceeds with startup only after the daemon has completed subsystem initialization

8.6.1.2 Daemonization, Logs, and PID Files

Each daemon performs a standard daemonization sequence:

- **First fork:** the parent exits.
- **Create a new session:** setsid().
- **Ignore hangup signals:** ignore SIGHUP.
- **Second fork:** the parent exits, preventing the daemon from acquiring a controlling terminal.
- **Permissions and working directory:** umask(0) and chdir("/").
- **Stream redirection:** redirect stdin to /dev/null; redirect stdout/stderr to a logfile.
- **PID file creation:** write the PID to /var/run/....

PID files:

- /var/run/dDatabase.pid
- /var/run/dWebServer.pid
- /var/run/SecureAssetCore.pid

Logfiles:

- /var/log/dDatabase.log
- /var/log/dWebServer.log
- /var/log/SecureAssetCore.log

The PID file is written after daemonization to ensure it contains the PID of the final background process. Each daemon removes its own PID file during shutdown. The launcher also removes PID files as a final cleanup step to handle abnormal termination scenarios.

```
static void daemonize(const char* pidfile, const char* logfile = nullptr) {
   pid_t pid = fork();
   if (pid < 0) std::exit(EXIT_FAILURE);
   if (pid > 0) std::exit(EXIT_SUCCESS);

   if (setsid() < 0) std::exit(EXIT_FAILURE);

   std::signal(SIGHUP, SIG_IGN);
```

```
    pid = fork();
    if (pid < 0) std::exit(EXIT_FAILURE);
    if (pid > 0) std::exit(EXIT_SUCCESS);

    umask(0);
    chdir("/");

    close(STDIN_FILENO);
    open("/dev/null", O_RDONLY);

    if (logfile) {
        int fd = open(logfile, O_WRONLY | O_CREAT | O_APPEND, 0644);
        if (fd >= 0) {
            dup2(fd, STDOUT_FILENO);
            dup2(fd, STDERR_FILENO);
            close(fd);
        } else {
            close(STDOUT_FILENO); close(STDERR_FILENO);
            open("/dev/null", O_WRONLY);
            open("/dev/null", O_WRONLY);
        }
    } else {
        close(STDOUT_FILENO); close(STDERR_FILENO);
        open("/dev/null", O_WRONLY);
        open("/dev/null", O_WRONLY);
    }

    int fd = open(pidfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd >= 0) {
        char buf[32];
        ssize_t n = std::snprintf(buf, sizeof(buf), "%d\n", getpid());
        (void)write(fd, buf, n);
        close(fd);
    }
}
```

8.6.1.3 Signal Handling

All daemons handle SIGTERM and SIGINT. The signal handlers are designed to stay simple:

- In dDatabase and SecureAssetCore, the handler sets a sig_atomic_t flag. The main loop checks this flag and exits cleanly.

- In dWebServer, the handler calls stop() on the server instance using a global pointer. This causes server.run() to return.

This approach avoids heavy work inside signal handlers and keeps shutdown logic in the normal execution path.

8.6.2 dDatabase Daemon

8.6.2.1 Purpose

dDatabase implements the database backend. It is responsible for opening the database file (secure_asset.db), ensuring the schema is created, and processing database requests received through message queues. It also sends responses to other components (web and core/environment).

8.6.2.2 IPC Queues

After daemonization, dDatabase opens the following existing queues:

- **Input requests:** /mq_to_db (DatabaseMsg)
- **Authentication/event related:** /mq_rfid_in, /mq_rfid_out, /mq_finger, /mq_move (AuthResponse)
- **Output responses:** /mq_db_to_web (DbWebResponse), /mq_db_to_env (AuthResponse)

The daemon assumes these queues already exist. If queue opening fails, the daemon treats initialization as failed.

8.6.2.3 Initialization and Readiness

The daemon initializes the database subsystem by executing:

- db.open()
- db.initializeSchema()

Readiness is reported only after these calls succeed. The daemon writes its PID to NOTIFY_FD. If any step fails, it writes -1 and exits.

This guarantees that when the launcher continues to start other daemons, the database is already open and the schema is available.

8.6.2.4 Runtime Loop

The daemon waits for messages on /mq_to_db and processes them. The implementation uses a timed receive (1 second timeout). This allows the daemon to periodically check if a shutdown was requested and prevents it from blocking forever.

When a message is received, it is passed to db.processDbMessage(msg) to execute the required database operation and publish any output messages.

8.6.2.5 Shutdown

When SIGTERM or SIGINT is received, the daemon exits the loop, closes the database (db.close()), and removes /var/run/dDatabase.pid. Message queues are not unlinked.

8.6.3 dWebServer Daemon

8.6.3.1 Purpose

dWebServer implements the HTTP/API interface (port 8080). It communicates with the database daemon through message queues. The web server does not access the database directly.

8.6.3.2 IPC Queues

dWebServer opens the existing queues:

- **Requests to database:** /mq_to_db (DatabaseMsg)
- **Responses from database:** /mq_db_to_web (DbWebResponse)

8.6.3.3 Initialization and Readiness

The daemon creates a server instance and calls server.start(). This call is treated as the readiness condition because it allocates and configures runtime resources needed to accept requests.

- If server.start() succeeds, the daemon writes its PID to NOTIFY_FD.
- If it fails, the daemon writes -1 and exits.

8.6.3.4 Runtime and Shutdown

After successful startup, the daemon runs server.run(). On SIGTERM/SIGINT, the handler calls server.stop() through the global pointer g_server. This causes the run loop to exit.
Finally, the daemon removes /var/run/dWebServer.pid. Message queues are not unlinked.

8.6.4 SecureAssetCore Daemon

8.6.4.1 Purpose

SecureAssetCore implements the core subsystem. It initializes the core through the singleton C_SecureAsset, which includes queue usage, hardware setup, and thread management. The daemon process stays alive while the functional workload is executed by internal threads.

8.6.4.2 Initialization and Readiness

After daemonization, the daemon obtains the singleton instance and performs:

- core->init()
- core->start()

Readiness is reported to the launcher only after these steps succeed. The daemon writes its PID to NOTIFY_FD. If initialization fails, it writes -1, destroys the singleton instance, and exits.

8.6.4.3 Runtime Loop

The daemon blocks waiting for signals using pause(). This keeps the process alive while core threads execute the workload and ensures termination is driven by signal delivery rather than periodic time-based wakeups.

8.6.4.4 Shutdown

On SIGTERM/SIGINT, the daemon stops the core subsystem in a controlled way:

- core->stop()
- core->waitForThreads()
- C_SecureAsset::destroyInstance()

It then removes /var/run/SecureAssetCore.pid. Message queues are not unlinked.

8.6.5 Summary

The daemon layer is implemented as three independent background services coordinated by the launcher. All daemons follow a consistent implementation pattern (daemonization, logging, PID files, readiness notification, and signal-based termination). IPC ownership remains centralized: daemons only open existing queues and never unlink shared resources. Each daemon reports readiness only after its subsystem initialization succeeds and performs local cleanup during shutdown.

8.7 Singleton core

C_SecureAsset was implemented as the central hardware core of the system. Its purpose is to aggregate the complete set of low-level interfaces (GPIO, UART, I2C and PWM) and expose a single orchestration point for the remaining modules, such as sensors, actuators, inter-thread communication queues, and execution threads. By concentrating

hardware ownership inside one core class, the initialization sequence becomes deterministic and the dependencies between modules are explicitly defined during construction.

```
C_SecureAsset::C_SecureAsset()

   : m_gpio_fingerprint_rst(PIN_FINGERPRINT_RST, OUT),
     m_gpio_yrm1001_enable(PIN_YRM1001_ENABLE, OUT),
     m_gpio_fan(PIN_FAN, OUT),
     m_gpio_alarm_led(PIN_ALARM_LED, OUT),
     m_gpio_alarm_buzzer(PIN_ALARM_BUZZER, OUT),


     m_uart_rfid_entry(UART_RFID_ENTRY),
     m_uart_rfid_exit(UART_RFID_EXIT),
     m_uart_fingerprint(UART_FINGERPRINT),
     m_uart_yrm1001(UART_YRM1001),


     m_i2c_temp_sensor(I2C_BUS, SHT30_ADDR),


     m_pwm_servo_room(PWM_CHIP, PWM_CHANNEL_SERVO_ROOM),
     m_pwm_servo_vault(PWM_CHIP, PWM_CHANNEL_SERVO_VAULT),


     m_temp_sensor(m_i2c_temp_sensor),
     m_rfid_entry(m_uart_rfid_entry),
     m_rfid_exit(m_uart_rfid_exit),
     m_rfid_inventory(m_uart_yrm1001, m_gpio_yrm1001_enable),
     m_fingerprint(m_uart_fingerprint, m_gpio_fingerprint_rst),


     m_servo_room(ID_SERVO_ROOM, m_pwm_servo_room),
     m_servo_vault(ID_SERVO_VAULT, m_pwm_servo_vault),
     m_fan(m_gpio_fan),
     m_alarm(m_gpio_alarm_led, m_gpio_alarm_buzzer),


     m_mq_to_database("/mq_to_db", sizeof(DatabaseMsg), 20, false),
     m_mq_to_actuator("/mq_to_actuator", sizeof(ActuatorCmd), 20, false),
     m_mq_to_verify_room("/mq_rfid_in", sizeof(AuthResponse), 10, false),
     m_mq_to_leave_room("/mq_rfid_out", sizeof(AuthResponse), 10, false),
     m_mq_to_check_movement("/mq_move", sizeof(AuthResponse), 10, false),
     m_mq_to_vault("/mq_finger", sizeof(AuthResponse), 10, false),
     m_mq_to_env_sensor("/mq_db_to_env", sizeof(AuthResponse), 10, false),
```

```
    m_monitor_reed_room(),
    m_monitor_reed_vault(),
    m_monitor_pir(),
    m_monitor_fingerprint(),
    m_monitor_rfid_entry(),
    m_monitor_rfid_exit()
{
   std::cout << "[SecureAsset] Construtor executado" << std::endl;


   m_actuators_list.fill(nullptr);
}
```

The class follows the Singleton design pattern, meaning that only one instance of C_SecureAsset exists during execution. This is achieved through a static pointer (s_instance) and a controlled access method (getInstance()), which creates the instance on the first call and returns the same instance afterwards. A dedicated destroyInstance() method releases the instance at shutdown, ensuring a controlled lifecycle.

```
C_SecureAsset* C_SecureAsset::getInstance() {
   if (s_instance == nullptr) {
       s_instance = new C_SecureAsset();
   }
   return s_instance;
}

void C_SecureAsset::destroyInstance() {
   if (s_instance == nullptr) {
       return;
   }
   delete s_instance;
   s_instance = nullptr;
}
```

To prevent accidental duplication, the constructor and destructor are declared private and copy/move operations are disabled. The following declarations:

```
C_SecureAsset(const C_SecureAsset&) = delete;
C_SecureAsset& operator=(const C_SecureAsset&) = delete;
```

```
C_SecureAsset(C_SecureAsset&&) = delete;
C_SecureAsset& operator=(C_SecureAsset&&) = delete;
```

explicitly forbid copying and moving the object. In practice, this prevents constructs such as C_SecureAsset a = *core; or returning/storing instances by value. Therefore, the instance can only be accessed through the singleton interface, enforcing the “single owner” model of the hardware resources.

Hardware dependency wiring inside the constructor

The C_SecureAsset constructor is responsible for wiring and instantiating the system’s hardware stack in a consistent order:

1. Low-level interfaces are created first (GPIO pins, UART channels, I2C bus and PWM channels).
2. Device drivers are built on top of these interfaces (RFID readers, fingerprint module, environmental sensor).
3. Actuators (servos, fan and alarm) are instantiated and linked to their underlying GPIO/PWM resources.
4. IPC mechanisms are prepared using message queues (C_Mqueue) to decouple producers and consumers.
5. Synchronization monitors (C_Monitor) are created for wake-up and coordination events between components.

This approach guarantees that each driver and actuator is constructed with its required dependencies (e.g., UART + control GPIO), and that the core object remains the single owner of these resources.

Initialization sequence

System startup is executed by init(), which structures initialization in well-defined stages:

- Sensor initialization (initSensors()), validating that each sensor driver is operational.
- Actuator initialization (initActuators()), ensuring each actuator can be controlled reliably.
- Actuator registry setup (initActuatorsList()), where m_actuators_list maps each actuator ID to its corresponding actuator object through a base pointer interface (C_Actuator*). This mapping enables generic actuator commands (e.g., “ID + value”) to be routed to the correct actuator instance.

- Thread creation (createThreads()), instantiating each subsystem thread with the required dependencies.

The separation into these stages improves maintainability and allows failure handling to be performed at the exact stage where a device initialization fails.
Thread ownership with std::unique_ptr

```
void C_SecureAsset::createThreads() {
   std::cout << "[SecureAsset] A criar threads..." << std::endl;


   m_thread_sighandler = std::make_unique<C_tSighandler>(
       m_monitor_reed_room,
       m_monitor_reed_vault,
       m_monitor_pir,
       m_monitor_fingerprint,
       m_monitor_rfid_entry,
       m_monitor_rfid_exit
   );
...
```

Each worker thread object is stored in a std::unique_ptr (e.g., m_thread_verify_room, m_thread_actuator, etc.). This design expresses exclusive ownership: the C_SecureAsset core is the only component responsible for the lifetime of the thread objects. Thread instances are created using std::make_unique<...>() during createThreads(), which provides clear semantics and avoids manual memory management. When the core shuts down, these pointers naturally ensure clean destruction of the thread objects after all threads are joined.
Common stop-and-exit behavior of threads
All worker threads follow a shared termination strategy based on:

- a periodic loop condition (e.g., while (!stopRequested()))
- timed waits (timedWait(...) / timedReceive(...)) to avoid blocking indefinitely
- a stop request mechanism triggered by the core during shutdown

A typical execution pattern is illustrated by C_tVerifyRoomAccess::run(). The thread continuously waits for an event (RFID activity signaled through a monitor), reads data from the sensor, sends a request to the database through a message queue, waits for an authorization response, and finally triggers actuator commands (e.g., door servo angle) based on the received response.

```
void C_tVerifyRoomAccess::run() {
   std::cout << "[VerifyRoomAccess] Thread iniciada. À espera de
tags..." << std::endl;


   while (!stopRequested()) {


       if (m_monitorrfid.timedWait(1)) {
           continue;
       }

       SensorData data = {};


       if (m_rfidEntry.read(&data)) {
           const char* rfidRead = data.data.rfid_single.tagID;
           std::cout << "[RFID entry] Cartão lido: " << rfidRead <<
std::endl;
(...)
```

The shutdown sequence is implemented by C_SecureAsset::stop(), which requests termination for each thread and then signals the involved monitors. This ensures that any thread blocked in a wait state is released, allowing it to re-check stopRequested() and exit gracefully. Finally, waitForThreads() joins all threads, ensuring that shutdown only proceeds after all subsystems have terminated properly.

```
void C_SecureAsset::stop() {
   if (m_thread_check_movement) m_thread_check_movement->requestStop();
   if (m_thread_env_sensor) m_thread_env_sensor->requestStop();
   if (m_thread_inventory) m_thread_inventory->requestStop();
   if (m_thread_verify_vault) m_thread_verify_vault->requestStop();
   if (m_thread_leave_room) m_thread_leave_room->requestStop();
   if (m_thread_verify_room) m_thread_verify_room->requestStop();
   if (m_thread_actuator) m_thread_actuator->requestStop();
   if (m_thread_sighandler) m_thread_sighandler->requestStop();

   AuthResponse stopMsg = {};
   stopMsg.command = DB_CMD_STOP_ENV_SENSOR;
   m_mq_to_env_sensor.send(&stopMsg, sizeof(stopMsg));
   m_monitor_reed_room.signal();
   m_monitor_reed_vault.signal();
   m_monitor_pir.signal();
```

```
    m_monitor_fingerprint.signal();
    m_monitor_rfid_entry.signal();
    m_monitor_rfid_exit.signal();
}

void C_SecureAsset::waitForThreads() {
    std::cout << "[SecureAsset] A aguardar término das threads..." <<
std::endl;

    if (m_thread_sighandler) m_thread_sighandler->join();
    if (m_thread_actuator) m_thread_actuator->join();
    if (m_thread_verify_room) m_thread_verify_room->join();
    if (m_thread_leave_room) m_thread_leave_room->join();
    if (m_thread_verify_vault) m_thread_verify_vault->join();
    if (m_thread_inventory) m_thread_inventory->join();
    if (m_thread_env_sensor) m_thread_env_sensor->join();
    if (m_thread_check_movement) m_thread_check_movement->join();

    std::cout << "[SecureAsset] Todas as threads terminadas" <<
std::endl;
}
```

8.8 IPC and Shared Structures

During the implementation of Secure Assent Guard, POSIX message queues were used to exchange data between the main process (drivers/threads) and the daemons (dDatabase and dWebServer). To keep message handling straightforward in the implementation, all message formats and shared data types were defined in a single header, SharedTypes.h. This ensures that every module compiles against the same struct layouts and enum values, avoiding mismatches when sending and receiving messages.

In the implementation, each message queue transports a specific C/C++ type (for example, the database request queue uses DatabaseMsg, the actuator command queue uses ActuatorCmd, and the database-to-web queue uses DbWebResponse). Because POSIX mqueues require a fixed message size per queue, the structures in SharedTypes.h were implemented using fixed-size fields and unions so that each message has a deterministic size.

8.8.1 Enumerations used by the implementation

To avoid magic numbers in the implementation and to simplify switch-based dispatch, SharedTypes.h defines the following enumerations:

- SensorID_enum assigns a unique ID to each sensor (SHT31, entry RFID, inventory RFID, fingerprint). This ID is used as a type tag in SensorData so consumer threads can interpret the payload correctly.
- ActuatorID_enum assigns IDs to each actuator (room servo, vault servo, fan, alarm). This ID is used in ActuatorCmd and actuator-related logs.
- LogType_enum standardizes log categories (sensor, actuator, access, system, alert, inventory), which are stored in the database through DatabaseLog.
- e_DbCommand enumerates all requests implemented by dDatabase (RFID authentication, log insertion, user/asset CRUD operations, settings updates, log filtering, and other system requests). dDatabase uses command to select the correct handler and to interpret the corresponding payload.
- ThreadPriority_enum: Implements standardized execution levels (PRIO_LOW, PRIO_MEDIUM, PRIO_HIGH) to ensure critical tasks, such as access verification, receive scheduling precedence.

```
enum e_DbCommand {
   DB_CMD_ENTER_ROOM_RFID,
   DB_CMD_LEAVE_ROOM_RFID,
   DB_CMD_UPDATE_ASSET,
   DB_CMD_USER_IN_PIR,
   DB_CMD_WRITE_LOG,
   DB_CMD_LOGIN,
   DB_CMD_GET_DASHBOARD,
   DB_CMD_GET_SENSORS,
   DB_CMD_GET_ACTUATORS,
   DB_CMD_ADD_USER,
   DB_CMD_DELETE_USER,
   DB_CMD_UPDATE_TEMP_THRESHOLD,
   DB_CMD_UPDATE_SAMPLING_TIME,
   DB_CMD_REGISTER_USER,
   DB_CMD_GET_USERS,
   DB_CMD_CREATE_USER,
   DB_CMD_MODIFY_USER,
   DB_CMD_REMOVE_USER,
   DB_CMD_GET_ASSETS,
   DB_CMD_CREATE_ASSET,
   DB_CMD_MODIFY_ASSET,
   DB_CMD_REMOVE_ASSET,
   DB_CMD_GET_SETTINGS,
   DB_CMD_GET_SETTINGS_THREAD,
   DB_CMD_UPDATE_SETTINGS,
   DB_CMD_FILTER_LOGS,
```

```
    DB_CMD_STOP_ENV_SENSOR
};
```

```
enum ThreadPriority_enum : int {
    PRIO_LOW        = 10,
    PRIO_MEDIUM   = 30,
    PRIO_HIGH      = 50
};
```

```
enum SensorID_enum : uint8_t {
    ID_SHT31 = 0,
    ID_RDM6300 = 1,
    ID_YRM1001 = 2,
    ID_FINGERPRINT = 3,
    ID_SENSOR_COUNT = 4
};
```

```
enum ActuatorID_enum : uint8_t {
    ID_SERVO_ROOM = 0,
    ID_SERVO_VAULT = 1,
    ID_FAN = 2,
    ID_ALARM_ACTUATOR = 3,
    ID_ACTUATOR_COUNT = 4
};
```

8.8.2 Sensor data messages used by threads

Each sensor driver produces a structured output using a dedicated fixed-size struct:

- Data_SHT31 stores temperature and humidity.
- Data_RFID_Single stores a single RFID tag ID.
- Data_RFID_Inventory stores an inventory scan (tag count + list of tags).
- Data_Fingerprint stores fingerprint authentication results.

Since only one of these formats is valid for a given reading, the implementation groups them in SensorData_Union. The SensorData wrapper adds a SensorID_enum type field. In practice, producing threads write type and fill the matching union member; consuming threads first check type and then read the corresponding payload. This avoids variable-length messages while still supporting different sensor outputs.

Actuator actions are transmitted using ActuatorCmd, which contains:

- actuatorID (ActuatorID_enum) identifying the actuator to control;
- value (uint8_t) encoding the target state/position.

In the implementation, multiple producer threads can send ActuatorCmd to the actuator queue, and the actuator thread consumes commands and applies them to hardware. Keeping this message compact and fixed-size reduces overhead and simplifies queue configuration.

```
struct Data_SHT31 {
   float temp;
   float hum;
};

struct Data_RFID_Single {
   char tagID[11];
};

struct Data_RFID_Inventory {
   int tagCount;
   char tagList[4][32];
};

struct Data_Fingerprint {
   bool authenticated;
   int userID;
};

union SensorData_Union {
   Data_SHT31 tempHum;
   Data_RFID_Single rfid_single;
   Data_RFID_Inventory rfid_inventory;
   Data_Fingerprint fingerprint;
};

struct SensorData {
   SensorID_enum type;
   SensorData_Union data;
};
```

```
enum ActuatorID_enum : uint8_t {
   ID_SERVO_ROOM = 0,
   ID_SERVO_VAULT = 1,
   ID_FAN = 2,
   ID_ALARM_ACTUATOR = 3,
```

```
    ID_ACTUATOR_COUNT = 4
};
```

8.8.3 Database request and response messages

All requests sent to dDatabase use DatabaseMsg. Each message contains:

- command (e_DbCommand) selecting the database operation;
- payload (union) carrying the parameters for that operation (e.g., RFID string, DatabaseLog, inventory data, login credentials, UserData, AssetData, SystemSettings, LogFilter, or userId).

The union-based payload ensures the message size is constant, while the command field determines which payload member is valid. In the implementation, the database daemon reads messages from its request queue and dispatches them with switch(command).

Database replies use two formats depending on the receiver:

- AuthResponse is used for responses to core threads (authorization results and, when applicable, settings). The command field mirrors the request so the receiver can correlate the response.
- DbWebResponse is used for responses to the web daemon. It contains a boolean success plus fixed-size buffers for JSON data and an error message.

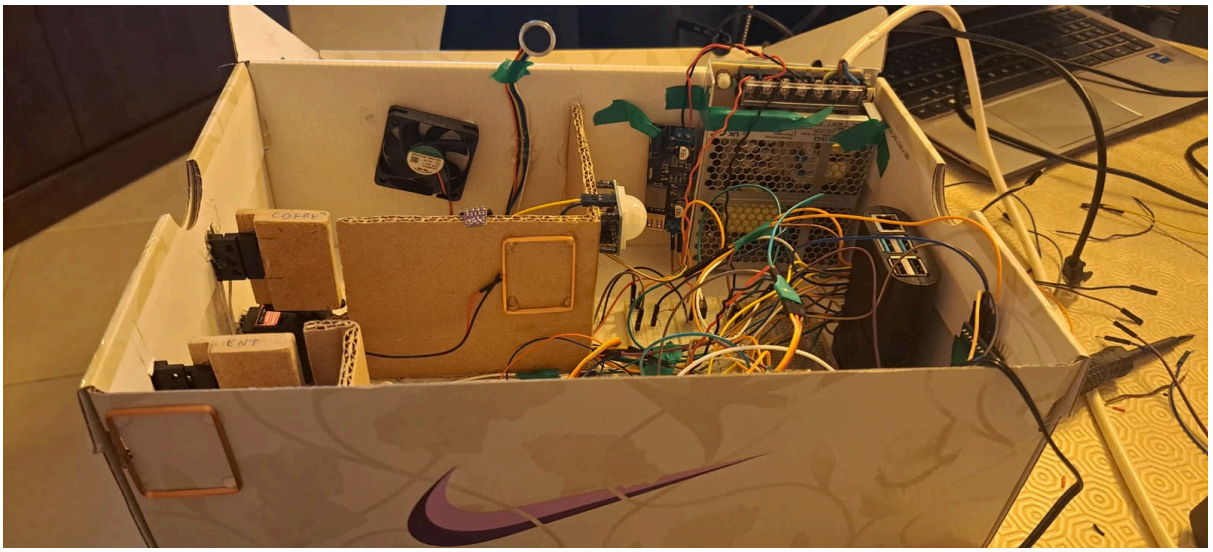
Implementation rationale (message size and parsing)

The implementation uses unions and fixed-length arrays to guarantee deterministic message sizes for mqueues. This avoids dynamic allocation in the IPC path and simplifies message parsing: receivers always read a fixed-size struct and then use a tag (type or command) to interpret the payload. This approach is aligned with the implemented control flow, where producer threads send requests and then block/wait for a response before issuing the next request.

```
struct DatabaseMsg {
    e_DbCommand command;
    union {
        char rfid[11];
        DatabaseLog log;
        Data_RFID_Inventory rfidInventory;
        LoginRequest login;
        UserData user;
```

```
        AssetData asset;
        SystemSettings settings;
        LogFilter logFilter;
        uint32_t userId;
    } payload;
};


struct AuthResponse {
    e_DbCommand command;

    union {
        struct {
            bool authorized;
            uint32_t userId;
            uint32_t accessLevel;
        } auth;

        SystemSettings settings;
    } payload;
};


struct DbWebResponse {
    bool success;
    char jsonData[8192];
    char errorMsg[256];
};
```

8.9 Mechanical structure



9. Verification

The Verification chapter serves as a comprehensive guide to the testing and validation procedures conducted during the development of the Secure Asset Guard. It outlines the testing strategy, execution procedures, and results, providing a thorough understanding of how the project's integrated hardware and software components were verified to meet the initial security and functional requirements. This phase is essential to ensure that the multi-threaded architecture, the database integrity, and the web interface operate as a cohesive and reliable security system.

9.1 Test Cases

In the Design chapter of this document, a set of test cases was defined to act as a rigorous method of verification. These cases were designed to simulate real-world scenarios—ranging from authorized access via RFID and Biometrics to intrusion detection and environmental alerts—ensuring that the system remains robust under different conditions. In this phase, the results of these test cases are presented, documenting the system's performance and confirming that the final implementation aligns with the project's safety and management objectives.

9.1.1 Power Supply Testing

Table 4. Power Supply Testing

Test case	Description	Expected Result	Real Result
Power Supply voltage	Apply mains power to the 12V 6A PSU. Measure the output voltage with a multimeter.	Output voltage is stable and within ±5% of 12V DC.	Correct 12V power supply output measured.
3.3V Converter Test	Measure DC-DC converter output voltage for sensors of 3.3V	Voltage stable at 3.3V±5%.	Correct 3.3V output measured.
5V Converter Test	Measure DC-DC converter output voltage for sensors of 5V	Voltage stable at 5V±5%	Correct 5V output measured.
Logic Level Comm (LLC)	Check if communication between 3.3V and 5V devices function via the converter.	Bidirectional communication established.	Correct bidirectional communication established.

9.1.2 Sensors

Table 5. Sensors testing.

Test case	Description	Expected Result	Real Result
Read Temp/Hum	Check if temperature and humidity values are obtained.	Valid temperature and humidity values.	Correct temperature and humidity values obtained.
Read RFID (125KHz - Entry)	Check if the tag ID is obtained correctly on the designated UART interface.	Tag ID read correctly.	Correct Tag ID obtained on the designated UART interface.
Read RFID (125KHz - Exit)	Check if the tag ID is obtained correctly on the designated UART interface.	Tag ID read correctly.	Correct Tag ID obtained on the designated UART interface.
Detect Motion (PIR)	Check if the PIR sensor detects motion.	Motion signal detected.	Correct motion detection signal received via interrupt.
Door Status (Reed Switch)	Check if the door's open/closed state is detected.	Door state read correctly.	Correct open and closed door states detected.
Read Fingerprint (Enroll)	Check if the fingerprint enrollment process functions.	Successful enrollment.	Correct fingerprint enrollment process performed.
Read Fingerprint (Verify)	Check if an enrolled fingerprint is recognized.	Successful verification.	Correct identification and verification of enrolled fingerprints.
Read RFID (UHF)	Check if the UHF tag ID is obtained correctly on the designated UART interface.	UHF tag ID read correctly.	Correct UHF tag IDs obtained on the designated UART interface.
Control RFID (UHF - Enable)	Check if the UHF module can be enabled/disabled via GPIO.	Module state control functions.	Correct module state control via GPIO established.

9.1.3 Actuators

Table 6 Actuators testing.

Test case	Description	Expected Result	Real Result
Actuate Servo (Room)	Check if the room door servo actuates correctly.	Servos move to correct positions. Hold against manual force.	Correct room door servo actuation to target positions verified.
Actuate Servo (Safe)	Check if the Safe door servo actuates correctly.	Servos move to correct positions. Hold against manual force.	Correct vault door servo actuation to target positions verified.

Actuate Fan	Verify ON/OFF control	Fan activates on command.	Correct fan control
Actuate Buzzer	Check if the buzzer emits sound when activated.	Buzzer emits sound.	Correct audible sound emission upon activation.
Actuate LED	Check if the LED turns on/off when activated/deactivated.	LED turns on and off.	Correct LED state transitions (ON/OFF) verified.

9.1.4 Database and GUI

Table 7. Database and GUI testing.

Test case	Description	Expected Result	Real Result
Database Connection	Test the system's ability to establish and terminate a connection to the database.	Connection successfully established and terminated.	Successful database connection and termination established via dDatabase.
SQLCipher Encryption	Test encrypted database with SQLCipher	Database encrypted with AES-256, requires key to open	Verified AES-256 encryption; database requires key for access.
Write Operations	Insert access logs, sensor data, and inventory records	Data written correctly with proper timestamps and foreign keys	Correct data insertion for logs, sensors, and inventory with timestamps.
Read Operations	Test if the user interface can request and display logs from the database.	The interface displays the event logs correctly.	Correct retrieval and display of logs on the web interface.
Update Operations	Modify user permissions and inventory status	Records updated correctly. Changes reflected immediately	Successful modification of user permissions and inventory status.
Web Communication (Interface)	Test the connection from the user interface to the Mongoose server on the Pi.	Web page loads correctly showing system status	Web page loaded correctly via Mongoose server.

Interface Authentication	Attempt to authenticate on the user interface with valid and invalid credentials.	Access is granted for valid credentials and denied for invalid ones.	Correct authentication logic for valid and invalid credentials verified.
Data Synchronization (UI)	Verify that an event generated by the hardware (e.g., intrusion) is reflected in the user interface.	The user interface displays the intrusion alert in near real-time.	Verified real-time update of hardware events on the interface.

9.1.5 Integration Tests

Table 9. Integration tests.

Test case	Description	Expected Result	Real Result
Room Access Authentication	Test room access using both a valid and an invalid 125KHz RFID tag on the entry reader.	Door opens for valid tag and access is logged. Door remains closed for invalid tag and access is logged as denied.	Correct access control and logging for both valid and invalid RFID tags.
Vault Access & Inventory	Open the vault using a valid fingerprint. Close the vault door (triggering the Reed switch).	Vault door opens and access is logged. Upon closing, the UHF inventory scan runs and the database is updated.	Correct fingerprint access and automatic UHF inventory scan upon vault closure.
Intrusion Detection System	With the room door closed (Reed switch),and no authorized person inside trigger the PIR motion sensor.	Alarms (Buzzer/LED) activate. Intrusion is logged in the database and sent to the GUI.	Successful alarm activation and logging during unauthorized motion detection.
Environmental Control System	Increase the temperature near the SHT31 sensor above the defined threshold.	The Fan is activated automatically. The environmental alert is logged.	Correct automatic fan activation upon reaching the temperature threshold.
Remote User Management	Use the remote interface to add/remove a new user's RFID tag and fingerprint.	The new user is added, successfully authenticate at the room and vault.	Successful remote user management with immediate hardware synchronization.

10. Bibliography

- https://safelockerdealer.com/?product=godrej-nx-pro-luxe-digital-biometric-48ltr-home-safe-locker-10x-stronger-than-a-wooden-cupboard
- https://securastock.com/secura-crib/
- https://securamsys.com/products/safe-monitor
- https://www.marketsandmarkets.com/Market-Reports/access-control-market-164562182.html
- https://www.marketsandmarkets.com/Market-Reports/physical-security-market-1014.html
- https://www.marketsandmarkets.com/Market-Reports/home-security-solutions-market-701.html
- https://www.botnroll.com/pt/rfid-nfc/2379-modulo-rfid-125khz-rdm6300-.html
- https://www.botnroll.com/pt/bussolas/4136-sensor-magn-tico-man-alcance-11-16mm-ip68.html
- https://www.botnroll.com/pt/buzzers-sirenes/3885-buzzer-ativo-3-3-5v-2-3khz.html
- https://mauser.pt/095-5099/modulo-sensor-de-movimento-pir-compativel-com-arduino
- https://www.botnroll.com/pt/biometricos/4786-leitor-impress-o-digital-capacitivo-redondo-processador-cortex.html
- https://www.botnroll.com/pt/servos/3168-servo-mg996r-180-11kg-cm.html
- https://www.botnroll.com/pt/ventiladores/864-ventilador-12vdc-50mm.html
- https://mauser.pt/095-6684/modulo-sensor-de-temperatura-e-humidade-c-sht31-e-saida-i2c
- https://www.botnroll.com/pt/5v/3559-conversor-dc-dc-step-down-12v-para-3-3v-e-5v.html
- https://www.botnroll.com/pt/5v/2986-step-down-9-38v-para-usb-5v-5a.html
- https://www.botnroll.com/pt/acdc-12v/5464-fonte-de-alimenta-o-comutada-fechada-12vdc-6a-72w.html